Topic Title: Comparison and Implementation of Data Compression Methods for SoC-based Radar Systems

Student Name: Jordan Mohindra **Student ID:** z5418455

A. **Problem statement**

In automotive radar, high-resolution cascaded MIMO sensors produce more raw data than a vehicle's network and central processor can comfortably absorb. Compressing at the sensor saves link bandwidth, but the compressor must be lossless (anything it discards is lost before target detection), keep up with the sensor, and be small enough for sensor hardware. Radar compression methods are rarely compared on the same real multi-transmitter data, and the lossless baseline was evaluated on a single-transmitter radar with no decoder.

B. **Objective**

Compare compression methods side by side on real, public, multi-transmitter radar data; select the best lossless methods; and implement their encoders and decoders on an FPGA, verified and measured on a board.

C. **My solution**

Eight methods were run in MATLAB on 50 ColoRadar frames (Texas Instruments cascade, 12 transmitters, 16 receivers) through one chain: range FFT, fixed 16-bit (FX16) scaling, compression, Doppler FFT, non-coherent integration and peak detection, against a double-precision reference. The two lossless predictive coders, DRHE and a second-order slow-time LPC, predict from the same transmitter (12 ramps back) and share an S_4/Huffman coder whose dictionary was re-derived from five frames of their prediction errors. Both, with matching decoders and a range-FFT front end, were written in C++ for Vitis HLS, built with Vivado into a DMA test system on an AMD KCU116 board at 100 MHz, and verified bit-exact from MATLAB through C-simulation and RTL co-simulation to the board.

D. **Contributions** (at most one per line, most important first)

Bit-exact lossless radar compression on an FPGA: 4.22× (DRHE), 4.40× (LPC)
Same-transmitter prediction lag for time-division MIMO radar
Comparison of eight compression methods on real cascade radar data
Breakdown of the compression ratio; Huffman dictionaries matched to the errors
Encoders, decoders and range FFT measured on the FPGA board
Reconstruction of, and comparison with, the baseline design

E. **Suggestions for future work**

Fixed-point DRHE datapath to cut logic and DSP use
Streaming (backward-adaptive) LPC without a frame buffer
All sixteen receive channels and power measurement

While I may have benefited from discussion with other people, I certify that this report is entirely my own work, except where appropriately documented acknowledgements are included.

Signature: _____ Date: _____/_____/_____

Pointers

List relevant page numbers in the column on the left. Be precise and selective: Don't list all pages of your report!

12	Problem Statement
14	Objective

Theory (up to 5 most relevant ideas)

20	Time-division MIMO radar and the data cube
21	Lossless compression and entropy coding
24	Fixed-point (FX16) quantisation
25	FPGAs and high-level synthesis
27	The baseline: Kiem's DRHE design

Method of solution (up to 5 most relevant points)

31	Eight candidate methods
32	Shared S_4/Huffman entropy coder
35	Prediction on a TDM-MIMO radar
46	Verification ladder, MATLAB to board
49	Hardware implementation

Contributions (most important first)

67	Bit-exact lossless radar compression on an FPGA
35, 63	Same-transmitter prediction lag for TDM-MIMO radar
60	Comparison of eight compression methods
64, 65	Breakdown of the ratio; matched Huffman dictionaries
68, 71	Encoders, decoders and range FFT measured on the board
74, 84	Reconstruction of, and comparison with, the baseline

My work

38–42, 49–57	System block diagrams/algorithms/equations solved
45–46	Description of assessment criteria used
44, 46, 57	Description of procedure (e.g. for experiments)

Results

72, 60	Succinct presentation of results
64, 74, 93	Analysis
91	Significance of results

Conclusion

97	Statement of whether the outcomes met the objectives
99	Suggestions for future research

Literature: (up to 5 most important references)

27–30, 74–90	[11] L. Kiem, master's thesis, 2025 (the baseline design)
23, 14	[12] A. Kramer *et al.*, ColoRadar dataset, 2022
38, 31, 26	[26] L. Li, MSc thesis, 2014 (origin of DRHE)
87, 42, 28	[27] G. Meucci and F. Mancuso, LPC-Huffman, 2022
42, 22	[15] D. A. Huffman, minimum-redundancy codes, 1952



UNSW
AUSTRALIA

SCHOOL OF ELECTRICAL ENGINEERING
AND TELECOMMUNICATIONS

Comparison and Implementation of Data Compression Methods for SoC-based Radar Systems

by

Jordan Mohindra

Student ID: z5418455

Thesis submitted as a requirement for the degree

Bachelor of Engineering (Electrical Engineering)

Submitted: November 2026
Supervisor: A/Prof. Beena Ahmed

Abstract

High-resolution cascaded MIMO radar produces more raw data than a vehicle's network and central processor can comfortably absorb. Compressing that data at the sensor buys back bandwidth, but only if the compressor is lossless, fast and small, because anything it discards is lost before target detection ever sees it.

This thesis compares data compression methods for automotive radar on real data and implements the most suitable of them in hardware. Eight methods were evaluated in MATLAB on 50 frames of the public ColoRadar dataset, recorded with a twelve-transmitter Texas Instruments cascade radar. The lossy quantisers reached compression ratios of up to 3.76 but missed up to 28.7 % of targets, and floating-point storage barely compressed at all. Two lossless predictive coders compressed by more than three: Doppler Redundancy Hybrid Encoding (DRHE), which predicts magnitude and phase with fixed filters, and a second-order linear predictive coder (LPC) whose coefficients are fitted to each frame. On a time-division MIMO radar both must predict each ramp from the previous ramp of the *same transmitter*, twelve ramps back; predicting from the adjacent ramp does worse than not predicting at all.

The compression ratio was broken down into its sources. Huffman coding of the 16-bit range-FFT data alone achieves 3.48; prediction adds a further 0.22–0.27; and a Huffman dictionary re-derived for the prediction errors adds 0.47 for DRHE and 0.71 for LPC, giving 4.22 and 4.40 overall, measured on the hardware, with every frame reconstructed exactly.

Both compressors and their decompressors were written in C++, synthesised with Vitis HLS and Vivado, and run on an AMD KCU116 (Kintex UltraScale+) board at 100 MHz. On the board all 50 frames were compressed byte-identically to the simulation, and the board's own decoders reconstructed every frame exactly. With a range-FFT front end also on the FPGA, the board starts from raw ADC samples; its 16-bit codes differ from MATLAB's in 0.016 % of values, by one step, so the ratios are unchanged to four significant figures. DRHE compresses a frame in 0.61 ms (5.1 Gbit/s) and streams its output with 13 µs of latency. LPC takes 3.25 ms and, because its coefficients depend on the whole frame, cannot output anything until the whole frame has arrived: its last output word follows its last input word by 3.0 ms. The decoders each reconstruct one value per clock, about 2 ms per frame. Encoder and decoder together use about 23 % of the chip's logic for either algorithm. LPC compresses about 4 % better; DRHE is

recommended for a streaming sensor front end, because it needs no frame buffer and has a small fraction of LPC's latency.

Acknowledgements

I would like to thank A/Prof. Beena Ahmed for her guidance throughout this project, and Dr. Mostafa Shahin for his assessment and feedback. I am also indebted to Lukas Kiem, whose master's thesis provided the algorithmic baseline this work builds on; it was specified carefully enough to be reimplemented and tested exactly, which made much of this thesis possible.

Abbreviations

ADAS Advanced Driver Assistance Systems

ADC Analog-to-Digital Converter

AR Autoregressive

ASIC Application-Specific Integrated Circuit

AXI Advanced eXtensible Interface

BAQ Block Adaptive Quantization

BRAM Block Random Access Memory

CR Compression Ratio

C-sim C-Simulation (the HLS C++ run as software)

DDR Double Data Rate (memory)

DMA Direct Memory Access

DRHE Doppler Redundancy Hybrid Encoding

DSP Digital Signal Processing / DSP slice

EGE Exponential-Golomb Encoding

FDR False Discovery Rate

FF Flip-Flop

FFT Fast Fourier Transform

FMCW Frequency-Modulated Continuous Wave

FN False Negative

FP16 16-bit (half-precision) floating point

FPGA Field-Programmable Gate Array

FX16 16-bit fixed-point (integer) representation

HLS High-Level Synthesis

IF Intermediate Frequency

IIR Infinite Impulse Response

II Initiation Interval

I/Q In-phase / Quadrature

LPC Linear Predictive Coding

LSB Least Significant Bit

LUT Look-Up Table

MIMO Multiple-Input Multiple-Output

MSB Most Significant Bit

RDVLE Range-Dependent Variable-Length Encoding

RTL Register-Transfer Level

RX Receive / Receiver

SerDes Serialiser/Deserialiser

SNR Signal-to-Noise Ratio

SoC System-on-Chip

TDM Time-Division Multiplexing

TX Transmit / Transmitter

UDRE Uniform Dynamic-Range re-Encoding

WNS Worst Negative Slack

Contents

Acknowledgements 1

Abbreviations 2

Contents 4

List of Figures 9

List of Tables 10

1 Introduction 12

1.1 The problem . . . 12
1.2 Motivation . . . 12
1.3 Why the compressor must be lossless . . . 13
1.4 Aim and objectives . . . 14
1.5 Contributions . . . 15
1.6 Thesis outline . . . 16

2 Background 17

2.1 How an FMCW radar measures range . . . 17
2.1.1 The chirp . . . 17
2.1.2 From beat frequency to a range profile . . . 18
2.1.3 Slow time and Doppler . . . 19
2.2 Angle, MIMO and the virtual array . . . 19
2.2.1 Time-division multiplexing, and why it matters here . . . 20
2.3 The radar data cube, and why it is large . . . 21
2.4 Lossless compression: how it actually works . . . 21

2.4.1 The two categories . . . 21
2.4.2 The bound: entropy . . . 22
2.4.3 The standard two-stage structure . . . 22
2.4.4 Open-loop versus closed-loop prediction . . . 22
2.5 The sensor and the dataset . . . 23
2.5.1 AWR2243 and the MMWCAS-RF-EVM . . . 23
2.5.2 ColoRadar . . . 23
2.6 Quantisation: the FX16 step . . . 24
2.7 FPGAs and high-level synthesis . . . 25
2.8 Related work . . . 26

3 The Baseline: Kiem's Thesis **27**
3.1 His problem and his data . . . 27
3.2 His design space and result . . . 28
3.3 His final scheme . . . 29
3.4 His hardware . . . 29
3.5 What he left open . . . 29

4 Compression Methods **31**
4.1 The eight candidate methods . . . 31
4.2 The shared entropy coder . . . 32
4.2.1 Residuals . . . 32
4.2.2 The category S_4 and the APPEND bits . . . 33
4.2.3 Cost, and the shape of the dictionary . . . 34
4.3 Predicting on a time-division MIMO radar . . . 35
4.3.1 Where the bits go over a whole frame . . . 37
4.3.2 Why not one model per transmitter? . . . 37
4.4 DRHE: Doppler Redundancy Hybrid Encoding . . . 38
4.4.1 Why predict in polar coordinates . . . 38
4.4.2 The two predictors . . . 38
4.4.3 The streaming property . . . 39
4.5 LPC: linear prediction with Yule–Walker coefficients . . . 40
4.5.1 Autoregressive modelling . . . 40

4.5.2 Stability . . . 40
4.5.3 Granularity and side information . . . 41
4.5.4 The cost: LPC is not causal . . . 41
4.5.5 Coefficient quantisation . . . 41
4.5.6 What this LPC is not . . . 42
4.6 The Huffman dictionary . . . 42
4.7 The two algorithms side by side . . . 43

5 Evaluation Method **44**
5.1 Two stages . . . 44
5.2 The reference signal chain . . . 44
5.3 Metrics . . . 45
5.4 The verification ladder . . . 46
5.5 Edge-case testing . . . 47
5.6 An independent implementation . . . 48
5.7 Tools . . . 48

6 Hardware Implementation **49**
6.1 Common interface and bit packing . . . 49
6.2 DRHE compressor . . . 50
6.2.1 State and parallelism . . . 50
6.2.2 Where the logic goes . . . 50
6.2.3 Initiation interval . . . 51
6.3 LPC compressor . . . 51
6.3.1 Three passes . . . 51
6.3.2 Reaching back n_{Tx} and $2n_{\mathrm{Tx}}$ ramps . . . 51
6.3.3 Where the cycles go . . . 52
6.3.4 Fixed-point choices . . . 52
6.3.5 An asymmetry that favours the decoder . . . 53
6.4 Exactly lossless on any input . . . 53
6.4.1 The one value the code cannot represent . . . 53
6.4.2 Wide-integer conversion in the coefficient solver . . . 54
6.5 Decompressors . . . 54

6.5.1 Why decoding is the harder half . . . 55
6.5.2 DRHE: a two-stage serial decoder . . . 55
6.5.3 LPC: the same two stages . . . 56
6.6 Range FFT on chip . . . 56
6.7 The board test system . . . 57
6.8 Building and timing closure . . . 59

7 Results **60**
7.1 Stage 1: screening eight methods . . . 60
7.1.1 The cost of the quantisation step . . . 62
7.2 Stage 2: the prediction axis . . . 63
7.3 Where the compression comes from . . . 64
7.4 The re-derived dictionary . . . 65
7.4.1 How close to the limit . . . 66
7.5 Frame-to-frame variation . . . 66
7.6 Verification . . . 67
7.7 Speed and latency on the board . . . 68
7.8 Chip resources . . . 70
7.9 Starting from raw ADC samples . . . 71
7.10 The two algorithms side by side . . . 72

8 Interpreting the Hardware Numbers **74**
8.1 Estimates against routed results . . . 74
8.2 Rebuilding the reference design . . . 75
8.3 From this thesis's datapath to Kiem's, one change at a time . . . 76
8.4 Latency . . . 78
8.5 Throughput . . . 80
8.6 Resources against the device . . . 80
8.7 Every difference from Kiem, accounted for . . . 81

9 Comparison with the Baseline **84**
9.1 Rebuilding Kiem's simulation . . . 84
9.2 What sets the ratio on Kiem's scene . . . 85
9.3 The LPC he excluded . . . 87

9.4 Adding transmitters . . . 87
9.5 The DRHE algorithm, line by line . . . 88
9.6 Every result, compared . . . 89

10 Discussion **91**
10.1 Which algorithm for a sensor front end? . . . 91
10.2 Scaling to larger frames . . . 92
10.3 What the compression ratio measures . . . 93
10.4 How far the dictionary and the lag generalise . . . 94
10.5 Against the baseline . . . 94
10.6 Observations on verifying a compressor . . . 95
10.7 Threats to validity . . . 95

11 Conclusions and Future Work **97**
11.1 Summary . . . 97
11.2 Findings . . . 97
11.3 Future work . . . 99

Bibliography **100**

A The Huffman Dictionaries **103**

B Bitstream Layouts **105**
B.1 DRHE . . . 105
B.2 LPC . . . 105

C Reproducing the Results **106**
C.1 Source files . . . 106
C.2 Commands . . . 107

D Verification Record **108**

List of Figures

1.1 Raw versus compressed radar transport 13

2.1 FMCW chirp and beat frequency 18
2.2 TDM-MIMO ramp interleaving 20
2.3 Radar data cube and processing chain 21

4.1 DRHE block diagram 39

5.1 One input frame 45
5.2 The verification ladder 47

6.1 LPC three-pass architecture 52
6.2 The KCU116 test system 58

7.1 Compression against missed targets 61
7.2 Compression ratio, step by step 64
7.3 Bits needed per value 65
7.4 Compression ratio of each frame 67
7.5 Time per frame on the FPGA 69
7.6 FPGA resources 71

9.1 CR against noise floor on Kiem's scene 86

10.1 Where the bits go 93

List of Tables

2.1 ColoRadar cascade capture parameters used throughout this thesis. 24

3.1 Kiem's design-space results 28
3.2 Kiem's hardware results 30

4.1 Raw values against residuals 33
4.2 S4 category ranges 33
4.3 Sixteen samples traced through both predictors 36
4.4 Bit budget by category 37
4.5 DRHE and LPC compared 43

5.1 Synthetic edge cases 48

6.1 The six board builds 59

7.1 Stage 1: the eight methods 60
7.2 Quantisation study 62
7.3 The prediction lag 63
7.4 Where the compression comes from 64
7.5 What the re-derived dictionary changes 65
7.6 How close the coder gets to the entropy 66
7.7 Measured timing on the board 68
7.8 Resources per core 70
7.9 Measured with the range FFT on chip 71
7.10 Detection loss with the range FFT on chip 72
7.11 DRHE and LPC compared 73

8.1 HLS estimates against implemented results 75

8.2 The replica against Kiem's published design 76
8.3 From DRHE-1 to the replica, one change per step 77
8.4 Throughput: measured, theoretical and required 80
8.5 Resources as a fraction of two devices 81
8.6 Every difference from Kiem's design 82

9.1 Coders on Kiem's rebuilt scene 85
9.2 DRHE audit against Kiem's specification 88

A.1 The original Huffman dictionary 103
A.2 The re-derived dictionaries 104

D.1 Verification record 108

Chapter 1

Introduction

1.1 The problem

A modern high-resolution automotive radar generates data faster than the vehicle it is mounted on can conveniently move it. The cascaded radar used in this thesis produces about 3 MiB of raw digitised samples every frame, and it is one of several such sensors on a car. Getting that data to the central processor means either expensive shielded high-speed cabling or compressing it at the sensor first. Compression is the cheaper answer, but it has to keep up with the sensor, fit in a small amount of silicon, and — because this data feeds a system that decides whether to brake — lose nothing. This thesis compares the available compression methods on real radar data, selects the ones that meet those conditions, and builds and measures them on an FPGA.

1.2 Motivation

Radar earns its place in a vehicle's sensor suite because it works when the other sensors do not. Cameras fail in fog, glare and darkness; lidar is degraded by rain and airborne dust; radar sees through all of them [1, 2]. That robustness is why radar has been in production vehicles for two decades, initially for simple distance warning and now for adaptive cruise control, automatic emergency braking, blind-spot detection and lane-change assistance [3].

What has changed recently is resolution. Supporting SAE Level 2+ autonomy needs more than "something is 40 m ahead". It needs a map: how many objects, where, how fast, how wide, and what they probably are. Angular resolution is what separates "an object" from "a car and

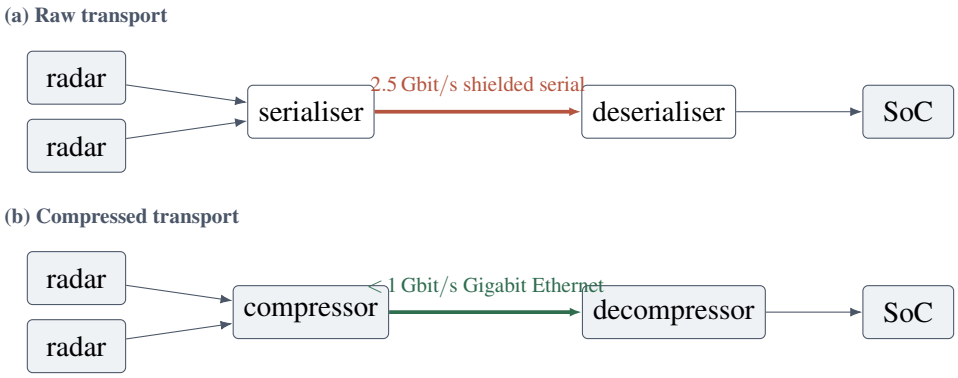


Figure 1.1: The economic argument for sensor-edge compression. Compressing by a factor of 2.5 or better at the radar module replaces an expensive shielded high-speed serial link with standard Gigabit Ethernet, without changing what the central processor receives.

a pedestrian standing beside it", and angular resolution comes from having many antennas. A radar with N_t transmitters and N_r receivers can synthesise an effective array of $N_t \times N_r$ elements, so resolution improves roughly with the product. The data volume, unfortunately, scales the same way.

At the same time, vehicle electrical architectures have been consolidating. Where a car once had a hundred small electronic control units each doing one job, the industry has moved to domain controllers, then to *zonal* architectures where sensors report to a nearby gateway, and toward fully centralised designs where one high-performance system-on-chip does everything [4, 5, 6]. Centralising the computation has obvious advantages, but it means the raw sensor data has to travel.

Figure 1.1 shows the consequence. A Serialiser/Deserialiser (SerDes) link [7, 8] carries the aggregated output of several radar modules at 2.5 Gbit/s or more. Those data rates are achievable, but they require shielded cable, they cost money and weight, and they put a matching ingest burden on the SoC's memory system [9]. If instead the data is compressed at the radar module by a factor of 2.5 or better, the same information fits inside Gigabit Ethernet [10] — a cheap, standard, unshielded interconnect. That single substitution is the economic argument for this entire body of work.

1.3 Why the compressor must be lossless

It is tempting to relax the requirement. Lossy compression reaches much higher ratios, and radar data is noisy anyway, so surely a little extra noise is harmless?

It is not, and the reason is specific rather than philosophical. The compressor sits *upstream* of detection. Whatever it discards is discarded before the detector ever sees it, and the detector's job is to find weak returns close to the noise floor — a pedestrian at range, a motorcycle between two trucks. Lossy compression raises the noise floor precisely where the margin is thinnest. Chapter 7 quantifies this on real data: Block Adaptive Quantization reaches the same compression ratio as the lossless coders studied here but misses nearly three times as many real detections and reports nearly nine times as many false ones.

Losslessness also makes verification unambiguous. The pass criterion is that every reconstructed sample equals its original exactly. There is no tolerance to argue about and no distortion metric to trade off: a single differing bit anywhere in the 157 million input bits of the test set is a failure, and the test reports it as one. Every design in this thesis is checked against that criterion, from MATLAB through C-simulation and RTL co-simulation to the encoders and decoders running on the FPGA board.

1.4 Aim and objectives

The starting point is the master's thesis of Lukas Kiem [11], which evaluated a range of radar compression schemes on recordings from a single-transmitter Infineon radar, selected Doppler Redundancy Hybrid Encoding (DRHE) as the best of them, and implemented its compressor on a Zynq UltraScale+ device (compression ratio 3.17 in hardware). Chapter 3 describes that work. Four questions that his work leaves open define the objectives of this thesis.

Aim. Compare compression methods side by side on real, public, multi-transmitter radar data; select the best *lossless* methods; and implement both their encoders and their decoders on an FPGA, measured on a real board.

1. **Validate on public MIMO data.** Kiem's results come from private recordings of a single-transmitter radar. This thesis uses 50 consecutive frames of the public ColoRadar dataset [12], recorded with a Texas Instruments twelve-transmitter, sixteen-receiver cascade, for every result from simulation to hardware.
2. **Examine the quantisation step.** Kiem identified the conversion to 16-bit fixed point (FX16) as the source of his detection errors. This thesis measures what that step costs in detection, and whether floating-point storage is a usable alternative.
3. **Broaden the comparison.** Eight methods are compared on the same data with the same

metrics: two reference formats, three lossy quantisers, DRHE on integer and on floating-point data, and a linear predictive coder (LPC) designed for this thesis as a controlled comparison with DRHE.

4. **Implement and measure the complete pipeline.** Kiem implemented compression only and measured it on his own system. This thesis implements compression *and* decompression for both selected algorithms, verifies them bit-exactly from MATLAB to the board, and measures throughput, latency and resources on an AMD KCU116 board.

1.5 Contributions

1. **A comparison of eight compression methods on real cascade radar data** (Section 7.1). Only the two predictive lossless coders, DRHE (3.75) and LPC (3.70), compress by more than three without losing detections; the lossy quantisers reach up to 3.76 but miss 10 % to 29 % of targets, and floating-point storage compresses by only 1.04.
2. **The correct prediction axis for time-division MIMO radar** (Section 4.3). On a TDM cascade consecutive ramps come from different transmitters, so slow-time prediction must use the previous ramp of the same transmitter, n_{Tx} ramps back. Predicting from the adjacent ramp, as single-transmitter descriptions of these algorithms do, compresses worse than not predicting at all.
3. **A breakdown of where the compression comes from** (Sections 7.3 and 10.3). Huffman coding alone achieves 3.48 on this data — mostly from unused fixed-point headroom and the sparsity of a range-FFT frame — prediction adds 6 % to 8 %, and the Huffman dictionary decides the rest.
4. **Huffman dictionaries matched to the prediction errors** (Section 7.4). Re-derived from five frames, they raise DRHE to 4.22 and LPC to 4.40, hold on the 45 frames they were not trained on, and reverse the ranking of the two algorithms.
5. **Complete, verified hardware for both algorithms** (Chapters 6 and 7). Encoders and decoders were implemented in Vitis HLS and Vivado and run in a board test system on an AMD KCU116. Compressed output is byte-identical to the simulation on all 50 frames with both dictionaries, the hardware decoders reconstruct every frame exactly, and throughput, latency and resources are measured for both halves. A range-FFT front end

on the FPGA lets the board start from raw ADC samples and shows that the MATLAB front end used for the study is a faithful stand-in.

6. **A comparison with the baseline** (Chapters 8 and 9), including a reconstruction of Kiem's simulation from his text, which reaches his simulated DRHE ratio at one plausible noise level and explains how his results relate to the ones here.

1.6 Thesis outline

Chapter 2 gives the background on FMCW and MIMO radar, lossless compression and FPGAs, and Chapter 3 summarises Kiem's thesis. Chapter 4 describes the eight methods and the two predictive coders in detail, and Chapter 5 how they are evaluated and verified. Chapter 6 describes the encoders, decoders and board test system, and Chapter 7 reports the results. Chapters 8 and 9 compare the hardware and the compression results with Kiem's. Chapter 10 discusses what the results mean for a radar sensor, and Chapter 11 concludes.

Chapter 2

Background

This chapter covers what the radar measures and how, why its data is large, how lossless compression works, and the FPGA concepts the hardware chapters use.

2.1 How an FMCW radar measures range

2.1.1 The chirp

Automotive radar is almost universally *frequency-modulated continuous wave* (FMCW). Instead of emitting a short pulse and timing the echo — which would demand impractically fast electronics for the distances involved — an FMCW radar transmits continuously, sweeping its frequency linearly upward over a short interval. One such sweep is called a *chirp*. The sensor used here sweeps 1.798 GHz of bandwidth around a 77 GHz centre frequency while its echo is sampled, and starts a new chirp every 78.5 µs.

When the chirp reflects off an object at range R, the echo arrives delayed by the round-trip time

$$\tau = \frac{2R}{c}, \tag{2.1}$$

where c is the speed of light. Because the transmitter has kept sweeping in the meantime, the echo's frequency at any instant is lower than what is currently being transmitted, by an amount proportional to the delay. If the chirp sweeps B hertz in T_c seconds, the sweep slope is $S = B/T_c$ and the frequency difference is

$$f_b = S\tau = \frac{2SR}{c}. \tag{2.2}$$

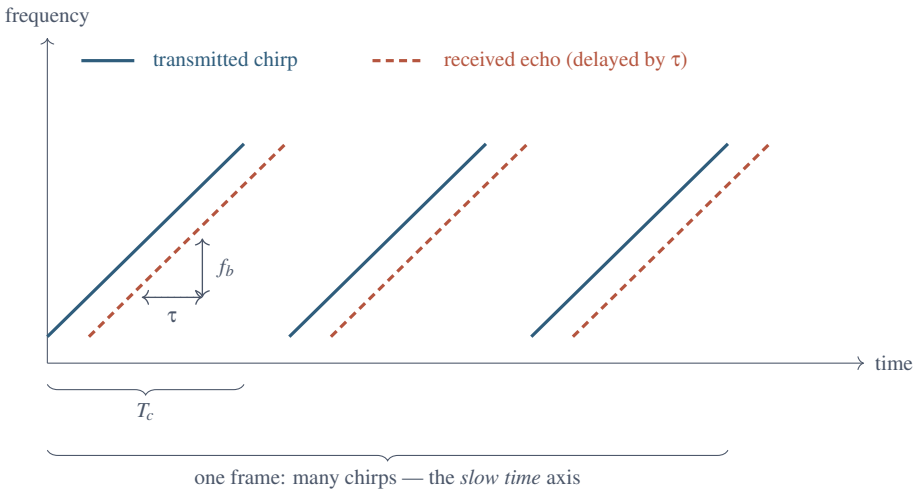


Figure 2.1: FMCW operation. Each chirp sweeps linearly in frequency. The echo is a delayed copy, so at any instant the transmitted and received frequencies differ by $f_b = S\tau$, proportional to target range. A frame consists of many such chirps in succession; the axis running across chirps is called *slow time* and is where Doppler information lives.

A mixer multiplies the transmitted and received signals, producing a low-frequency *beat* or intermediate-frequency signal at exactly f_b. That signal is low-pass filtered and digitised by an analog-to-digital converter (ADC) at a modest rate — 8 MHz here, rather than the tens of gigahertz the raw RF would need. This is the central trick of FMCW: it converts a timing measurement that would need picosecond resolution into a frequency measurement that an ordinary ADC can make.

2.1.2 From beat frequency to a range profile

Measuring f_b is a spectral estimation problem, so the natural tool is the Fourier transform. Taking an N_s-point FFT across the N_s ADC samples of a single chirp produces a *range profile*: bin n of the FFT corresponds, through Equation (2.2), to range

$$R_n = n \cdot \frac{c}{2B}, \tag{2.3}$$

so the range resolution is

$$\Delta R = \frac{c}{2B} \approx 8.3\,\text{cm} \tag{2.4}$$

for the 1.798 GHz bandwidth used here. A peak in the range profile is an object. This first transform is called the **Range FFT**, and its output is the data this thesis compresses.

2.1.3 Slow time and Doppler

A single chirp gives range but not velocity. To get velocity, the radar transmits many chirps in rapid succession — a *frame* — and looks at how the *phase* of a given range bin evolves from chirp to chirp.

The reasoning is worth spelling out, because the whole of DRHE rests on it. A target at range R produces a range-FFT value whose phase is $\theta = -4\pi R/\lambda$, where λ is the wavelength. Over the chirp-to-chirp interval T, a target moving at radial velocity v changes range by vT, so its phase changes by

$$\Delta\theta = -\frac{4\pi v T}{\lambda}. \tag{2.5}$$

At 77 GHz, $\lambda \approx 3.9\,\text{mm}$, so even a very slow target produces a measurable phase step — which is exactly why radar is good at velocity. Critically for compression, $\Delta\theta$ is *constant* for a target at constant velocity. The phase of a given range bin therefore advances linearly across chirps, and its magnitude barely changes at all. That is a highly predictable sequence, and predictable sequences compress.

The axis running across chirps within a frame is called **slow time**, as opposed to the *fast time* axis of samples within a chirp. Taking a second FFT along slow time resolves $\Delta\theta$ into velocity: this is the **Doppler FFT**.

2.2 Angle, MIMO and the virtual array

Angle is measured by comparing the phase of the same echo at separated receive antennas: a wavefront arriving at angle ϕ reaches adjacent antennas, spaced d apart, with a phase difference

$$\Delta\psi = \frac{2\pi d \sin\phi}{\lambda}, \tag{2.6}$$

and an FFT across the antennas — the **Angle FFT** — turns these into a bearing. Angular resolution grows with the size of the array. Multiple-Input Multiple-Output (MIMO) radar enlarges the array cheaply: if the signal of each of N_t transmitters can be separated at each of N_r receivers, every transmit–receive pair acts as an element of a *virtual array*. The sensor used here has $N_t = 12$ and $N_r = 16$, giving 192 virtual elements from 28 physical antennas.

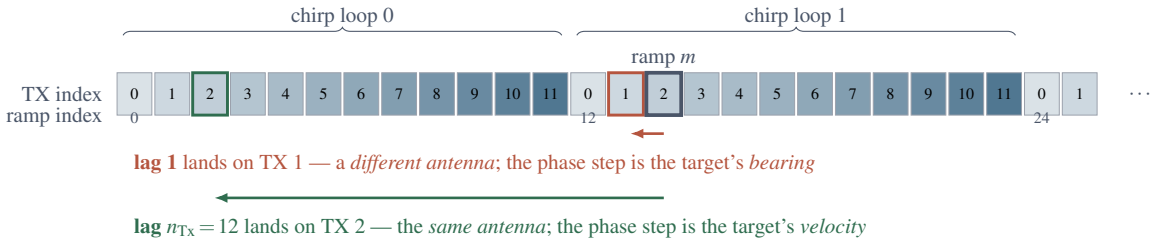


Figure 2.2: Why "the previous ramp" is the wrong neighbour on a TDM-MIMO cascade. Each cell is one ramp, labelled with the transmitter that produced it; the twelve transmitters rotate inside every chirp loop. Predicting ramp m from ramp $m-1$ (red) crosses a transmitter boundary eleven times out of twelve, so the two ramps differ by the target's array steering phase. Predicting from ramp $m - n_{\mathrm{Tx}}$ (green) stays on the same physical antenna and therefore on the genuine Doppler axis.

2.2.1 Time-division multiplexing, and why it matters here

The separability requirement has to be met somehow. The simplest method, and the one this sensor uses, is *time-division multiplexing* (TDM): the transmitters fire one at a time, in a fixed rotation, so whatever a receiver hears during transmitter t's slot must have come from transmitter t.

TDM is simple and robust, and it has one consequence that matters throughout this thesis. Inside every repetition of the frame — every "chirp loop" — all N_t transmitters fire in sequence before the loop repeats. When the resulting data is laid out as a two-dimensional array of (ramp, range bin), which is how a streaming compressor sees it, the ramp index is

$$m = qN_t + t, \qquad q \in [0, N_c), \quad t \in [0, N_t), \tag{2.7}$$

where q indexes the chirp loop and t the transmitter. The transmitter index varies *fastest*.

This is the correct physical time order. It is also, as Figure 2.2 makes visual, a trap. Ramp m and ramp $m-1$ are different physical transmitters $N_t - 1$ times out of N_t. They are separated in *space* by the array baseline, not in *time* by a pulse interval, and the phase relationship between them is governed by Equation (2.6) — the target's bearing — not by Equation (2.5) — the target's velocity. The genuine slow-time neighbour of ramp m, the one whose phase differs by a Doppler step, is ramp $m - N_t$. From Chapter 4 on, this distance is written n_{Tx}; on this sensor $n_{\mathrm{Tx}} = N_t = 12$.

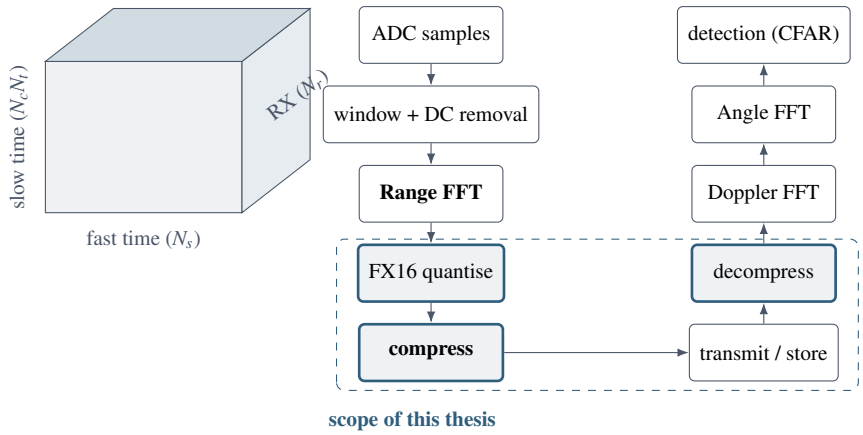


Figure 2.3: The radar data cube and the processing chain. The compressor sits immediately after range-FFT and fixed-point quantisation, before the data leaves the sensor. Everything downstream — Doppler FFT, angle FFT, detection — sees the reconstructed data, which for a lossless compressor is bit-identical to the original.

2.3 The radar data cube, and why it is large

Putting the three axes together gives the *radar data cube*: fast time (samples within a chirp) $\times$ slow time (ramps) $\times$ receive channel. Three FFTs are applied along the three axes in turn, yielding range, velocity and angle. Figure 2.3 shows the arrangement and where in the chain this thesis intervenes.

The volume follows directly. For the sensor used here, one frame is

$$N_s \times N_c \times N_r \times N_t = 256 \times 16 \times 16 \times 12 = 786{,}432 \tag{2.8}$$

complex samples. Each is two 16-bit integers, so a frame is 3,145,728 B — exactly 3 MiB. At 20 frames per second, a common automotive rate, that is about half a gigabit per second from one sensor, and a vehicle carries several.

2.4 Lossless compression: how it actually works

2.4.1 The two categories

Compression divides into *lossless*, where the decompressed data is bit-identical to the input, and *lossy*, where some information is permanently discarded in exchange for a smaller payload [14]. Lossy compression can reach far higher ratios; lossless compression is bounded by the information content of the data itself. Section 1.3 explained why this application needs the lossless kind.

2.4.2 The bound: entropy

There is a hard limit on lossless compression, and knowing it is what makes the results in Chapter 7 interpretable. If a source emits symbols x with probabilities $p(x)$, its *order-0 entropy* is

$$H = -\sum_{x} p(x) \log_2 p(x) \quad \text{bits per symbol,} \tag{2.9}$$

and no code that treats symbols independently can average fewer bits than H [14]. This gives a concrete yardstick: measure H for the data, compare it against what the coder actually achieves, and the gap is the coder's inefficiency. Section 7.3 uses exactly this to separate what the predictor contributes from what the Huffman dictionary contributes.

2.4.3 The standard two-stage structure

Almost every practical lossless compressor has the same shape: a *decorrelation* stage followed by an *entropy coding* stage.

The **decorrelation** stage exploits structure. If consecutive samples are similar, predict each one from its neighbours and transmit only the *residual*, the difference between prediction and truth. When the prediction is good, residuals cluster tightly around zero even though the original samples spanned the full dynamic range. Nothing is lost: given the residual and the same prediction rule, the decoder reconstructs the original exactly.

The **entropy coding** stage exploits the resulting skew in the distribution. Values that occur often get short codewords; values that occur rarely get long ones. Huffman coding [15] is the classic construction and produces the optimal prefix-free code for a known symbol distribution.

Crucially, the two stages are independent. A predictor that fails — that produces residuals no more skewed than its input — does not break the coder; the coder simply carries on compressing whatever it is handed, and the compression ratio can still look respectable. Detecting this requires deliberately measuring what the coder achieves *without* the predictor. This thesis reports that no-prediction baseline alongside every predictive result.

2.4.4 Open-loop versus closed-loop prediction

One subtlety in predictive coding matters enough to state now, because the hardware of Chapter 6 is built around it (Section 6.4).

The encoder predicts from past samples. So does the decoder — but the decoder does not have the past *original* samples, only the past *reconstructed* ones. In *open-loop* prediction the encoder uses the originals; in *closed-loop* prediction (classically, differential pulse-code modulation, DPCM) the encoder reconstructs each sample exactly as the decoder will, and predicts from that.

When compression is perfectly lossless the two are identical, because reconstruction equals the original. The moment anything — a saturation, a rounding, a clamp — makes reconstruction differ from the original by even one bit, an open-loop encoder and its decoder begin predicting from different histories and *diverge*. With an IIR predictor, as in DRHE, that divergence is then carried forward through every subsequent ramp. Closing the loop confines the error to the affected sample.

2.5 The sensor and the dataset

2.5.1 AWR2243 and the MMWCAS-RF-EVM

The radar front end assumed throughout is the Texas Instruments AWR2243, a 76–81 GHz FMCW transceiver with three transmit and four receive channels [13]. Importantly for this work, the AWR2243 performs no signal processing of its own: it transmits, receives, mixes and digitises, then streams raw ADC samples out over a MIPI CSI-2 interface. It therefore exposes exactly the payload that creates the transport bottleneck.

Four AWR2243 devices are cascaded on the MMWCAS-RF-EVM evaluation module, giving 12 transmit and 16 receive channels on one synchronised board — the 192-element virtual array of Section 2.2.

2.5.2 ColoRadar

Rather than synthetic stimulus, this thesis uses real captures from the public ColoRadar dataset [12], which includes cascade recordings from exactly this module across indoor, outdoor and subterranean environments. Table 2.1 lists the relevant parameters. Fifty consecutive frames were used throughout; that is 39,321,600 complex ADC samples, or 150 MiB of raw data, which is enough to make a claim of losslessness meaningful.

Each complex sample is stored as two contiguous `int16` values. The index of the sample

Table 2.1: ColoRadar cascade capture parameters used throughout this thesis.

Parameter	Symbol	Value
ADC samples per chirp	N_s	256
Chirp loops per frame	N_c	16
Receive channels	N_r	16
Transmit channels (TDM)	N_t	12
Ramps per frame	$N_c N_t$	192
Fast-time sample rate	f_s	8 MHz
Chirp bandwidth	B	1.798 GHz
Ramp repetition interval	T_r	78.5 µs
Centre frequency	f_c	77 GHz
Range resolution	ΔR	8.34 cm
Frame size (raw)	—	3,145,728 B

at (fast-time s, chirp loop q, receiver r, transmitter t) in the file is

$$i = 2\big(s + N_s(q + N_c(r + tN_r))\big), \tag{2.10}$$

so the layout from slowest to fastest varying is $[t, r, q, s]$. Reshaping this into the (fast time, ramp, receiver) cube the processing chain expects is what produces the transmitter-interleaved ramp axis of Equation (2.7).

2.6 Quantisation: the FX16 step

The range-FFT output is naturally a complex floating-point quantity. Before it can be compressed by a fixed-point pipeline it has to be quantised to 16-bit integers — the **FX16** step. This is a scaling followed by rounding:

$$\sigma = \frac{32767}{A_{\max}\, 2^{\text{shift}} \sum_k w_k \,/\, (2N_s)}, \qquad x_{\text{FX16}} = \text{round}(\sigma \cdot x), \tag{2.11}$$

where $A_{\max}$ is the full-scale ADC amplitude and w is the window applied along fast time.

Two things about Equation (2.11) matter later. First, the rounding introduces an error bounded by half a least-significant bit, which acts as a wideband noise floor [18]; this is the dominant source of detection degradation in the whole chain, and Chapter 7 separates it cleanly from the cost of compression. Second, and more consequentially for this thesis, the scale factor is *fixed* — it is calibrated from a theoretical full-scale ADC sinusoid, not from the data. If the actual capture sits well below full scale, the top bits of every resulting `int16` are only copies of

the sign bit. On ColoRadar the largest FX16 code is 437, so fewer than ten of the sixteen bits ever carry information; Section 10.3 shows how much of the compression ratio this accounts for.

2.7 FPGAs and high-level synthesis

A field-programmable gate array (FPGA) is a chip of configurable logic blocks and wiring that *becomes* the circuit described, rather than executing instructions. It can process four receive channels in the same clock cycle, with deterministic timing, and can be changed after manufacture, which is why it suits a proof of concept; a production design would normally move into an ASIC or a fixed-function block of a larger SoC [19, 21]. Its resources are counted in four units: **LUTs** (look-up tables, which implement logic), **flip-flops** (one-bit registers), **block RAM** (on-chip memory, counted here in 36 kbit blocks, each with two access ports) and **DSP slices** (hardened multiply–accumulate units). The device used throughout, an AMD Kintex UltraScale+ `xcku5p-ffvb676-2-e`, provides 216,960 LUTs, 433,920 flip-flops, 480 block RAMs and 1,824 DSP slices. Fixed-point arithmetic maps onto these cheaply; floating point needs a multi-cycle operator for every operation [16, 17].

A **pipeline** splits a computation into stages separated by registers so that several inputs are in flight at once; its *depth* is the number of cycles one input takes to pass through. The **initiation interval** (II) is the number of cycles between accepting one input and the next: $\mathrm{II} = 1$ is one input per clock, and $\mathrm{II} = 2$ halves the throughput. $\mathbf{F}_{\max}$ is the fastest clock at which every path still settles within one period. A frame of K input beats takes about $K \cdot \mathrm{II}/f$ seconds, plus the pipeline depth once per loop invocation.

High-level synthesis (HLS) generates the circuit from annotated C++: loops become pipelined state machines, arrays become memories, and pragmas set the II and the interfaces. Because the source is C++, the same code also runs as software against the MATLAB reference before any hardware exists, which makes the verification of Section 5.4 possible. The cores here use AXI4-Stream interfaces: a valid/ready handshake carrying one fixed-width word per beat.

2.8 Related work

Commercial radar processors use lightweight deterministic compressors with bounded latency: Texas Instruments' exponential-Golomb encoding [20] and NXP's block floating-point formats such as CP4D [21], both "near lossless" at ratios of about 2 to 4. Block adaptive quantisation (BAQ), from synthetic aperture radar [22], adapts the quantiser step to each block's statistics. Range-dependent variable-length encoding (RDVLE) [26] spends fewer bits on far range bins, where received power is lower. All three are lossy. Doppler Redundancy Hybrid Encoding (DRHE) [26, 25] predicts each range-FFT sample along slow time with two IIR filters in polar form and entropy-codes the error; linear predictive coding (LPC) is the general form of the same idea, applied by Meucci and Mancuso [27] to raw ADC samples. Autoencoder and spectral approaches reach higher ratios but are lossy and expensive at the sensor [23, 24].

The direct predecessor of this work is Lukas Kiem's master's thesis [11], which compared industrial schemes with DRHE on a single-transmitter Infineon radar, selected DRHE (3.25 in simulation) and implemented it on a Zynq UltraScale+ device (3.17 in hardware). Chapter 3 summarises it.

Chapter 3

The Baseline: Kiem's Thesis

Most results in this thesis are set beside a result from Lukas Kiem's master's thesis [11], carried out at Infineon. This chapter sets out what he did, on what data, and what he concluded. Section and table numbers quoted here ("his Table 3.5") refer to his thesis.

3.1 His problem and his data

Kiem addressed the transport problem of Section 1.2 with a use case (his Section 3.3): one Infineon CTRX8191F MMIC with four receive channels and 12-bit samples at 50 MS/s produces

$$50\,\mathrm{MS/s} \times 12\,\mathrm{bit} \times 4 = 2.4\,\mathrm{Gbit/s}, \tag{3.1}$$

so one Gigabit Ethernet link needs a compression ratio of 2.4, or 2.7 with protocol headers. The sensor cycles at 20 Hz; a frame of 1024 ramps takes about 21 ms to acquire, and he allowed compression 15 ms of the remaining 29 ms.

Two features of his setting differ from this thesis's. His radar uses **one transmitter**, so consecutive ramps are consecutive observations by the same antenna pair and the question on which this thesis's prediction design turns (Section 4.3) never arises. And it delivers **real** samples, whose spectrum is symmetric, so he keeps half of each range FFT because the other half is redundant; ColoRadar's samples are complex.

His evaluation chain (his Figure 3.1), which this thesis adopts almost unchanged, shifts the ADC samples into a 32-bit word, applies a Hanning window and a range FFT divided by the number of samples, compresses and decompresses, and then runs a Doppler FFT, non-coherent integration (NCI) over the receive channels and a peak detector, comparing the detections with

those of an FP16 reference path. Every compression result he publishes, in simulation and in hardware, was measured on his own recordings: one MMIC, 512×512 samples, four receive channels, 14-bit data. They are not public. He also shows a synthetic five-target scene in white noise (his Table 3.2), but only as an illustration of the chain.

3.2 His design space and result

He compared FP16 (the reference), 16-bit fixed point (FX16), two Infineon common-exponent formats (CMPRA and CMPRB, fixed ratios 2 and 4), Texas Instruments' exponential-Golomb engine at four settings (EGE10 to EGE4, ratios 1.6 to 4), and Doppler Redundancy Hybrid Encoding (DRHE) [26, 25]: prediction along the slow-time axis of the range-FFT output with two IIR filters in polar form, then Huffman coding of the differences. He excluded one method, the LPC-Huffman of Meucci and Mancuso [27], in these words:

> "LPC-Huffman compression is not taken because it has the same approach as DRHE, but instead of compressing range FFT data, it operates on raw ADC data; the results for DRHE are more promising." (his Section 3.4.1)

He assumed that method's domain from its int16 input, did not run it, and compared published figures from different data. Table 3.1 is his result. DRHE reached a ratio of 3.25 with exactly FX16's detection metrics, since it is lossless with respect to the 16-bit codes; the lossy schemes either stayed below the requirement or lost detections. One miss in about 170 (0.59 %) shows that his recording held about 170 reference detections, at an SNR of about 22 dB.

Table 3.1: Kiem's design-space exploration on his recordings (his Table 3.5). FP% is normalised by test detections, so it is a false discovery rate (Section 5.3).

Algorithm	CR	FN%	FP%	NF (dBFS)	SNR (dB)
FP16	1	0	0	−70.714	22.407
FX16	1	0.59	1.18	−70.691	22.330
CMPRA	2	0.59	0.59	−70.687	22.418
CMPRB	4	7.69	4.73	−70.253	22.111
EGE10	1.6	1.18	1.18	−70.689	22.403
EGE8	2	2.37	2.37	−70.640	22.289
EGE6	2.67	7.69	7.10	−70.367	22.197
EGE4	4	40.24	26.63	−74.689	28.245
DRHE	**3.25**	0.59	1.18	−70.691	22.330

3.3 His final scheme

Two simplifications made DRHE suitable for hardware. He removed the run-length half of the original coder, which changed the ratio only from 3.25 to 3.24 and lets every value be coded independently in one clock. And he fixed the Huffman dictionary, built once from the S_4 statistics of one of his recordings (his Appendix A); it is the *original dictionary* of this thesis (Section 4.6). For each range bin n of ramp m and each channel, his final scheme (his Section 3.5.3) predicts

$$\hat{a}[m,n] = \alpha\hat{a}[m-1,n] + (1-\alpha)\,a[m-1,n], \tag{3.2}$$

$$\hat{\theta}[m,n] = \beta\hat{\theta}[m-1,n] + (2-\beta)\,\theta[m-1,n] - \theta[m-2,n], \tag{3.3}$$

with $\alpha = 0.6$ and $\beta = 0.4$, wraps the phase once into $[-\pi,\pi]$, converts the prediction back to Cartesian with rounding, and codes the 16-bit difference as its size category S_4 (Table 4.2) and the APPEND bits. The state starts at zero and is updated from the input samples. These are Equations (4.6) and (4.7) of Chapter 4 at a lag of one ramp.

3.4 His hardware

He implemented the scheme with Vitis HLS and Vivado 2022.2 on an AMD Zynq UltraScale+ `XCZU3CG-SFVC784-1-e`, in a system that streams pre-windowed ADC data from an SD card through a Xilinx FFT core into the compressor. The compressor reads one 128-bit word per cycle (one complex bin for each of four channels), uses fixed point throughout, holds its state in block-RAM arrays of 1024 words per channel, packs each channel's codes into a 58-bit word, and runs at $\mathrm{II} = 1$ at 100 MHz. Table 3.2 collects the figures he reports. His throughput is theoretical, 128×100 MHz expressed in binary units; his hardware ratio is below his simulated one because the hardware FFT is fixed point.

3.5 What he left open

He concluded that DRHE met every requirement of his use case. His future work included floating-point arithmetic for the prediction, since his core far exceeded its real-time requirement; a bit-accurate decompression model; tests with a real sensor; safe error handling; and a

Table 3.2: Kiem's published hardware figures (his Section 5.2, Tables 5.2, 5.3 and 5.5). BRAM in 36 Kb blocks.

Quantity	Value
Compression IP: LUT / FF / BRAM / DSP	45,892 / 9,243 / 10 / 44
Whole system: LUT / FF / BRAM / DSP	59,353 / 32,552 / 38 / 80
IP latency (HLS estimate)	23 cycles, 230 ns
System latency (last DMA read to last DMA write)	79.4 µs
Input throughput	"11.92 Gbit/s"
CR: MATLAB / bit-accurate simulation / hardware	3.25 / 3.24 / 3.17

guaranteed compression ratio. This thesis takes up the first two: its DRHE predicts in floating point, and Chapter 8 measures what that costs, and it implements and verifies a hardware decompressor for every design. Its saturation of the one residual the code cannot represent (Section 6.4) is a form of the safe error handling he asked for.

Chapter 4

Compression Methods

This chapter describes the eight candidate methods, then the two taken to hardware: their shared entropy coder, the axis along which they predict, the two predictors, and the Huffman dictionary.

4.1 The eight candidate methods

Every method operates on the same input: the range-FFT output of each frame, quantised to 16-bit integers by the FX16 step of Section 2.6, except where a method replaces that step. All eight were implemented in the same MATLAB chain and scored with the same metrics (Chapter 5).

FP16, FX16

The two reference formats, as in Kiem's thesis: 16-bit floating point, and the 16-bit integer codes of the FX16 step. Neither compresses; FX16 is the input to every integer method below, so its detection losses are the floor that any of them can reach.

BAQ

Block adaptive quantisation, implemented in block-floating-point form: each block of 16 range bins shares an 8-bit exponent chosen from the block maximum, and each real and imaginary part keeps a 4-bit mantissa, so the ratio is fixed at $32/(8+8/16) = 3.76$. It stands in for the block-floating-point family (NXP's CP4/CP8 and Infineon's CMPRA/CMPRB are of the same kind) and for BAQ proper [22]. Lossy.

RDVLE

Range-dependent variable-length encoding [26]: fewer bits for far range bins, following the fall of received power with range, here capped at three bits removed. Lossy.

RDVLE+UDRE

RDVLE followed by uniform dynamic-range re-encoding of blocks of 16 bins at six bits [26]. Lossy.

DRHE

Doppler Redundancy Hybrid Encoding, as specified by Kiem (Section 4.4): slow-time prediction in polar form, followed by the entropy coder of Section 4.2. Lossless with respect to the FX16 codes.

DRHE fp16

The same predictor applied to FP16 values instead of FX16 codes, to test whether floating-point storage — the remedy Kiem suggested for quantisation error — can be compressed.

LPC

A second-order linear predictive coder designed for this thesis (Section 4.5). It predicts along the same axis as DRHE and uses the same entropy coder, so that only the predictor differs. Lossless with respect to the FX16 codes.

NXP's formats were not implemented separately, since their exact bit layouts are proprietary and the block-floating-point quantiser represents the family. Neural and spectral methods such as AdaRadar [24] were left out of scope: they are lossy, and their decoders run on the central processor.

The rest of this chapter concerns DRHE and LPC, which Section 7.1 shows are the only two methods that compress by more than three without losing detections.

4.2 The shared entropy coder

4.2.1 Residuals

A predictor P estimates the sample $x[m]$ from samples that have already been transmitted, $\hat{x}[m] = P\big(x[m-L], x[m-2L], \ldots\big)$, and the encoder sends the *residual*

$$d[m] = x[m] - \hat{x}[m]. \tag{4.1}$$

The decoder runs the identical rule on the samples it has already reconstructed and recovers $x[m] = d[m] + \hat{x}[m]$. Nothing is lost: given $\hat{x}[m]$ the map from x to d is a bijection. The lag L

Table 4.1: The same sixteen real samples, written two ways. Range bin 75, receive channel 1, ramps 168–183. Both rows carry identical information.

m	168	169	170	171	172	173	174	175	176	177	178	179	180	181	182	183
x (raw)	−69	−52	59	−6	−29	−15	59	−20	−69	−73	−45	84	−73	−50	59	−5
d (residual)	3	−3	0	−1	1	3	−4	3	−1	−2	−1	0	−2	4	−1	1

is the distance back along the ramp axis; Section 4.3 explains why it is $n_{\mathrm{Tx}} = 12$ on this radar rather than 1. The residuals are cheaper to write down because their *distribution* is narrower (Table 4.1): on one range bin of ColoRadar the raw values swing over $[-73, +84]$, while the residuals never leave $[-4, +4]$.

A fixed four-bit field cannot carry the occasional large residual, and a code that gives small values fewer bits must still let the decoder find where each value ends. Both compressors here do this as JPEG codes its DC coefficients: each value is sent as a short prefix-free codeword for its *size*, followed by the value itself in exactly that many raw bits. The prefix-free table then has sixteen entries rather than 65,536, can be fixed at design time, and is decoded with a sixteen-way comparison.

4.2.2 The category S_4 and the APPEND bits

The size is the category

$$S_4(v) = \begin{cases} 0 & v = 0, \\ \min\bigl(15,\ \lfloor \log_2 |v| \rfloor + 1\bigr) & v \neq 0, \end{cases} \tag{4.2}$$

the number of bits needed to write $|v|$ (Table 4.2).

Table 4.2: The first few categories. Category $k \geq 1$ covers $|v| \in [2^{k-1}, 2^k - 1]$, which counting both signs is exactly 2^k distinct values — precisely what k raw bits can enumerate.

S_4	values covered	count	
0	$v = 0$	1	the only category with no payload
1	$v \in \{-1, +1\}$	2	
2	$\lvert v\rvert \in \{2, 3\}$	4	
3	$\lvert v\rvert \in [4, 7]$	8	
4	$\lvert v\rvert \in [8, 15]$	16	
⋮			
15	$\lvert v\rvert \in [16384, 32767]$	32,768	exactly fills 15 bits

The payload is

$$A(v) = \begin{cases} v & v \geq 0, \\ v + 2^{S_4(v)} - 1 & v < 0, \end{cases} \tag{4.3}$$

written in $S_4(v)$ raw bits. Category k holds exactly 2^k values, and the offset maps the negative values onto the k-bit patterns with the top bit clear and leaves the positive ones with it set, so the sign is recovered from one bit without being transmitted. For example, $v = -5$ has $S_4 = 3$ and $A = 2$, sent as the category's codeword followed by `010`. Section 6.4 deals with the one value, -32768, that has no code.

4.2.3 Cost, and the shape of the dictionary

The total cost of coding one residual is

$$L(v) = \underbrace{\ell\big(S_4(v)\big)}_{\text{Huffman codeword}} + \underbrace{S_4(v)}_{\text{payload}}, \tag{4.4}$$

where $\ell(\cdot)$ is the codeword length for that category. The dictionary is *fixed* — not computed per frame — which saves transmitting it and removes a serial dependency from the hardware. The *original* dictionary, taken from Kiem's thesis, has lengths

$$\ell = [4,3,2,2,2,5,6,7,8,9,10,11,14,14,13,12] \text{ bits}. \tag{4.5}$$

The full dictionary, codewords included, is in Appendix A. Every one of the sixteen entries was verified against the published table [11, Table A.1]; all sixteen agree once the published most-significant-bit-first codes are reversed for the least-significant-bit-first packer used in this implementation.

Substituting into Equation (4.4) gives the cost curve of the original dictionary:

Three features of this curve deserve attention.

A residual anywhere in $[-15,+15]$ costs four to six bits, a ratio of 2.7 to 4.0 against sixteen raw bits. Cost then jumps from six bits to ten at category 5, and above category 8 the code expands the data, which is why random 16-bit input grows by 67 % (Section 5.5). The dictionary gives its two-bit codewords to categories 2, 3 and 4, so it assumes residuals of magnitude 2 to 15, and it can never code a value in fewer than four bits. Section 4.6 derives dictionaries matched to this thesis's smaller prediction errors.

S_4	$\lvert v\rvert$ range	ℓ	payload	**total**
0	0	4	0	**4**
1	1	3	1	**4**
2	2–3	2	2	**4**
3	4–7	2	3	**5**
4	8–15	2	4	**6**
5	16–31	5	5	**10**
6	32–63	6	6	**12**
7	64–127	7	7	**14**
8	128–255	8	8	**16**

4.3 Predicting on a time-division MIMO radar

Both predictors work along *slow time*: they predict a range bin's value in one ramp from its values in earlier ramps. Section 2.2.1 showed that on a TDM-MIMO cascade the ramp axis interleaves the transmitters, so ramp m and ramp $m-1$ come from different antennas eleven times in twelve. The two predictors are therefore applied at a lag of $L = n_{\mathrm{Tx}} = 12$ ramps: each ramp is predicted from the previous ramp of the *same* transmitter. This section shows on real data why that choice matters.

Table 4.3 traces sixteen consecutive ramps of one range bin — the same bin as Table 4.1 — through the DRHE predictor twice. The left half predicts from ramp $m-1$, which is how slow-time prediction is described for a single-transmitter radar. The right half predicts from ramp $m-n_{\mathrm{Tx}}$. Everything else — the filters, the coefficients, the entropy coder, the dictionary — is identical between the two halves.

The left half is a predictor working across the wrong axis. Its residuals (-136, $+23$, -22, -70, …) are frequently *larger in magnitude than the samples they replace*, they land in categories 5 through 8, and they cost 11.00 bits per value — a compression ratio of 1.455, which is to say the entropy coder is salvaging a little from a predictor that has actively made things worse.

The right half is the same predictor twelve ramps back. Its residuals never exceed 4 in magnitude, they land in categories 0 through 3, and they cost 4.12 bits per value.

The difference comes from the phase. Against ramp $m-1$ the phase step swings over most of $[-\pi,\pi]$ (-2.244, $+0.592$, $+1.230$, …) and the magnitude by up to 22; against ramp $m-n_{\mathrm{Tx}}$ the phase step is essentially zero (0.002, -0.040, 0.017, …) and the magnitude moves by at most 4. Ramps 168 and 167 came from different antennas, so the phase be-

Table 4.3: Sixteen real samples traced through DRHE at two prediction lags. Range bin 75, receive channel 1, frame 0, ramps 168–183; real part shown, the imaginary part behaves identically. The window is late enough in the frame that both predictors' filters are fully converged. The two halves differ only in which earlier ramp the prediction comes from.

			predicted from ramp $m-1$				**predicted from ramp $m-n_{\mathrm{Tx}}$**			
m	TX	x	$\hat{x}$	d	S_4	bits	$\hat{x}$	d	S_4	bits
168	0	−69	67	−136	8	16	−72	3	2	4
169	1	−52	−75	23	5	10	−49	−3	2	4
170	2	59	81	−22	5	10	59	0	0	4
171	3	−6	64	−70	7	14	−5	−1	1	4
172	4	−29	24	−53	6	12	−30	1	1	4
173	5	−15	−29	14	4	6	−18	3	2	4
174	6	59	−6	65	7	14	63	−4	3	5
175	7	−20	−83	63	6	12	−23	3	2	4
176	8	−69	−93	24	5	10	−68	−1	1	4
177	9	−73	−65	−8	4	6	−71	−2	2	4
178	10	−45	−44	−1	1	4	−44	−1	1	4
179	11	84	28	56	6	12	84	0	0	4
180	0	−73	70	−143	8	16	−71	−2	2	4
181	1	−50	−75	25	5	10	−54	4	3	5
182	2	59	81	−22	5	10	60	−1	1	4
183	3	−5	67	−72	7	14	−6	1	1	4
				total bits		**176**				**66**
				bits per value		11.00				4.12
				compression ratio		1.455				**3.879**

Table 4.4: Where the bits go, one frame, four receive channels, 196,608 values, original dictionary. "% values" is how often a category occurs; "% bits" is how much of the output it accounts for. The two columns are very different, and the difference is the whole point of entropy coding.

		no prediction		**DRHE, lag 1**		**DRHE, lag n_{Tx}**	
S_4	cost	% values	% bits	% values	% bits	% values	% bits
0	4	16.71	14.61	11.78	9.81	**18.85**	17.87
1	4	28.57	24.97	22.01	18.33	**32.94**	31.22
2	4	31.45	27.49	31.95	26.60	**34.40**	32.60
3	5	14.66	16.02	22.21	23.12	11.31	13.40
4	6	4.31	5.65	6.46	8.07	1.49	2.12
5	10	1.75	3.82	2.36	4.92	0.50	1.18
6	12	1.15	3.01	1.43	3.58	0.29	0.82
7	14	1.14	3.47	1.09	3.17	0.18	0.59
8	16	0.19	0.66	0.65	2.17	0.03	0.13
9	18	0.07	0.29	0.06	0.23	0.01	0.05
bits/value		4.576		4.803		**4.220**	
compression ratio		3.497		3.331		**3.792**	

tween them is set by the target's bearing (Equation (2.6)); ramps 168 and 156 came from the same antenna, so it is set by the target's velocity (Equation (2.5)), and this target is stationary. Coded with the original dictionary, the first six lag-n_{Tx} residuals become the 24-bit stream `1011 1000 0101 0010 0011 1011`, where the same six samples at lag 1 take 68 bits.

4.3.1 Where the bits go over a whole frame

The sixteen-sample window is illustrative but small. Table 4.4 gives the same accounting over an entire frame — 196,608 values, all four receive channels — which is what the compression ratio actually integrates.

The lag-n_{Tx} predictor works by emptying the tail. It moves 86.19 % of values into categories 0–2, which all cost four bits, and cuts categories 5–8 from 5.53 % to 1.00 % of values. The lag-1 predictor does the opposite: against no prediction it pushes values *up* into categories 3, 4 and 8, which is how a predictor on the wrong axis loses to doing nothing.

4.3.2 Why not one model per transmitter?

An alternative way to keep prediction on one antenna is to split each frame into twelve independent 16-ramp blocks, one per transmitter, and predict within each. For DRHE that is equivalent to the lag-n_{Tx} indexing above. For LPC it is not: fitting separate coefficients to each block

multiplies the side information by twelve, from 1.04 % to 12.5 % of the uncompressed frame, and shortens each autocorrelation estimate from 192 ramps to 16, which makes the coefficients much noisier. Measured over five frames it compresses to only 2.63, worse than not predicting at all. Keeping one coefficient pair per (bin, channel, part) and moving the lags to n_{Tx} and $2n_{\mathrm{Tx}}$ costs no extra side information, and is the design used here. On a TDM-MIMO radar the right change is to the *lag*, not to the *partitioning*.

4.4 DRHE: Doppler Redundancy Hybrid Encoding

DRHE [26, 25] exploits the redundancy of radar data along the Doppler (slow-time) axis, and is a *hybrid* of prediction and entropy coding: a predictor removes the slow-time structure, and an entropy coder — originally run-length plus Huffman, here the S_4 coder of Section 4.2 — codes what is left.

4.4.1 Why predict in polar coordinates

Section 2.1 established what a range-bin sequence looks like across slow time for a target at constant velocity: the *magnitude* is nearly constant, and the *phase* advances by a constant increment. In Cartesian (I, Q) coordinates that is a point rotating on a circle — both components oscillate, and a simple difference between consecutive samples is large. In polar coordinates the same sequence is two nearly-trivial signals: a constant and a ramp.

DRHE therefore converts to polar, predicts magnitude and phase separately with filters matched to those two shapes, converts the prediction back to Cartesian, and subtracts.

4.4.2 The two predictors

For ramp m and range bin n, with $a[m,n] = |x[m,n]|$ the magnitude and $\theta[m,n] = \angle x[m,n]$ the phase, and $L = n_{\mathrm{Tx}}$ the prediction lag of Section 4.3, the predictions are

$$\hat{a}[m,n] = \alpha\,\hat{a}[m-L,n] + (1-\alpha)\,a[m-L,n], \tag{4.6}$$

$$\hat{\theta}[m,n] = \beta\,\hat{\theta}[m-L,n] + (2-\beta)\,\theta[m-L,n] - \theta[m-2L,n], \tag{4.7}$$

with $\alpha = 0.6$ and $\beta = 0.4$ [11, Table 4.4], and $\hat{\theta}$ wrapped into $[-\pi, \pi]$.

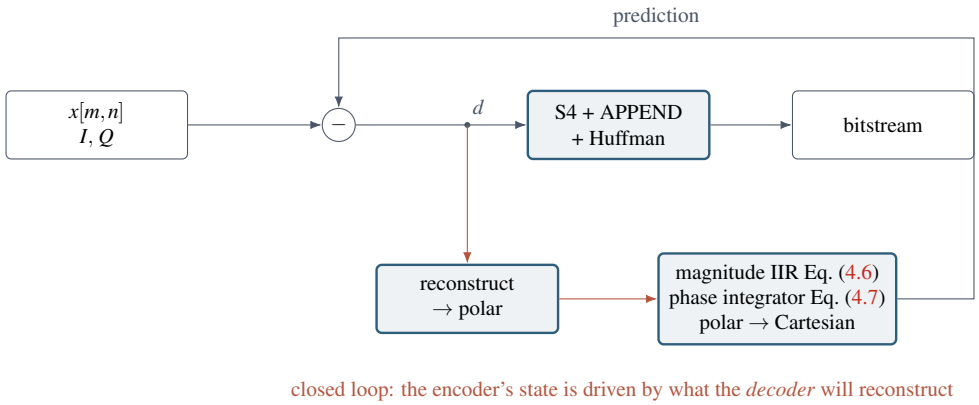


Figure 4.1: DRHE as implemented. Prediction happens in the polar domain with two filters matched to the expected behaviour of a constant-velocity target; the prediction is converted back to Cartesian before subtraction. The red path is the closed-loop correction of Section 6.4: the encoder drives its state from what the decoder will reconstruct, not from the original sample.

Equation (4.6) is a first-order infinite-impulse-response smoother: it blends the previous prediction with the previous observation. For a constant magnitude it converges to that constant, and the α term gives it some immunity to noise on any single ramp.

Equation (4.7) is more interesting. Setting $\beta = 0$ reduces it to $\hat{\theta}[m] = 2\theta[m-L] - \theta[m-2L]$, which is a *double integrator*: it extrapolates the last two phases along a straight line. For a phase advancing by a constant $\Delta\theta$ per ramp — exactly Equation (2.5) — that prediction is perfect. The β term mixes in the previous prediction to damp the response to noise.

The prediction is then converted back and subtracted, with int16 wraparound $\mathcal{W}(\cdot)$:

$$d_{\Re} = \mathcal{W}\big(x_{\Re} - \mathrm{round}(\hat{a}\cos\hat{\theta})\big), \tag{4.8}$$

$$d_{\Im} = \mathcal{W}\big(x_{\Im} - \mathrm{round}(\hat{a}\sin\hat{\theta})\big), \tag{4.9}$$

and $d_{\Re}$, $d_{\Im}$ are handed to the entropy coder of Section 4.2.

4.4.3 The streaming property

DRHE has a property that matters more than its compression ratio: it is *causal and streaming*. Each output depends only on the current sample and on state carried from earlier ramps of the same transmitter. Concretely, the implementation holds five floating-point values per (transmitter, receive channel, range bin) — the magnitude prediction, the previous magnitude, the phase prediction, and the previous two phases — and nothing else. That is 120 KiB of state for this radar, against 384 KiB for the frame itself.

It therefore needs no frame buffer, and it emits its first compressed bits as soon as the first

sample arrives. For a compressor sitting between an FFT and a memory bus, that is exactly the right shape. Section 10.1 returns to this when weighing DRHE against LPC.

4.5 LPC: linear prediction with Yule–Walker coefficients

4.5.1 Autoregressive modelling

The second algorithm takes a more general approach. Rather than assuming a particular signal shape and hand-designing filters for it, it *fits* a model to the data.

A second-order autoregressive (AR) model says each sample is approximately a linear combination of the previous two:

$$\hat{x}[m] = a_1 x[m-L] + a_2 x[m-2L], \tag{4.10}$$

Given a sequence, the coefficients that minimise the mean-square prediction error satisfy the Yule–Walker equations, which for order two are

$$\begin{bmatrix} R_0 & R_1 \\ R_1 & R_0 \end{bmatrix} \begin{bmatrix} a_1 \\ a_2 \end{bmatrix} = \begin{bmatrix} R_1 \\ R_2 \end{bmatrix}, \tag{4.11}$$

where $R_k = \sum_m x[m]\,x[m-kL]$ is the autocorrelation at lag kL, summed over every ramp of the frame. Solving directly,

$$a_1 = \frac{R_1 (R_0 - R_2)}{R_0^2 - R_1^2}, \qquad a_2 = \frac{R_0 R_2 - R_1^2}{R_0^2 - R_1^2}. \tag{4.12}$$

4.5.2 Stability

An AR model whose poles lie outside the unit circle is unstable: prediction errors grow rather than decay. For order two the stability triangle gives the conditions

$$|a_2| < 1 \qquad \text{and} \qquad |a_1| < 1 - a_2. \tag{4.13}$$

If either fails, or if the denominator $R_0^2 - R_1^2$ vanishes, the implementation sets $a_1 = a_2 = 0$, which degrades gracefully to “predict zero” — i.e. code the raw value. On the real data used here the test never fires: 100.00 % of coefficient sets pass.

4.5.3 Granularity and side information

The model is fitted per *(range bin, receive channel, real/imaginary part)*, across all ramps of the frame. That granularity is a deliberate compromise: fitting one model to the whole frame would ignore that different range bins contain targets at different velocities, while fitting per (bin, ramp) would have no data to fit to.

The coefficients are side information: the decoder cannot reproduce them, so they must be transmitted. Two 16-bit values per (bin, channel, part) is

$$128 \times 4 \times 2 \times 2 \times 16 = 32{,}768 \text{ bits} \tag{4.14}$$

per frame, which is 1.04 % of the uncompressed frame. That is a small but real tax, and Section 4.3 shows why fitting more coefficients — one set per transmitter, for instance — makes it the dominant cost.

4.5.4 The cost: LPC is not causal

R_0, R_1 and R_2 are sums over *all* ramps of the frame. The coefficients therefore cannot be known until the frame is complete, and the compressor cannot emit a single bit before then. Where DRHE holds five values per (channel, bin), LPC must buffer the entire frame: $4 \times 192 \times 128 \times 32 = 3{,}145{,}728$ bits, 3.0 Mibit.

This is the fundamental trade between the two algorithms, and Section 10.1 argues it matters more than the difference in compression ratio.

4.5.5 Coefficient quantisation

The Yule–Walker solution of Equation (4.12) is computed in floating point and then quantised before use, because the decoder must use exactly the coefficients the encoder used and must be able to read them from the bitstream. a_1 is stored as `ap_fixed<16,2>` (range $[-2, 2)$, step 2^{-14}) and a_2 as `ap_fixed<16,1>` (range $[-1, 1)$, step 2^{-15}), both with truncation towards $-\infty$, which is the HLS default. The prediction is then formed in fixed point and rounded to an integer before the residual is taken, so encoder and decoder agree to the bit. The MATLAB model reproduces this quantisation; the independent Python implementation of Appendix D does too, and matches the HLS bit counts exactly on all 50 frames.

4.5.6 What this LPC is not

Kiem's thesis mentions an "LPC-Huffman" method [27] that he judged less promising than DRHE and did not implement. That method predicts each raw ADC sample from earlier samples of the *same chirp* — along fast time, before the range FFT — and codes the residuals with a Huffman code derived from the data. The LPC here predicts each range-FFT code from the same range bin in earlier *ramps* — along slow time, after the range FFT, exactly where DRHE predicts — and uses DRHE's own entropy coder. It was designed that way on purpose: by keeping DRHE's input, axis and coder and changing only the predictor, any difference in compression ratio can be attributed to the predictor alone. Section 9.3 returns to how the two methods called LPC compare.

4.6 The Huffman dictionary

The entropy coder needs one design-time constant: the sixteen codeword lengths $\ell_0, \ldots, \ell_{15}$, one per S_4 category. Two dictionaries are used in this thesis.

The original dictionary. Kiem derived his dictionary once, from the S_4 statistics of one of his own recordings, and fixed it for simulation and hardware. Its two-bit codewords go to categories 2, 3 and 4, so it is matched to residuals of magnitude 2 to 15 — the size his single-transmitter data produced. No value can be coded in fewer than four bits, which caps the compression ratio of any frame at $32/(2\times 4) = 4.0$.

The re-derived dictionaries. The prediction errors of DRHE and LPC at lag n_{Tx} are smaller than that: most fall in categories 0, 1 and 2 (Section 7.4). A new set of codeword lengths was therefore derived for each algorithm, by building a Huffman code [15] on the S_4 category counts of the algorithm's own residuals over frames 0–4 only. One is added to every count before the code is built, so that categories never seen in training still receive a codeword and the code remains lossless on any input. The lengths are assigned as canonical codes and, like the original, bit-reversed for the least-significant-bit-first packer:

$$\ell^{\mathrm{orig}} = [4,3,2,2,2,5,6,7,8,9,10,11,14,14,13,12],$$

$$\ell^{\mathrm{DRHE}} = [3,2,1,4,5,6,7,8,9,10,11,14,14,13,13,13],$$

$$\ell^{\mathrm{LPC}} = [3,1,2,4,5,6,7,8,9,10,13,13,13,13,12,12].$$

Table 4.5: The two lossless algorithms compared on structure, before any measurement.

	DRHE	**LPC**
Prediction domain	polar (magnitude, phase)	Cartesian, I and Q independently
Model	fixed IIR filters, α, β chosen a priori	second-order AR, coefficients fitted per frame
Prediction lag	n_{Tx}	n_{Tx} and $2n_{\mathrm{Tx}}$
Causal?	yes — streaming, first output immediate	**no** — needs the whole frame before any output
Compressor state	5 values per (TX, channel, bin)	full frame, 3.0 Mibit
Side information	none	32,768 bits/frame (1.04 %)
Per-sample arithmetic	4 floating-point transcendentals (`sqrt`, `atan2`, `sin`, `cos`)	2 fixed-point multiply–accumulates
Per-frame arithmetic	—	one division per (channel, bin, part)
Entropy back end	identical: S_4 + APPEND + Huffman dictionary	

DRHE's one-bit codeword goes to category 2 and LPC's to category 1, so their commonest residuals cost three bits and two bits respectively, rather than four. Because only the contents of a sixteen-entry table change, a re-derived dictionary costs nothing in hardware; the FPGA cores of Chapter 6 use them. Both dictionaries are listed in full in Appendix A, and Section 7.4 reports the difference they make, including on the 45 frames they were not trained on.

4.7 The two algorithms side by side

Chapter 5

Evaluation Method

5.1 Two stages

The evaluation runs in two stages, each answering one question.

Stage 1 — which methods are worth building? All eight methods of Section 4.1 are run in MATLAB over the same 50 consecutive ColoRadar frames with all 16 receive channels. Each is scored on compression ratio and on what it does to target detection. A method goes forward only if it is lossless — it loses no detections beyond those of the FX16 step every method shares — and compresses by more than three, which leaves a margin over the 2.4–2.7 that a Gigabit Ethernet link needs (Section 2.8 and Chapter 3).

Stage 2 — where does the compression come from, and what does it cost in hardware? The selected methods, DRHE and LPC, are evaluated on the four-receiver frames that the hardware processes. Their compression ratio is broken down into what the entropy coder achieves alone, what prediction adds, and what a re-derived dictionary adds. They are then implemented, with decoders, in hardware (Chapter 6), verified bit for bit at every level of the ladder of Section 5.4, and measured on the board for time per frame, latency and chip resources.

5.2 The reference signal chain

A ColoRadar frame is read and reshaped into a complex cube of (fast time $\times$ ramp $\times$ receiver) with the TDM flattening of Equation (2.7), and the per-ramp mean is subtracted to remove

the DC and transmitter-leakage term. A Hann window is applied along fast time, and an N_s-point range FFT normalised by N_s. Only the positive half of the spectrum is kept, giving 128 range bins; the input is complex, so this is a choice to keep positive ranges, not a consequence of symmetry. The result is scaled by Equation (2.11) and rounded to `int16`. Four receive channels are exported to the hardware path, because four complex `int16` samples make one 128-bit AXI4-Stream beat; one such frame is 393,216 B (Figure 5.1).

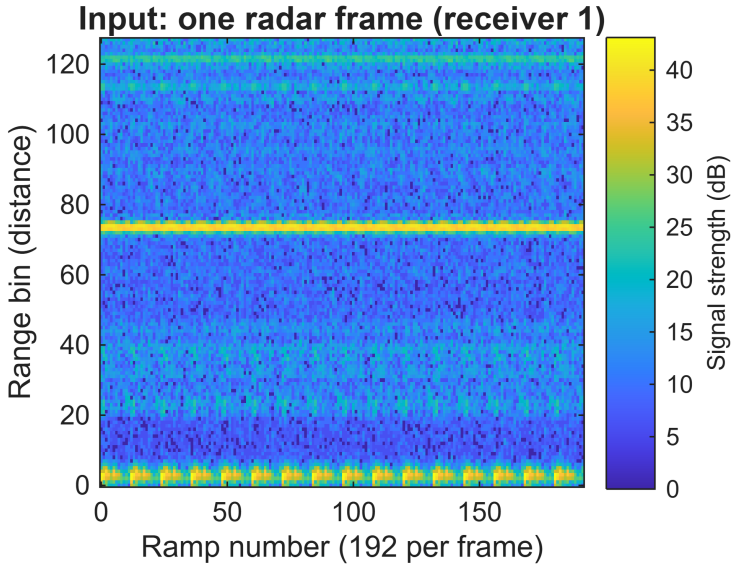


Figure 5.1: One input frame as the compressor receives it: receiver 1 of ColoRadar frame 0 after the range FFT and FX16 quantisation, 128 range bins by 192 ramps. Strong returns appear as horizontal bands at fixed range; most of the frame is low-level noise, which is why it compresses.

For detection, the reconstructed data goes through a 192-point Doppler FFT across all ramps in recorded order (the transmitter-interleaved axis of Section 2.2.1), non-coherent integration over the 16 receive channels, and a detector whose threshold is the mean of the lowest 75 % of magnitudes plus 15 dB, with 8-connected local-maximum suppression. There is no angle FFT. None of this affects a lossless result, which returns the input exactly; it matters only for the lossy comparison points.

5.3 Metrics

Compression ratio.

$$\mathrm{CR} = \frac{B_{\mathrm{in}}}{B_{\mathrm{out}}}, \qquad B_{\mathrm{in}} = N_{\mathrm{ramps}} \times N \times N_{\mathrm{RX}} \times 32, \tag{5.1}$$

with B_{out} the compressed length *including* the padding of the last 256-bit word. Ratios are computed per frame and reported as the mean over 50 frames.

Losslessness. The maximum absolute difference between input and reconstruction over every real and imaginary component must be zero. There is no tolerance.

Detection. With D_{ref} the detections of the double-precision reference and D_{test} those of the reconstructed data,

$$\text{FN}\% = \frac{|D_{\text{ref}} \setminus D_{\text{test}}|}{|D_{\text{ref}}|}, \qquad \text{FDR}\% = \frac{|D_{\text{test}} \setminus D_{\text{ref}}|}{|D_{\text{test}}|}. \tag{5.2}$$

The first is the share of targets missed. The second is labelled a false positive rate in Kiem's thesis, but it normalises by the test detections, so it is a false discovery rate. Matching is cell by cell with no positional tolerance, and the 15 dB threshold is not swept. FX16, DRHE and LPC reconstruct identical data, so their detection metrics are identical by construction: for the lossless methods they measure the cost of the FX16 step alone.

Baselines. Every predictive result is reported against a *no-prediction baseline*, the FX16 codes fed straight into the same entropy coder; a predictor that does not beat it is not helping. The order-0 entropy of Equation (2.9), evaluated on the residuals, separates what the predictor achieves from what the dictionary wastes.

Timing. *Time per frame* runs from the start of a frame's DMA transfer to the last output word written. *Throughput* is input bits per frame divided by it. *Latency* runs from the last input word read from memory to the last output word written back, measured by the board firmware with a hardware timer; this is Kiem's definition, and it measures how long the compressed data lags the raw data.

5.4 The verification ladder

A compressor that is *nearly* lossless is useless, so every design is taken down five levels, each checked against the one above, so that a discrepancy localises to one transition (Figure 5.2).

MATLAB models the chain in double precision and gives the exact bits each algorithm should produce. **HLS C-simulation** compiles the hardware C++ as software, on stimulus ex-

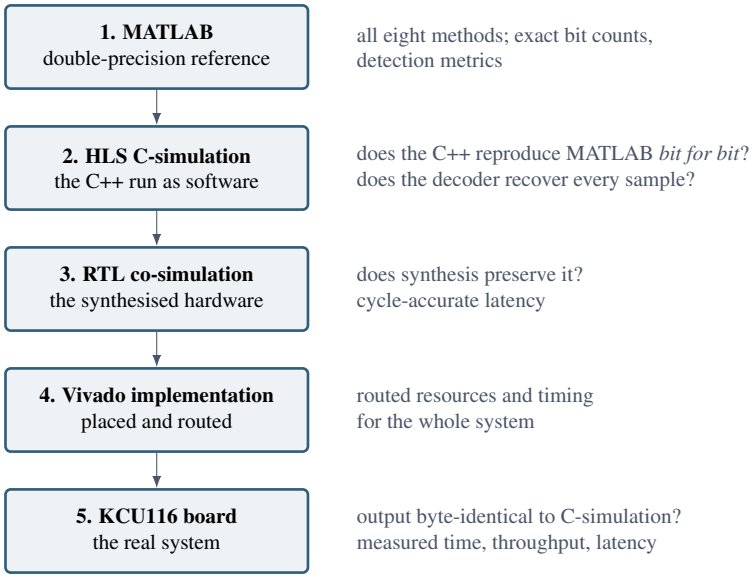


Figure 5.2: The verification ladder. Each level is checked against the one above it, so a mismatch localises to a single transition rather than to "somewhere in the design".

ported from MATLAB as `int16`, and checks over all 50 frames that it produces exactly MATLAB's bit counts and that the decompressor recovers every sample. **RTL co-simulation** runs the same testbench on the generated Verilog for one frame, to check that synthesis preserved behaviour and to count cycles. **Vivado implementation** places and routes the complete board system and gives real resources and timing. **The board** streams all 50 frames through the cores and compares the output byte for byte with the C-simulation.

5.5 Edge-case testing

Real data exercises the typical case well and the extreme case not at all. The ColoRadar residuals span roughly $[-514, +416]$, which is entirely inside category 10 of a 16-category code. Everything above that is never exercised by real data.

An eight-case synthetic testbench was therefore built (Table 5.1), generating its stimulus internally so that it runs identically in C-simulation and co-simulation.

The suite exercises the parts of the design that real data never reaches, including the two situations handled by the robustness measures of Section 6.4, neither of which can arise on ColoRadar data.

Table 5.1: The synthetic edge-case suite. Each case has its own allowed reconstruction error — zero for all but the three that can produce the value −32768 and therefore exercise the saturation described in Section 6.4. Uniformly random `int16` input produces a residual of −32768 by chance, so it is one of them.

Case	What it exercises	Allowed MaxDiff
all-zeros	the $S_4 = 0$ path and the best case	0
dc-constant	a perfectly predictable sequence	0
random-int16	the worst case: incompressible input	1
tone-192ramp	a periodic sequence the AR model should fit	0
tone-1ramp	a sequence too short to fit	0
max-amplitude	full-scale values, and 128-bit correlations	0
max-alternating	±32767 alternating: the largest residuals	1
int16-min	−32768 specifically	1

5.6 An independent implementation

As a final check that the compression figures are right — not merely consistent between MATLAB and C++, which share a specification and could share a misunderstanding — the whole compression chain was written a third time, independently, in Python and NumPy, from the descriptions in Chapters 4 and 5 without reusing any code. It reads raw ColoRadar frames, applies the TDM flattening, DC removal, window, range FFT and FX16 scaling, runs DRHE and LPC with the same coefficient quantisation as the hardware, and counts the bits of the S_4, APPEND and Huffman coder with and without 256-bit padding. With the original dictionary it reproduces the HLS bit counts of both algorithms exactly on every one of the 50 frames, and with the re-derived dictionaries it reproduces the hardware's residuals exactly on the frames checked. It is also the source of the no-prediction baseline, the entropy figures and the LPC coefficient statistics reported in Chapter 7. Appendix D lists what was checked.

5.7 Tools

All MATLAB work used R2026a with the Signal Processing Toolbox. All hardware work used AMD Vitis HLS 2026.1 and Vivado 2026.1, targeting the `xcku5p-ffvb676-2-e` device of the KCU116 at a 100 MHz clock, with XSIM for co-simulation and Vitis for the MicroBlaze firmware. The board is controlled from a host PC over JTAG with `xsdb` scripts, and results are checked with Python. Appendix C lists the source files and build commands.

Chapter 6

Hardware Implementation

Both algorithms were implemented as a compressor core and a matching decompressor core in C++ for Vitis HLS, and built into a test system on an AMD KCU116 evaluation board. This chapter describes the cores, the measures that keep them exactly lossless on any input, and the board system that runs and measures them.

6.1 Common interface and bit packing

All four cores share one interface. A compressor accepts AXI4-Stream input at 128 bits per beat — four receive channels, each a 16-bit real and a 16-bit imaginary part — and emits a compressed AXI4-Stream at 256 bits per beat, with TLAST marking the end of the frame. A decompressor does the reverse. Frame dimensions (range bins, ramps, receive channels and n_{Tx}) arrive on an AXI4-Lite control interface, so there is no header in the bitstream.

Both compressors share the bit packer. Codewords accumulate in a 512-bit register, least-significant-bit first, and a 256-bit word is written out whenever at least 256 bits are waiting, with one final partial word at the end of the frame carrying TLAST. If the frame ends exactly on a word boundary there is no partial word, so an all-zero word is sent to carry TLAST; without it the DMA and the decoder would wait for an end of frame that never comes. No ColoRadar frame ends on a boundary — the case was found by the synthetic edge-case frames of Section 6.5, and the fix leaves the output for all 50 ColoRadar frames byte-identical. Flushing per frame rather than per ramp avoids wasting up to 255 bits per ramp, about 2.9 % of the ratio. The Huffman codewords are stored bit-reversed relative to their usual most-significant-bit-first form, because the packer is least-significant-bit first; the reversed code set is still prefix-free, so

a decoder can match candidates from the low end of its buffer with no ambiguity. Compressor and decompressor read the same sixteen-entry table. Each core was built with both dictionaries of Section 4.6; the two builds differ only in the contents of this table, and have the same schedule and timing. Both were run on the board (Chapter 7); the compressors' cycle counts agree to within 13 cycles per frame.

6.2 DRHE compressor

6.2.1 State and parallelism

DRHE holds five `float` state arrays — the magnitude prediction, the previous magnitude, the phase prediction and the previous two phases — each indexed by (transmitter, receive channel, range bin). The transmitter index is the ramp number modulo n_{Tx}, which is all it takes to put the predictor on the per-transmitter axis of Section 4.3: indexing by transmitter changes *which* address is read, not *when*, so the schedule is the same as for a lag-1 design. Each transmitter's history starts on its own first ramp, so the state is reset while $m < n_{\mathrm{Tx}}$. Frames are independent: nothing carries over from one frame to the next, so a frame lost in transit cannot corrupt its successors.

The channel dimension is completely partitioned, so the four receive channels are four parallel copies of the datapath inside one pipelined iteration of the sample loop. The state occupies 40 block RAMs (36 Kb).

6.2.2 Where the logic goes

Per sample and per channel, DRHE evaluates `sqrt` and `atan2` to convert the reconstructed sample to polar form, and `sin` and `cos` to convert the prediction back — four floating-point transcendental functions, for each of four channels, on every one of the 24,576 beats of a frame. These operators dominate the design: they account for about two thirds of its LUTs and all 132 of its DSP slices, and the two arctangent units alone are about a third of the logic. Kiem's design uses fixed point for the same functions; Chapter 8 measures what that choice costs.

6.2.3 Initiation interval

DRHE runs at II = 2, and the reason has nothing to do with arithmetic. Each state array is partitioned by channel, so each channel's history is one dual-port block RAM. Every iteration reads a value and writes the updated one — two accesses, which two ports can serve. On the first ramps of a frame the iteration also writes a reset value: three accesses against two ports, which forces the tool to serialise, and the loop settles at II = 2 for the whole frame. Reading the state as zero on the first ramps instead of writing zeros to it reaches II = 1. It was not adopted, because it doubles the floating-point arithmetic (units can no longer be shared between two channels) and the sensor does not need the throughput: the II = 2 design already runs many times faster than the radar produces frames (Section 7.7).

6.3 LPC compressor

6.3.1 Three passes

LPC cannot stream, so its structure is explicitly multi-pass (Figure 6.1):

1. **Ingest.** Read the whole frame into a buffer.
2. **Correlate, solve and residual**, once per (channel, bin): compute R_0, R_1 and R_2 at lags 0, n_{Tx} and $2n_{\mathrm{Tx}}$ from the buffer, solve Equation (4.12), then walk the ramps again computing residuals and writing them back into the buffer *in place*.
3. **Emit.** Write the coefficient table, then the coded residuals.

Rewriting the buffer in place means one 3.0 Mibit memory serves both purposes rather than two.

6.3.2 Reaching back n_{Tx} and $2n_{\mathrm{Tx}}$ ramps

Both the correlation and the residual loops need $x[m-n_{\mathrm{Tx}}]$ and $x[m-2n_{\mathrm{Tx}}]$. A shift register of depth $2n_{\mathrm{Tx}} = 24$ would move 24 entries every iteration. Instead a *ring* of exactly $2n_{\mathrm{Tx}}$ entries gives both taps for free. At ramp m, let $k = m \bmod 2n_{\mathrm{Tx}}$. Slot k still holds the value written at ramp $m-2n_{\mathrm{Tx}}$, because it is about to be overwritten and has not been touched since; and slot $(k+n_{\mathrm{Tx}}) \bmod 2n_{\mathrm{Tx}}$ holds the value from ramp $m-n_{\mathrm{Tx}}$. So:

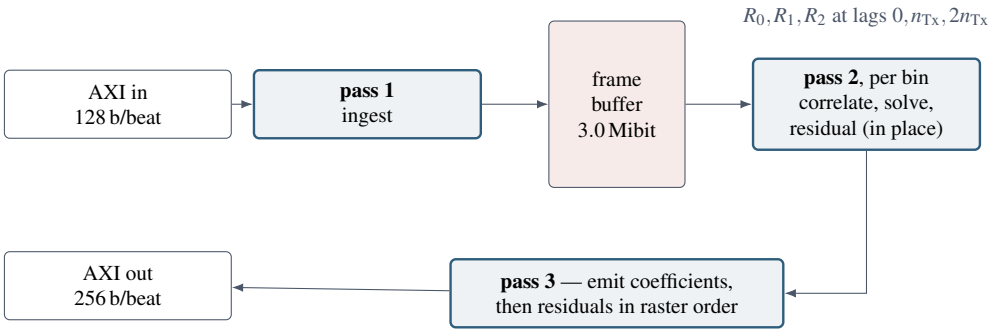


Figure 6.1: The LPC compressor. Because the Yule–Walker coefficients depend on the whole frame, the compressor must buffer it; each (channel, bin) is then correlated, solved and turned into residuals, which overwrite the buffer in place so that only one memory is needed. Emitting residuals in the original raster order is what lets the *decompressor* avoid a frame buffer entirely.

$$\begin{array}{ll} \text{read } x[m-2n_{\mathrm{Tx}}] & \text{from slot } k \\ \text{read } x[m-n_{\mathrm{Tx}}] & \text{from slot } (k+n_{\mathrm{Tx}}) \bmod 2n_{\mathrm{Tx}} \\ \text{write } x[m] & \text{to slot } k \end{array}$$

Nothing is shifted; the cost is $2n_{\mathrm{Tx}}$ registers and two multiplexers. Every loop in the design reaches $\mathrm{II} = 1$.

6.3.3 Where the cycles go

A frame takes 325,366 cycles on the board. Ingest and emit are 24,576 cycles each, one per beat. The remaining 276,000 or so are the per-bin pass, which runs two short pipelined loops — correlation and residual — for each of the 512 (channel, bin) pairs. A pipelined loop pays its pipeline depth once on entry before reaching steady state, about 71 cycles for the correlation loop and 56 for the residual loop, so 512 invocations add roughly $512 \times (71+56) \approx 65{,}000$ cycles of fill that a trip-count model does not show. Merging the two loops, or processing several bins per invocation, would recover most of it; overlapping the three passes across frames would bring the time per frame down further. Neither was necessary for this thesis, and both are listed as future work.

6.3.4 Fixed-point choices

Coefficients are quantised to `ap_fixed<16,2>` for a_1 (range $[-2,2)$) and `ap_fixed<16,1>` for a_2 (range $[-1,1)$), which follows from the stability conditions of Equation (4.13). Over 50 frames this quantisation costs nothing measurable: the residuals are identical to the bit to those of double-precision coefficients.

The autocorrelations are accumulated as exact 64-bit integers rather than in floating point. $R_0^2 - R_1^2$ is a difference of two nearly equal quantities whenever the data is strongly correlated — precisely the regime the predictor exists for — so computing it in floating point would lose most of its significant digits exactly when accuracy matters most. The numerators and the determinant are formed exactly as 128-bit integers before the single division.

6.3.5 An asymmetry that favours the decoder

Pass 3 emits the coefficient table first — the decoder needs all of it before it can reconstruct anything — and then the residuals *in the raster order the samples arrived in*, not in the (channel, bin, ramp) order pass 2 produced them. The decompressor therefore needs no frame buffer at all: it holds the coefficient table and the last $2n_{\mathrm{Tx}}$ reconstructed samples of each (channel, bin), and writes each output sample as its residual arrives. The reordering is paid for once, by the side that already had to buffer. LPC's non-causality is a compressor-side cost only.

6.4 Exactly lossless on any input

Real radar data exercises only the lower part of the entropy coder's range (Section 5.5). Two situations that real data never produces, but synthetic input does (Section 5.5), are handled explicitly so that both designs are lossless on *any* input.

6.4.1 The one value the code cannot represent

Category 15 of the S_4 code covers $|v| \in [16384, 32767]$: counting both signs, exactly the 2^{15} patterns its fifteen payload bits can hold. But 16-bit arithmetic can produce $v = -32768$, for which Equation (4.3) gives $A(-32768) = -1$, which truncated to fifteen bits is `0x7FFF` — the same payload as $v = +32767$. The two would be indistinguishable on the wire.

Three remedies were considered: extend the code with a category 16, which changes the bitstream format; reject the input; or saturate -32768 to -32767. Saturation was chosen. It costs at most one least-significant bit on the affected sample, leaves the format untouched, and never fires on real data.

Saturation alone is not enough, though, and this is where the closed-loop prediction of Section 2.4.4 becomes concrete. If the encoder drove its predictor from the *original* sample while

the decoder drives its own from the *reconstructed* one, a single saturated sample would make them diverge, and because DRHE's filters are recursive that divergence would carry through every remaining ramp. The encoder therefore reconstructs exactly what the decoder will see and updates its state from that:

```
int diff_re = wrap_int16(re - pred_re);
if (diff_re == -32768) diff_re = -32767;      // saturate
int recon_re = wrap_int16(diff_re + pred_re); // what the decoder will see
float curr_mag   = hls::sqrt(recon_re*recon_re + recon_im*recon_im);
float curr_phase = hls::atan2(recon_im, recon_re);   // state from recon
```

On real data the reconstruction always equals the original, so the closed loop changes no output bit. Its cost is in the schedule: the reconstruction sits ahead of the polar conversion on the dependency chain, and accounts for 26 of the pipeline's 58 stages.

6.4.2 Wide-integer conversion in the coefficient solver

The LPC solver forms its numerators and determinant as 128-bit integers. The conversion of such a value directly to `float` is not correct above 2^{64} in this toolchain. All three operands are therefore normalised by one common arithmetic right shift into 62 bits before the division, with the shift chosen from the largest of them:

$$k = \max\big(\mathrm{msb}|D|,\ \mathrm{msb}|N_1|,\ \mathrm{msb}|N_2|\big) - 61, \tag{6.1}$$

clamped at zero. Because numerator and denominator are shifted alike, the quotient is unchanged. On ColoRadar the correlations reach only 44 bits, so the shift is zero; on full-scale synthetic input it is what keeps the coefficients correct.

6.5 Decompressors

A decompressor exists for each algorithm, for two reasons. It is what makes bit-exact verification possible — every frame is compressed, decompressed and compared — and it completes the pipeline a vehicle would need, with the encoder at the sensor and the decoder at the central processor. Each decompressor is built twice from the same C++ source: as an FPGA core, and as a PC program compiled through Vitis C-simulation, which uses the same bit-accurate models of the floating-point functions.

6.5.1 Why decoding is the harder half

Entropy decoding is inherently serial: the length of a codeword is known only once it has been matched, and the next codeword starts where it ends. A DRHE decoder that keeps pace with the compressor — eight values (four channels, real and imaginary) per two clocks — needs eight chained codeword matches and very wide variable shifters per cycle, plus four copies of the floating-point predictor. HLS estimates such a design at about 182,000 LUTs. HLS overestimates (by about three times for the serial decoder below), but even a third of that would be four times the serial decoder's logic. Keeping pace is also unnecessary: a decoder that takes 2 ms per frame is a hundred times faster than the radar produces frames. The DRHE decoder was therefore designed to be small rather than fast.

6.5.2 DRHE: a two-stage serial decoder

The DRHE decompressor is therefore a dataflow pair of stages connected by a stream:

- **Unpack** decodes one value per clock. It cuts a 29-bit window from two 256-bit input words, matches all sixteen codewords against it in parallel, extracts the payload, and reads the next input word just in time, stopping at TLAST.
- **Rebuild** runs one floating-point predictor that handles one receive channel per step at II = 2 (limited by the ports of the sine/cosine tables), keeps its state in block RAM, and uses exactly the same expressions as the compressor so that its arithmetic matches bit for bit.

The two stages are balanced at about 196,608 clocks per frame, roughly 2 ms at 100 MHz; on the board it takes 196,881 (Section 7.7). HLS estimated 42,846 LUTs; after placement and routing the core uses 14,155.

Bit-for-bit agreement of the floating-point arithmetic is not optional for DRHE. The predictor feeds back on itself, so a decoder whose `atan2` or `sin` differs from the encoder's in the last bit will drift. A double-precision Python decoder written as a cross-check is exact for LPC but drifts by 1 to 6 least-significant bits on 8 of the 50 DRHE frames; the decoder built from the HLS source, which the FPGA core shares, is exact on all of them.

6.5.3 LPC: the same two stages

The LPC decompressor has the same structure as the DRHE decoder, a dataflow pair of stages joined by a stream:

- **Unpack** first reads the coefficient table, one 64-bit entry per clock, and passes it on. It then decodes one residual per clock with the DRHE decoder's matcher: a 29-bit window cut from two 256-bit input words, all sixteen codewords compared with it at once, the payload extracted, and the next input word read just in time, stopping at TLAST.
- **Rebuild** stores the coefficients and then reconstructs one complex sample per clock: two fixed-point multiply–accumulates for the real part and two for the imaginary part, the same rounding, and the same 16-bit wrap as the compressor. It holds the last $2n_{\mathrm{Tx}}$ reconstructed values of each (channel, bin) in a ring, as the compressor does. The rings are in distributed (LUT) RAM, split so that both past values are read and the new one written in the same clock.

Unpack is the slower stage, at one real value per clock, so a frame takes about 196,608 clocks plus 512 for the coefficient table — about 2 ms, the same as DRHE. Every loop is scheduled at II = 1, and the decoder needs no frame buffer and no block RAM. After routing it uses 15,212 LUTs (just over half of them as distributed RAM for the rings), 7,207 flip-flops and 23 DSP slices.

The first LPC decoder built was simpler: it tried the sixteen codewords one after another in category order and stopped at the first match. It was exact, but its speed depended on the data. On the board it took 15.4 ms per frame, 7.6 to 8.3 clocks per value, and 20,034 LUTs after routing — slower and larger than the parallel matcher, which therefore replaced it. In C-simulation the new decoder's output is bit-identical to the first one's on all 50 ColoRadar frames and on eight synthetic edge-case frames: all zeros, constant $+32767$ and -32768, alternating full scale, uniform random values, small noise, a random walk along the ramps, and small noise with full-scale spikes.

6.6 Range FFT on chip

The compressors take FX16 range-FFT codes, and the method study takes those codes from MATLAB (Section 5.2). To check that no result depends on that, and to let the board start from

raw ADC samples as a sensor would, the MATLAB chain was also written in Vitis HLS as a range-FFT front end. For each ramp of 256 complex samples on each of the four channels it applies the symmetric Hann window, a 256-point FFT, removes the ramp mean, keeps bins 0 to 127 and rounds to FX16 exactly as Equation (2.11) does, saturating at the `int16` limits.

Structure. The core is a dataflow pipeline of ten processes: the window, eight radix-2 decimation-in-frequency stages, each holding one ramp of the four channels, and an output stage that undoes the bit-reversed order of the bins and scales. All four channels are transformed in parallel, so one 128-bit beat of raw samples enters per step; the core takes about 1.5 clocks per beat, 75,175 cycles per frame in RTL co-simulation.

The ramp mean. MATLAB subtracts each ramp's mean before the window, but a streaming core knows the mean only once the last sample of the ramp has arrived. Since the FFT is linear, the core instead accumulates the mean μ while the ramp streams in and subtracts $\mu W[k]$ from each bin $X[k]$ at the output, where W is the FFT of the quantised window, held in a table. This adds a multiply–subtract per bin and no buffering.

Arithmetic. The datapath is fixed point: 36-bit values with 26 integer bits through the FFT, an 18-bit window and 18-bit twiddle factors. The final scaling by $1/63.75$ rounds half away from zero, as MATLAB's `round` does. Fixed-point twiddles cannot match double precision exactly, so a value that lies within a hair of a rounding boundary can round the other way; Section 7.9 measures how often.

In the system. The front end sits between the input DMA and the compressor. An AXI4-Stream broadcaster also sends its output to a third DMA, `dma_f`, which writes the FX16 codes to DDR4 so that the host can check them against simulation. Post-route the front end uses 14,511 LUTs, 17,142 flip-flops, 64.5 block RAMs and 296 DSP slices, most of them in the butterflies' complex multipliers. These builds are called `_fe_full`.

6.7 The board test system

The cores run in a complete system on an AMD KCU116 evaluation board, whose Kintex UltraScale+ `XCKU5P` provides 216,960 LUTs, 433,920 flip-flops, 480 block RAMs (36 Kb) and

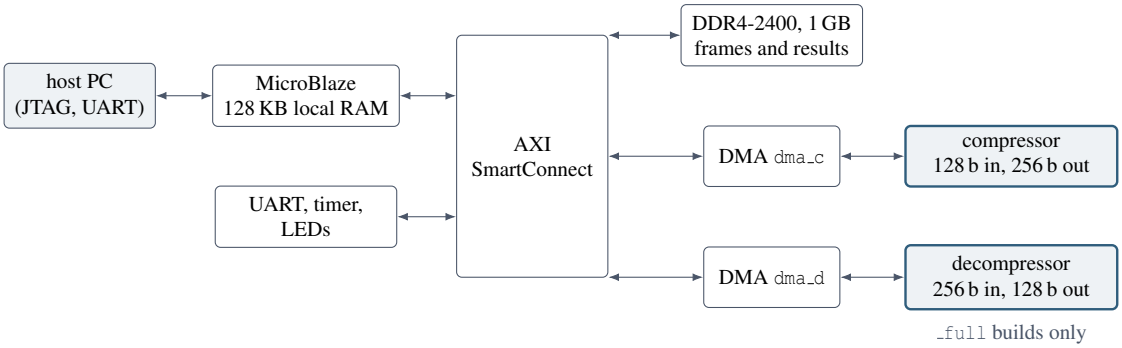


Figure 6.2: The board test system. The host places the frames in DDR4 over JTAG; the MicroBlaze starts the DMAs, times each frame and writes a result record; the host then reads the results and compressed frames back. Everything runs at 100 MHz.

1,824 DSP slices (Figure 6.2).

Hardware. A MicroBlaze processor with 128 kB of local memory controls the system. A DDR4-2400 controller gives access to 1 GB of board memory, which holds the input frames, the compressed frames, the reconstructed frames and the results. An AXI DMA (`dma_c`) streams each frame from DDR4 through the compressor and writes the compressed stream back. In the `_full` build a second DMA (`dma_d`) streams the compressed frame through the decompressor and writes the reconstruction back, so the board compresses, decompresses and checks every frame by itself. A UART, a hardware timer and LEDs complete the system, all on one AXI interconnect at 100 MHz. Each algorithm therefore has two bitstreams: a compress-only build and a `_full` build with the decompressor, and a third, `_fe_full`, adds the range-FFT front end of Section 6.6.

Firmware. For each frame the MicroBlaze configures the core's frame dimensions, starts the DMA transfers, and reads a hardware timer at the start, when the DMA's read channel reports the last input word read, and when its write channel reports the last output word written. From these it records the cycles per frame and the end-to-end latency of Section 5.3, together with the compressed length; the `_full` firmware also records the decompressor's cycles and latency and compares the reconstruction with the original word by word. Each frame's results are a 16-word record in DDR4.

Host. A script on the PC (`run_board.bat`, driving `xsdb`) programs the FPGA, loads the frames into DDR4 over JTAG, runs the firmware, and reads back the result records and compressed frames. For the compress-only build the PC then decompresses every frame with

Table 6.1: The six board builds, whole system (processor, DDR4 controller, DMAs and cores), post-route on the `XCKU5P` at 100 MHz, with the re-derived dictionaries; the original-dictionary builds also meet timing. BRAM is in 36 Kb blocks.

Build	LUT	BRAM	DSP	WNS (ns)
DRHE, compress only	61,011 (28 %)	104.5	138	+0.243
LPC, compress only	59,203 (27 %)	162.5	164	+0.174
DRHE, `_full`	81,573 (38 %)	144	184	+0.341
LPC, `_full`	80,986 (37 %)	179.5	187	+0.220
DRHE, `_fe_full`	101,383 (47 %)	211	480	+0.340
LPC, `_fe_full`	100,709 (46 %)	246.5	483	+0.374

the decompressor built from the HLS source. Finally `compare_results.py` checks every compressed frame byte for byte against the C-simulation output, checks every reconstruction against the original frame, and tabulates compression ratio, cycles and latency.

6.8 Building and timing closure

Each core is exported from Vitis HLS as an IP block and placed in a Vivado block design with the system above; a script builds each of the six bitstreams. HLS estimated both compressors below the 100 MHz target. Its estimate is the tool's own model of operator delays against a budget that reserves about a quarter of the period for clock uncertainty, not a timing analysis of a real circuit, and once placed and routed all six systems meet 100 MHz with positive slack (Table 6.1). Two system details were needed for that: a false path from the DDR4 calibration-complete signal to the processor reset, and the DDR4 interface at 2,400 MT/s.

Chapter 7

Results

This chapter follows the two stages of Section 5.1. It first reports the screening of all eight methods, then breaks down the compression of the two selected algorithms, and then reports their verification and their measured performance on the FPGA board. It closes with the two algorithms side by side.

7.1 Stage 1: screening eight methods

All eight methods were run over the same 50 consecutive ColoRadar frames, with all 16 receive channels and 9,934 reference detections. Table 7.1 and Figure 7.1 give the result.

Table 7.1: Stage 1: all eight methods, 50-frame average, 16 receive channels. "Missed" is the false negative rate and FDR the false discovery rate (Section 5.3); NF is the noise floor. DRHE and LPC predict at lag n_{Tx} and use the original dictionary.

Method	Type	CR	Missed (%)	FDR (%)	NF (dBFS)	SNR (dB)
FP16	reference	1.00	0.00	0.00	−105.64	35.50
FX16	reference	1.00	3.77	1.11	−105.37	35.36
BAQ	lossy	3.76	10.44	9.69	−105.08	35.08
RDVLE	lossy	1.12	24.01	4.62	−102.27	33.17
RDVLE+UDRE	lossy	2.56	28.73	4.44	−101.78	33.00
DRHE fp16	lossless*	1.04	0.05	0.01	−105.64	35.51
DRHE	lossless	**3.75**	3.77	1.11	−105.37	35.36
LPC	lossless	**3.70**	3.77	1.11	−105.37	35.36

*lossless with respect to its FP16 input, which is itself rounded from the reference.

Four observations decide what goes forward.

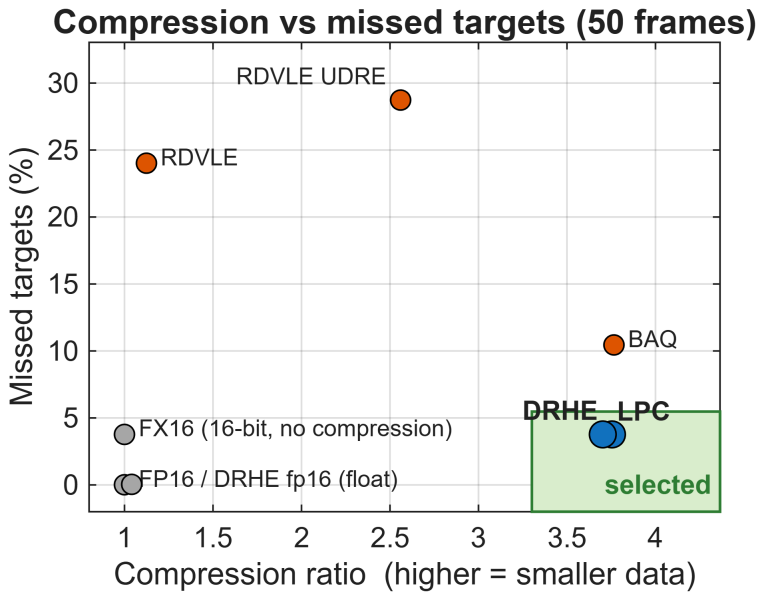


Figure 7.1: Compression ratio against missed targets for the eight methods. A good method sits to the right (small data) and at the bottom (no extra missed targets). Only DRHE and LPC reach the shaded region; the lossy methods pay for their ratio in missed targets, and storing the data as floating point barely compresses.

The lossless coders share FX16's detection metrics exactly. FX16, DRHE and LPC report identical missed-target rates, false discovery rates, noise floors and SNR. This is a consequence of losslessness rather than a coincidence: both coders reconstruct the FX16 codes bit for bit, so the detector sees identical data. What those rows measure is the cost of the *quantisation* step, 3.77 % of targets missed and 1.11 % of detections spurious, none of it the cost of compression.

The lossy methods pay for their ratio in detections. BAQ reaches 3.76, the same ratio as DRHE, but misses 10.44 % of targets instead of 3.77 %, and 9.69 % of what it reports is spurious against 1.11 %: nearly three times the miss rate and nearly nine times the false discovery rate, for no gain in ratio. RDVLE and RDVLE+UDRE are worse on both counts. In a system that brakes a car none of these trades is acceptable, which is the empirical form of the argument of Section 1.3.

Floating-point storage does not compress. DRHE applied to FP16 values almost removes the quantisation loss (0.05 % missed) but compresses by only 1.04. A half-precision mantissa keeps fine noise detail at every magnitude, so the prediction errors never settle near zero. The compressor's *input* must be integer; whether the arithmetic *inside* the predictor is floating point is a separate question, taken up in Section 10.5.

Two methods qualify. DRHE (3.75) and LPC (3.70) are the only methods that compress by more than three without adding a single missed target, and both clear the 2.4–2.7 a Gigabit Ethernet link needs with margin. Both were taken forward to Stage 2 and to hardware.

7.1.1 The cost of the quantisation step

Since FX16 accounts for all the detection loss of the lossless methods, it is natural to ask whether the 16-bit codes could be made more accurate. Table 7.2 compares four ways of producing them on one frame. The standard FX16 scaling, calibrated to a full-scale ADC signal, uses only 1.33 % of the `int16` range: the largest code in the frame is 436. Scaling each frame so that its own peak fills the container cuts the quantisation error by a factor of 67, and the companding curve of Deng and Huang [28] reduces it further.

Table 7.2: Four ways of producing the 16-bit codes, ColoRadar frame 0, all 16 receive channels. "relRMS" is the RMS error of the range-FFT values relative to their RMS; "RD error" is the RMS error of the range–Doppler map in dB; "int16 use" is the largest code as a fraction of 32,767; the last column is the lossless DRHE ratio on the resulting codes.

Quantiser	relRMS (%)	PSNR (dB)	RD error (dB)	int16 use (%)	DRHE CR
FP16 (reference)	0.0205	101.97	0.0049	—	—
FX16, fixed scaling	1.8636	64.44	0.4336	1.33	**3.80**
FX16, scaled to frame peak	0.0278	101.09	0.0054	89.20	1.18
FX16, companded [28]	0.0048	114.38	0.0011	94.28	0.78

The compression ratio moves the other way. With the container filled, lossless DRHE compresses by only 1.18; with companding, which spreads the values over the whole code range, it expands the data. Quantisation error and compression ratio are two sides of the same unused bits: the fixed scaling leaves about six of the sixteen bits as copies of the sign bit and rounds away everything below its coarse step, and stretching the data to fill the container turns those six bits into fine detail at the bottom — detail that is mostly receiver noise, which a lossless coder must keep. The quantisation error can be removed, but only by giving up almost all of the compression. The fixed FX16 scaling was therefore kept, as in Kiem's design, and Section 10.3 returns to what this means for the ratios.

7.2 Stage 2: the prediction axis

Stage 2 uses the four-receiver frames that the hardware processes. The first question is whether the predictors help at all, and the answer depends on the prediction lag (Section 4.3). Table 7.3 compares each predictor against the no-prediction baseline, at lag 1 and at lag n_{Tx}, all with the original dictionary.

Table 7.3: Compression ratio against the no-prediction baseline, 50 frames, four receive channels, original dictionary, 256-bit padding. The predictor rows are HLS C-simulation; all are lossless.

Configuration	CR	vs. no prediction
No prediction (Huffman coding only)	3.479	—
DRHE, lag 1 (adjacent ramp)	3.310	−4.9 %
LPC, lags 1, 2	3.385	−2.7 %
DRHE, lag n_{Tx}	3.751	**+7.8 %**
LPC, lags n_{Tx}, $2n_{\mathrm{Tx}}$	3.697	**+6.3 %**

Predicting from the adjacent ramp is worse than not predicting at all, for both algorithms: on this radar the adjacent ramp is another transmitter, and the difference between the two is set by the target's bearing, not its velocity. Predicting from the same transmitter turns a 4.9 % loss into a 7.8 % gain for DRHE, and a 2.7 % loss into a 6.3 % gain for LPC. The tail of the error distribution shows the same thing more starkly: the 99th percentile of |residual| is 80.6 with no prediction, 102.4 for DRHE at lag 1, and 15.8 for DRHE and 8.2 for LPC at lag n_{Tx}.

LPC loses less than DRHE on the wrong axis, and the reason is instructive. Yule–Walker chooses the coefficients that minimise the residual energy, and when the previous ramp is another transmitter there is little correlation to exploit, so the fitted coefficients shrink towards zero — and $a_1 = a_2 = 0$ *is* no prediction. Over the 51,200 coefficient sets of the 50 frames, the median fitted a_1 at lags $1, 2$ is -0.02 and half of all values lie between -0.13 and 0.09; at lags $n_{\mathrm{Tx}}, 2n_{\mathrm{Tx}}$ the median is 0.40, with a cluster near 1 from bins holding strong, slowly changing returns. DRHE's coefficients are fixed, so on the wrong axis it goes on extrapolating a phase trend that is not there. A fitted model's worst case is "predict nothing"; a fixed model's worst case is "predict wrongly".

Everything that follows uses lag n_{Tx}.

7.3 Where the compression comes from

Both algorithms share one entropy coder, so their compression ratio can be built up in three steps: what the coder achieves on the FX16 codes alone, what prediction adds, and what a dictionary re-derived for the prediction errors adds. Table 7.4 and Figure 7.2 show the result.

Table 7.4: The compression ratio built up step by step, 50 frames, four receive channels (MATLAB). The "held out" column is the total on frames 5–49 only, which the re-derived dictionary was not trained on.

	Huffman coding only	+ prediction	+ re-derived dictionary	**Total**	Total, held out
DRHE	3.479	+0.272	+0.466	4.218	4.207
LPC	3.479	+0.218	+0.707	4.404	4.372

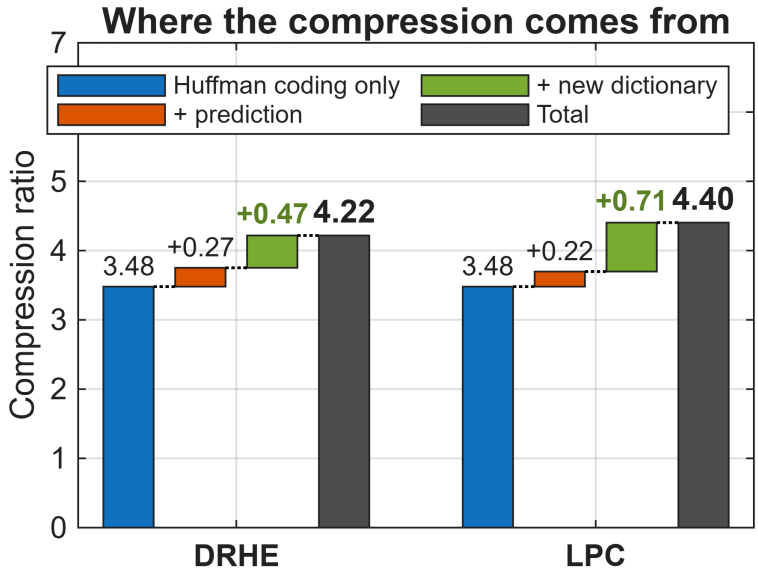


Figure 7.2: The compression ratio built up step by step. Huffman coding alone reaches 3.48; prediction adds 0.22 to 0.27; the re-derived dictionary adds more than prediction does, for both algorithms.

Three things stand out. First, most of the ratio is there before any prediction: the entropy coder alone achieves 3.48. Section 10.3 explains why — largely the unused top bits of the FX16 container, and the sparsity of a range-FFT frame. Second, prediction is worth a modest but real 6 % to 8 %. Third, the dictionary adds more than the predictor does: 0.47 for DRHE and 0.71 for LPC.

Figure 7.3 shows why prediction helps in the currency the coder charges in. The share of values that need zero, one or two bits rises, and the share that need three or more falls; the expensive tail almost disappears.

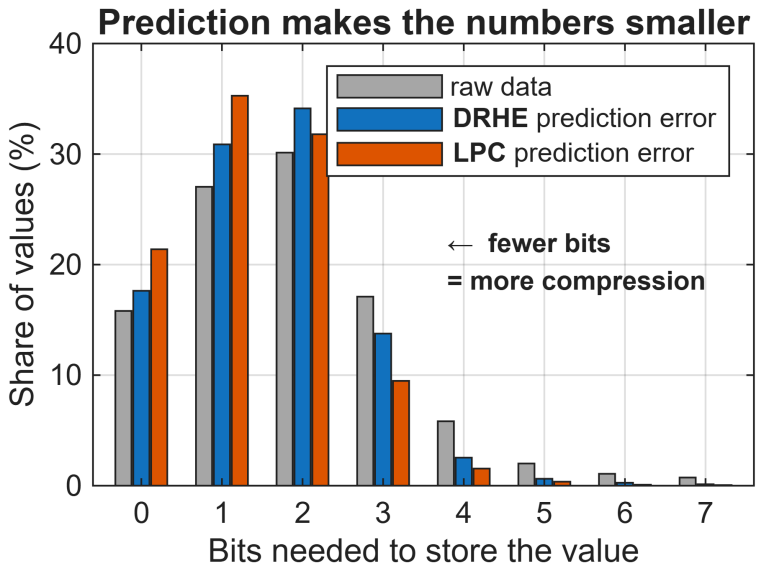


Figure 7.3: How many bits each value needs (S_4), for the raw FX16 codes and for the two algorithms' prediction errors. Prediction moves values towards the cheap end; most prediction errors need two bits or fewer.

7.4 The re-derived dictionary

Table 7.5 sets the share of DRHE's prediction errors in each size category beside the codeword length each dictionary gives it. The original dictionary gives its shortest codes to categories 2 to 4; on this data most errors fall in categories 0 to 2, and categories 3 and 4 hold only 16 %. The re-derived dictionary gives the three commonest categories codes of three, two and one bits, and the average codeword falls from 2.70 to 2.24 bits per value (from 2.80 to 2.12 for LPC).

Table 7.5: Share of DRHE's prediction errors by size S_4, and the codeword length each dictionary assigns (50 frames, four receive channels). Bold marks the three commonest sizes.

Size S_4	0	1	2	3	4	5	Average code
Share of errors (%)	**17.6**	**30.9**	**34.1**	13.8	2.5	0.6	—
Code length, original	4	3	2	2	2	5	2.70 bits
Code length, re-derived	**3**	**2**	**1**	4	5	6	2.24 **bits**

The dictionaries were trained on frames 0–4, so their real test is the 45 frames they did not see. On frames 5–49 DRHE reaches 4.207 and LPC 4.372 (Table 7.4), within 1 % of their all-frame figures: five frames of a scene are enough to fix a dictionary for the rest of the recording. The FPGA cores with the re-derived tables give 4.21704 and 4.40345 per frame, matching MATLAB, both in C-simulation and measured on the board (Section 7.6), and reconstruct all 50 frames exactly.

The dictionary also reverses the ranking of the two algorithms. With the original dictionary

DRHE leads by 1.5 %; with dictionaries matched to each algorithm's own errors LPC leads by 4.4 %. LPC's errors were always the smaller of the two — over five frames, 4.10 against 4.24 bits per value with the original dictionary, before LPC's 0.17 bits per value of coefficients — but a dictionary that cannot code anything in fewer than four bits hides most of that difference.

7.4.1 How close to the limit

The order-0 entropy of Equation (2.9) gives the floor for any code that treats values independently, and an *ideal* S_4 dictionary — a Huffman code built on each design's own category counts — shows how much of the gap to that floor is the dictionary and how much the S_4 construction itself. Table 7.6 gives both, in bits per value, over five frames.

Table 7.6: Bits per value of the residuals (coefficient table excluded), five frames, four receive channels: with the original dictionary, with an ideal S_4 dictionary, and the order-0 entropy of the values.

Residuals	Original dictionary	Ideal S_4	Entropy	Overhead of original
No prediction (raw FX16 codes)	4.586	4.214	4.094	12.0 %
DRHE, lag n_{Tx}	4.240	3.711	3.516	20.6 %
LPC, lags n_{Tx}, $2n_{\mathrm{Tx}}$	4.102	3.242	3.040	34.9 %

Two things follow. The S_4 construction itself is efficient: the ideal dictionary comes within 2 to 7 % of the entropy, so splitting each value into a coded size and raw payload bits loses little. Almost all of the original dictionary's excess is the dictionary, and it grows as prediction improves — from 12 % on the raw codes to 35 % on LPC's errors — because the better the predictor, the further its errors sit below the sizes the original dictionary favours. That is why re-deriving the dictionary gains more for LPC than for DRHE, and why the gain grows with prediction quality rather than shrinking.

7.5 Frame-to-frame variation

Figure 7.4 shows the ratio of every frame. It varies by about a third across the recording, from about 3.5 to 4.8, and the two algorithms move together. The variation tracks how much the scene contains: frames with more strong returns have larger prediction errors and compress less, so the per-frame ratio follows the scene rather than any property of the algorithms. For a system that must carry every frame, it is the worst frame, not the mean, that sizes the link; at

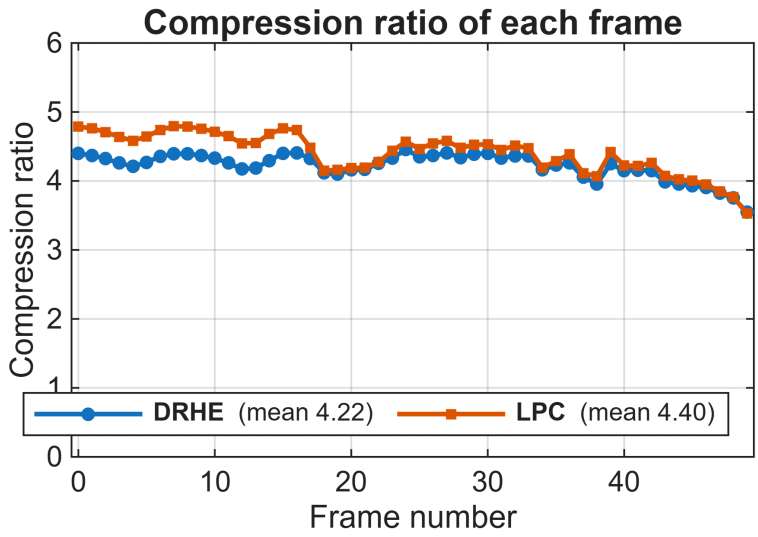


Figure 7.4: Compression ratio of each of the 50 frames, re-derived dictionaries. LPC is ahead on almost every frame; both vary with the scene in the same way.

3.5 it still leaves a wide margin over the 2.4–2.7 required.

7.6 Verification

Every level of the ladder of Section 5.4 that has been run passed with MaxDiff = 0:

- **C-simulation**, 50 frames (4,915,200 complex samples, 157,286,400 input bits per design): both compressors, with both the original and the re-derived dictionaries, reconstruct every sample exactly. The bit counts agree with MATLAB on every frame once the zero padding of the last 256-bit output word is counted: MATLAB's LPC model includes it and agrees exactly, and its DRHE model omits it, so the hardware's DRHE counts exceed MATLAB's by 15 to 252 bits, always less than one word.
- **RTL co-simulation**: both compressors reconstruct exactly.
- **Edge cases**: all eight synthetic cases of Table 5.1 pass for both algorithms. Uniformly random input *expands* by 67 % (CR 0.60), as any lossless coder must on incompressible data: every value lands in a high category, costing a long codeword plus up to 15 payload bits.
- **Board, compressors**: with both dictionaries, both compressors produced compressed output *byte-identical* to the C-simulation on all 50 frames, and every frame, decompressed on the PC, matched the input exactly. With the re-derived dictionaries the mean compression ratio on the board is 4.21704 for DRHE (range 3.548 to 4.454) and 4.40345

for LPC (3.526 to 4.793); with the original dictionary it is 3.7508 (3.517 to 3.815) and 3.6968 (3.461 to 3.761). All four are exactly the C-simulation figures.

- **Board, on-chip decompression**: in the `_full` builds the board compresses each frame, decompresses it with the hardware decoder and compares the reconstruction with the original word by word. Both decoders reconstruct all 50 frames exactly, and the compressed streams are again byte-identical to the C-simulation.
- **Board, from raw ADC samples**: in the `_fe_full` builds the range FFT also runs on the FPGA. Its FX16 codes are identical to its C-simulation on all 50 frames, both compressed streams are byte-identical to the C-simulation of the compressors on those codes, and both decoders again reconstruct every frame exactly (Section 7.9).

Agreement at the board level is the strongest result here: the bytes produced by the FPGA are the bytes the simulation predicted, and the board's own decoder turns them back into the input exactly. Every compression ratio reported for the hardware is therefore a measured figure, and every link of the chain — raw ADC samples in, range FFT, compressed stream, reconstructed data out — has been run on the FPGA.

7.7 Speed and latency on the board

Table 7.7 gives the timing measured on the board over 50 frames, and Figure 7.5 draws it.

Table 7.7: Timing measured on the KCU116 at 100 MHz, 50 frames of 393,216 B. Latency is from the last input word read to the last output word written (Section 5.3); for the decompressor, from the last compressed word read to the last reconstructed word written. Compressor figures are from the original-dictionary run; with the re-derived dictionaries they differ by at most 13 cycles.

	DRHE	**LPC**
Clock cycles per frame	61,472	325,366
Time per frame	0.61 ms	3.25 ms
Frames per second	1,627	307
Input throughput	5.12 Gbit/s	0.97 Gbit/s
End-to-end latency	13.1 µs	3,006 µs
as a share of the frame time	2 %	92 %
Decompressor cycles per frame	196,881	197,366
Decompression time per frame	1.97 ms	1.97 ms
Decompressor latency	339 µs	365 µs

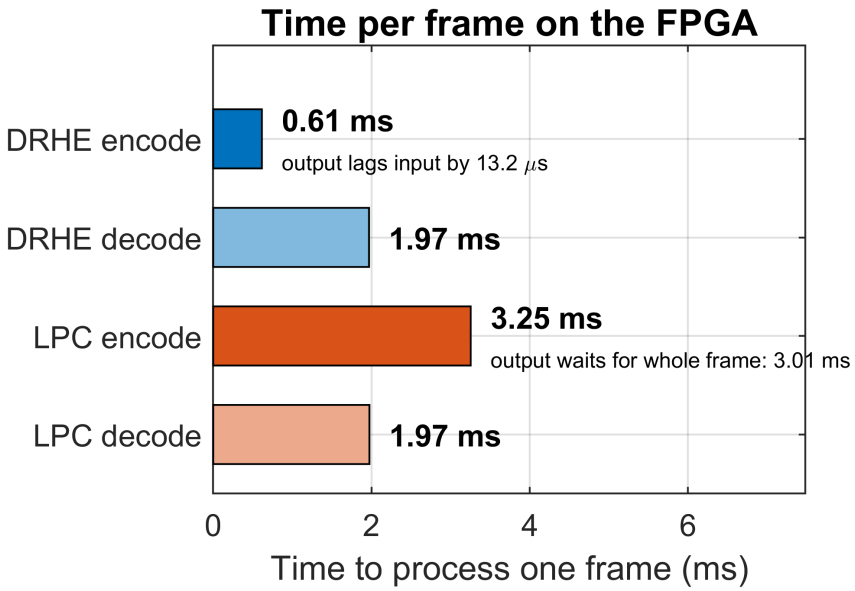


Figure 7.5: Time to encode and decode one frame on the FPGA, measured on the board with the re-derived dictionaries, with each encoder's output delay. DRHE outputs as the data arrives; LPC must wait for the whole frame before it can output anything. Both decoders decode one value per clock.

DRHE streams. Its time per frame, 0.61 ms, is two clocks per input beat (II = 2) plus a short pipeline fill on each ramp. Its latency is 13.1 µs: the last compressed word leaves almost as soon as the last raw word arrives. The core's own pipeline accounts for about a microsecond of that; the rest is most likely data still in flight in the DMA and interconnect when the last input word is read, though this was not measured separately.

LPC waits. Its time per frame is 3.25 ms, 5.3 times DRHE's, and 92 % of it — 3.0 ms — comes *after* the last input word has arrived, because the coefficients cannot be computed until every ramp of the frame is in. That is 230 times DRHE's latency. It is not an implementation weakness that better scheduling would remove: as long as the coefficients are fitted to the frame being compressed, the first output bit has to wait for the last input sample.

Decoding. The DRHE decoder takes 1.97 ms per frame (196,881 cycles, range 196,858 to 196,919), almost exactly the one value per clock it was designed for (Section 6.5): a frame holds 196,608 values. It is 3.2 times slower than the compressor, by design, since it decodes one value per clock where the compressor encodes four channels at once. Its latency of 339 µs is the backlog this leaves: when the last compressed word has been read, about a sixth of the frame is still to be rebuilt.

The LPC decoder shares the DRHE decoder's unpack stage and runs at the same rate:

197,366 cycles per frame on average (range 197,340 to 197,411), or 1.97 ms, with 365 µs of latency (291 µs to 408 µs), the same kind of backlog. Its time does not depend on the data, because every codeword is matched in one clock whatever its length. The first LPC decoder, which tried the codewords one at a time (Section 6.5), took 15.4 ms per frame and 7.6 to 8.3 cycles per value; matching all sixteen at once makes it about eight times faster with fewer LUTs. Nothing in LPC makes its decoder slower than DRHE's: once the coefficients are read, its arithmetic is two multiply–accumulates per value.

Both are fast in absolute terms. The ColoRadar cascade records at 5 Hz, a frame every 200 ms, so DRHE's compressor uses 0.3 % of the frame period, LPC's under 2 %, and each decoder about 1 %. Section 10.2 considers what happens on larger frames.

7.8 Chip resources

Table 7.8 gives the post-route resources of each core, taken from the hierarchical utilisation report of the `_full` builds, and Figure 7.6 draws them as a share of the chip.

Table 7.8: Post-route resources of each core on the `XCKU5P` (216,960 LUT, 433,920 FF, 480 BRAM, 1,824 DSP). BRAM is in 36 Kb blocks.

		LUT	FF	BRAM	DSP
DRHE	compressor	35,444	23,528	40	132
	decompressor	14,155	9,245	32.5	46
	both	49,599 (23 %)	32,773	72.5	178
LPC	compressor	33,820	20,083	108	158
	decompressor	15,212	7,207	0	23
	both	49,032 (23 %)	27,290	108	181

Encoder and decoder together use under a quarter of the chip's logic for both algorithms, and the whole test system — processor, memory controller, DMAs and both cores — uses 38 % (DRHE) and 37 % (LPC) (Table 6.1). The two compressors cost almost the same logic. They differ in memory: LPC's compressor needs 108 block RAMs, 96 of them for its frame buffer, against DRHE's 40 for predictor state. The decoders invert this. The DRHE decoder carries its own floating-point predictor and its state in block RAM, while the LPC decoder needs no block RAM at all, because the coefficients arrive ready-made in the bitstream and its few past samples per (channel, bin) fit in distributed RAM.

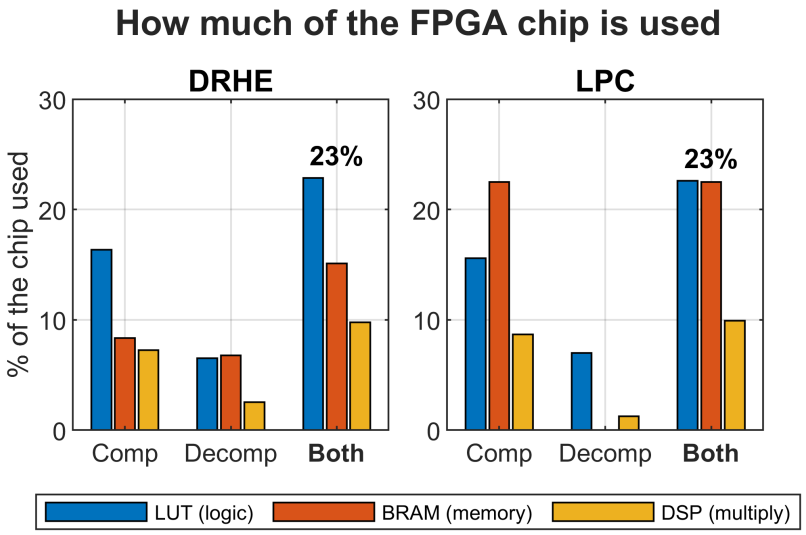


Figure 7.6: Share of the chip used by each core, for the compressor, the decompressor, and both together.

7.9 Starting from raw ADC samples

With the range FFT on chip (Section 6.6) the board is given the raw ADC samples of each frame, 786,432 B, and does everything else itself. Table 7.9 sets these builds beside the ones fed MATLAB's codes.

Table 7.9: Board measurements, 50 frames, with the FX16 codes from MATLAB (as in the rest of this chapter) and from the range FFT on the FPGA. Input rate is FX16 codes for the first and raw ADC samples for the second; latency is from the last input word read to the last compressed word written.

	DRHE		**LPC**	
FFT in	MATLAB	FPGA	MATLAB	FPGA
Compression ratio	4.21704	4.21696	4.40345	4.40338
Cycles per frame	61,472	75,626	325,366	375,957
Time per frame	0.61 ms	0.76 ms	3.25 ms	3.76 ms
Input rate	5.12 Gbit/s	8.32 Gbit/s	0.97 Gbit/s	1.67 Gbit/s
End-to-end latency	13.1 µs	21.3 µs	3,006 µs	3,024 µs

The codes. Against MATLAB's codes, 1,522 of the 9,830,400 values of the 50 frames differ, 0.016 %, each by exactly one step: values that lie almost on a rounding boundary, where the fixed-point FFT and MATLAB's double precision fall on opposite sides. The compression ratios change in the fifth significant figure.

The detections. Table 7.10 runs both sets of codes through the detection chain of Section 5.3, on the four channels the hardware compresses. The rounding error of both is 0.2887 LSB RMS, exactly the $1/\sqrt{12}$ of ideal rounding, so the fixed-point FFT adds no error of its own. The detection maps are identical in 44 of the 50 frames, and the totals differ by two missed targets and one false detection out of 11,148. The losses of FX16 itself are those of Section 7.1.1: the quietest cells are only about one step in size, so rounding raises the noise floor by 0.26 dB and the detection threshold with it. MATLAB's range FFT is therefore a faithful stand-in for the hardware throughout this thesis.

Table 7.10: Detections of the two sets of FX16 codes against the double-precision reference, 50 frames, four receive channels (11,148 reference detections). FP% is a false discovery rate (Section 5.3).

Codes	FN (%)	FP (%)	NF (dBFS)	SNR (dB)	error (LSB RMS)
Reference (double)	—	—	−106.06	36.56	—
FX16, MATLAB	4.19	1.81	−105.81	36.40	0.2887
FX16, FPGA	4.17	1.80	−105.81	36.40	0.2887

Speed. The front end takes about 1.5 clocks per input beat and now sets the pace: DRHE's time per frame rises by 23 % to 0.76 ms, which is still 8.3 Gbit/s of raw ADC data. Its latency rises by 8 µs, the front end's own pipeline. For LPC the front end lengthens only the ingest pass, which waits for the data anyway; the 3.0 ms after the last input word is essentially unchanged.

Cost. The front end is about the size of a decoder in logic (7 % of the chip's LUTs) but uses 296 DSP slices, more than either compressor. With it the whole system uses 47 % (DRHE) and 46 % (LPC) of the LUTs (Table 6.1).

7.10 The two algorithms side by side

Table 7.11 collects the results. On compression ratio the two are close, and which one leads depends on the dictionary: DRHE by 1.5 % with the original one, LPC by 4.4 % with dictionaries matched to each. On logic they are level. On time, latency and memory DRHE is clearly ahead: it compresses a frame five times faster, needs no frame buffer, and its output lags its input by microseconds rather than milliseconds. Their decoders run at the same rate, one value per clock. Chapter 10 weighs these against each other.

Table 7.11: DRHE and LPC compared on every measured quantity. Bold marks the better of the two on each row.

	DRHE	**LPC**
Lossless on the board (50 frames, both dictionaries)	yes	yes
Compression ratio, original dictionary (board)	3.75	3.70
Compression ratio, re-derived dictionary (board)	4.22	4.40
Time per frame	0.61 ms	3.25 ms
End-to-end latency	13.1 **µs**	3,006 µs
First output available	**immediately**	after the whole frame
Decompression time per frame	1.97 ms	1.97 ms
Compressor frame buffer	**none**	3.0 Mibit
Encoder + decoder LUTs	23 %	23 %
Encoder + decoder BRAM (36 Kb blocks)	**72.5**	108
Side information	**none**	1.04 %

Chapter 8

Interpreting the Hardware Numbers

Kiem reports a comparable set of hardware numbers for his DRHE compressor [11], and set side by side the two sets differ on almost every line. A difference on its own explains nothing, so this chapter puts the numbers on the same footing in three steps. Every design is implemented through placement and routing, because Kiem's figures are post-route (Section 8.1). Kiem's compressor is rebuilt in this thesis's tool flow (Section 8.2). And the path from this thesis's datapath to the rebuilt design is taken one design decision at a time, with every step implemented (Section 8.3). The remaining sections read latency, throughput and resources against the sensor and the device, and the chapter closes with a ledger of every difference from Kiem (Section 8.7).

Kiem's sensor has one transmitter, so his lag-1 predictor is the per-transmitter predictor for that sensor. The comparison therefore starts from *DRHE-1*: this thesis's DRHE datapath predicting at lag 1. Its datapath is that of the final lag-n_{Tx} compressor; only the indexing of the predictor state differs.

8.1 Estimates against routed results

HLS reports what it *expects* a circuit to cost; Vivado reports what the placed and routed circuit *does* cost. Each core was implemented on its own (out of context) on the `xcku5p-ffvb676-2-e` at 10 ns, with `vivado.flow=impl` in its HLS configuration. Out of context and in the board system, the DRHE compressor's LUTs and flip-flops agree to within 2 % and its DSPs and block RAM exactly; LPC's block RAM differs by about 10 %, because Vivado maps its memories differently. The in-system figures of Table 7.8 are the ones quoted for the final cores.

Table 8.1 compares the two kinds of number.

Table 8.1: HLS estimates against the same compressors after placement and routing, out of context, on the `xcku5p` with a 10 ns constraint. BRAM in 36 Kb blocks. $F_{max} = 1/(10\,\text{ns} - \text{WNS})$ after routing.

	LUT		FF		DSP		BRAM		F_{max} (MHz)	
Design	HLS	impl.	HLS	impl.	HLS	impl.	HLS	impl.	HLS	impl.
DRHE, lag n_{Tx}	115,736	34,804	57,263	23,514	132	132	40	40	81.9	**131.0**
LPC, lags $n_{Tx}, 2n_{Tx}$	86,298	34,503	23,154	20,186	156	158	96	108	85.4	**106.3**
DRHE-1 (lag 1)	115,534	35,343	57,105	23,546	132	132	20	20	81.9	135.5

HLS over-estimates logic by about three times (3.3 for DRHE, 2.5 for LPC). Most of the excess is logic HLS generates itself from C++. It costs each of the packer's eight inserts into the 512-bit register as a full 512-bit barrel shifter, although the value shifted is at most 29 bits wide and Vivado removes most of each shifter; and it costs each floating-point `atan2`, a CORDIC built from generic C++, at 20,129 LUTs against about 5,980 implemented. DSPs and block RAM, which are fixed blocks HLS allocates itself, are estimated almost exactly. The clock estimate is pessimistic in the same way: HLS predicts 81.9 MHz and 85.4 MHz, yet every design meets 100 MHz after routing, and the board systems do so with 0.17 ns to 0.34 ns of slack (Table 6.1). HLS estimates are useful for comparing versions of one design; every number in this chapter compared with Kiem's is post-route.

8.2 Rebuilding the reference design

This thesis's DRHE and Kiem's differ in at least seven ways at once: prediction axis, arithmetic, bit packer, state reset, feedback structure, tool version (Vitis HLS 2026.1 against 2022.2) and device. To separate them, his compressor was rebuilt from the description in his Section 4.3.4: his fixed-point types (magnitude `ap_ufixed<16,16>`, phase `ap_fixed<16,3>`, intermediates `ap_fixed<32,4>`), the `hls::sqrt`, `atan2`, `sin` and `cos` he names, twenty 1024-word state arrays driven from the input sample (open loop), a 58-bit word per channel packed into the 256-bit output, and all four channels at II = 1. He does not describe how the state is cleared at the start of a frame; the replica reads it as zero on the first ramp, which is the only reading consistent with II = 1 (Section 8.3). The replica and DRHE-1 are one source file with four compile-time switches (packer, state reset, arithmetic, loop), and every variant decodes ColoRadar losslessly in C-simulation.

Table 8.2: The rebuilt design against Kiem's published compression IP (his Table 5.3 [11]). Replica figures are post-route, out of context, on the `xcku5p` at speed grades −2 and −1; Kiem's are post-route, in system, on the `XCZU3CG` at speed grade −1. BRAM in 36 Kb blocks.

	Replica (−2)	Replica (−1)	Kiem [11]
LUT	17,141	17,228	45,892
FF	5,050	5,598	9,243
DSP	48	48	44
BRAM	10	10	10
Initiation interval	1	1	1
Pipeline depth (HLS, cycles)	12	14	23
Achieved clock (MHz)	129.5	125.0	100 (met)

The replica matches Kiem where his design is fully specified: block RAM exactly, DSPs closely, II = 1, and 100 MHz on a speed-grade −1 part (Table 8.2). It does not reproduce his LUT and flip-flop counts, which his design as described does not need. Three explanations were tested by building each into the replica: his slower speed grade, the most literal reading of his one-sentence packer description, and his deeper pipeline. If their effects add, they account for about 45 % of the flip-flop gap and 5 % of the LUT gap. The rest most likely lies in the four-year-older fixed-point math library of his tool version, or in logic his thesis does not describe, such as the debug probes his Figure 5.4 shows. His LUT figure therefore measures his build, not the cost of the DRHE algorithm; the replica is the fair yardstick for what this thesis's design decisions cost.

8.3 From this thesis's datapath to Kiem's, one change at a time

Table 8.3 is the central result of this chapter. Each row is a complete lossless compressor implemented through routing; each changes one design decision from the row above, from DRHE-1 to the replica. The difference between adjacent rows is the cost of that one decision.

Block RAM is the word width. DRHE's state is five values per range bin and channel, twenty arrays. DRHE-1 stores them as 32-bit floats, so each 1024-word array fills a 36 Kb block: 20 blocks. Kiem's 16-bit words fit two arrays to a block: 10 blocks, which the replica matches exactly. The final compressor keeps one word per transmitter and bin, $12 \times 128 = 1536$ words per array, which HLS rounds up to two blocks: 40 in total. Against DRHE-1 it changes

Table 8.3: From DRHE-1 to the replica of Kiem's design, one design decision per step. Every row is lossless and implemented through routing, out of context, on the `xcku5p` at 10 ns. II and depth are from HLS. BRAM in 36 Kb blocks.

Step	Change from the row above	LUT	FF	DSP	BRAM	II	Depth	MHz
0	DRHE-1 (float, closed loop, lag 1)	35,343	23,546	132	20	2	58	135.5
1	packer shift offset made unsigned	33,502	24,423	132	20	2	58	128.4
2	Kiem's 58-bit per-channel packer	31,124	21,261	127	20	2	62	120.6
3	state cleared by read, not write	52,432	30,373	264	20	1	59	131.4
4	Kiem's fixed-point arithmetic	16,882	5,380	48	10	1	16	133.3
5	open loop = replica of Kiem	17,141	5,050	48	10	1	12	129.5
—	Kiem, published [11]	45,892	9,243	44	10	1	23	100
—	DRHE, lag n_{Tx} (final design)	34,804	23,514	132	40	2	58	131.0

nothing else — the same 132 DSPs and the same schedule — so at ColoRadar's size the per-transmitter prediction costs 20 blocks of block RAM and nothing more. The state grows with bins times transmitters: at Kiem's 512 bins and 12 transmitters it would be 160 blocks in floating point and 80 in his 16-bit types.

DSPs are floating point. Every DSP in DRHE belongs to a floating-point operator: 16 multipliers at 3 slices, 12 adders at 2, four sine/cosine units at 13 and two arctangents at 4, 132 in all. A 16-bit fixed-point product needs one slice and an addition none. Four channels would need eight sine/cosine units; at II = 2 HLS shares each unit between two channels, and step 3 shows that without the sharing the float design needs 264.

The pipeline depth is the closed loop. DRHE's pipeline is 58 cycles deep against 23 in Kiem's HLS report. It is two chains in series: the prediction (state, filters, sine and cosine, rounding), then the polar conversion that updates the state, whose `atan2` alone is 28 stages. The second chain must wait for the first because DRHE updates its state from the *reconstructed* sample, which is what guarantees the encoder and decoder never disagree (Section 6.4). Driving the state from the input instead lets the chains overlap and cuts the float depth to 32; fixed point shortens both chains further, to 16 closed loop and 12 open loop. Of DRHE's 58 cycles, 26 are the closed loop; at 260 ns it is a cost this application does not notice.

The initiation interval is one write. HLS reports why DRHE runs at II = 2: "unable to schedule load operation ... due to limited memory ports". Each state array is a block RAM with two ports, and each cycle needs one read and one write. DRHE also clears the state at

the start of a frame with an *extra* write; it happens on only the first ramps, but HLS schedules for the worst case, and three accesses on two ports take two cycles. Step 3 clears the state by ignoring the read on the first ramp instead, and the same float design schedules at $\mathrm{II} = 1$ — at the price of losing the operator sharing, which doubles its DSPs and grows its logic. It is the only step that makes the design substantially larger, and it is why the final design keeps $\mathrm{II} = 2$: with 24 times more throughput than four ColoRadar channels need (Section 8.5), half the rate buys half the arithmetic.

LUTs and flip-flops. Against Kiem's published figures, the final compressor uses fewer LUTs (0.77×) but more flip-flops (2.55×). The apparent contradiction comes from the unexplained gap between his build and the replica, which is large for LUTs and small for flip-flops. Against the replica, DRHE-1 is larger in both, by amounts the ladder accounts for completely. The floating-point operators are 69 % of DRHE-1's LUTs, the two arctangents alone a third. Steps 1 and 2 show that DRHE's packer costs about 4,200 LUTs more than Kiem's: a signed shift offset keeps logic for negative shifts that never happen, and eight inserts into a 512-bit register cost more than one 232-bit insert. Step 4, fixed point, removes most of the rest, and flip-flops fall further than LUTs because the values in flight become both narrower and fewer. Step 5, the open loop, is almost free in fixed point. Changing *only* DRHE-1's arithmetic to Kiem's types, keeping everything else, gives 20,549 LUTs (42 % fewer), exactly Kiem's 44 DSPs and half the block RAM, and compresses ColoRadar to within 0.07 % of the floating-point design. It meets 100 MHz with little margin (102.8 MHz), because its short closed loop puts 26 carry-chain levels before the state write; one or two more pipeline stages would fix that.

The clock. Every design meets 100 MHz. The replica closes at 125.0 MHz on a speed-grade -1 part and 129.5 MHz on a -2, and DRHE-1 closes at 138.3 MHz on a -1 part, so this thesis's floating-point datapath meets Kiem's clock on Kiem's grade of silicon.

8.4 Latency

Three quantities are called latency, and they answer different questions.

Pipeline latency is how long one sample takes to pass through: 58 cycles, 580 ns, for DRHE, against Kiem's HLS estimate of 23 cycles. The difference is the closed loop and the latency

of floating-point operators (Section 8.3). It is 0.7 % of a ColoRadar ramp interval and has no practical consequence here.

Frame time is how long a whole frame takes when it is fed as fast as the core accepts it: 0.61 ms for DRHE and 3.25 ms for LPC on the board. It is really a throughput figure. DRHE's is set by its initiation interval — 24,576 beats at two cycles each, plus the pipeline draining once per ramp — and LPC's by its three passes (Section 6.3). The sensor delivers a frame over 15.07 ms, so DRHE is busy about 4 % of the time.

Added latency is how much later the downstream processor receives the last bit of a frame because of compression, measured with Kiem's definition from the last input word read to the last compressed word written. This is the latency that matters. DRHE streams, so on the board only the data still in flight remains: 13.1 µs. In a sensor-paced system no backlog forms, and only the last ramp's 320 cycles, 3.2 µs, remain. LPC cannot emit anything until it has seen the whole frame, so after the last input word it still has its solve and emit passes to run: 3.0 ms, however the frame is fed. That is a factor of 230 on the board and about a thousand in a sensor-paced system. Kiem's 79.4 µs uses the same definition but includes the range FFT of the last ramp and its DMA transfers. Measured over the same chain, with the range FFT on chip (Section 7.9), this thesis's DRHE takes 21.3 µs; both are tens of microseconds and negligible against the frame. The decoders' latencies, 339 µs and 365 µs, are their own backlog when the compressed input ends, not a dependence on the whole frame.

When latency becomes a problem. There are three thresholds. The hardest is throughput: a compressor slower than the sensor builds a backlog and loses data. DRHE uses 320 of the 7,850 cycles each ColoRadar ramp provides. The second is the system's schedule. Kiem allotted 15 ms of his 50 ms cycle to compression; on ColoRadar, at 5 Hz, both designs use a tiny fraction of the gap after acquisition. Scaled to Kiem's frame, 21 times larger, DRHE still keeps up ramp by ramp (10.9 µs of work per 20.5 µs ramp), but LPC's post-ingest passes grow to about 64 ms, four times his budget and longer than his whole cycle, and its frame buffer to 67 Mbit, more than the block RAM of either device. Even the one-third saving available from overlapping its passes would leave it near 45 ms. The third threshold is the application: at 100 $\mathrm{km\,h^{-1}}$ LPC's 3.0 ms is 8.4 cm of travel and DRHE's 13.1 µs a third of a millimetre, both small next to a 5 Hz radar's own 200 ms. On ColoRadar LPC's latency is acceptable; on a

long-frame sensor like Kiem's it is the limiting factor.

8.5 Throughput

With a 128-bit input beat at clock f, the theoretical throughput is $128f/\mathrm{II}$. Table 8.4 sets it and the measured rate against what the sensors require.

Table 8.4: Throughput in decimal Gbit/s. Measured figures include pipeline fill and drain: on the board for the final designs, in RTL co-simulation for the replica. Kiem's 11.92 Gbit/s is 12.8 Gbit/s, theoretical.

	Theoretical	Measured
DRHE, II = 2 (board)	6.40	**5.12**
LPC, frame average (board)	—	**0.97**
Replica of Kiem, II = 1 (co-sim)	12.80	11.54
Kiem [11]	12.80	not reported
Required: ColoRadar, 4 RX	0.209	
Required: ColoRadar, all 16 RX	0.835	
Required: Kiem's sensor	3.20	

Kiem's 11.92 Gbit/s is stream width times clock, not a measurement. Theoretical against theoretical, the gap is exactly the factor of two between II = 2 and II = 1; the further 20 % in the measured figure is the per-ramp pipeline drain, which the replica shows too. The comparison that matters is with the sensor: DRHE accepts data 24.5 times faster than four ColoRadar channels produce it and 1.6 times faster than Kiem's sensor, with no buffer. That headroom could let one instance serve all sixteen channels, or run at a slower clock. LPC's 0.97 Gbit/s is a frame average; its ingest pass alone accepts 12.8 Gbit/s, and only for a sensor delivering frames back to back would its 1.16 times headroom over ColoRadar's 16-channel rate matter. At II = 2 DRHE accepts data at half the rate a streaming FFT core can produce it within a ramp, so the FFT must be allowed to pause, or a FIFO of half a block RAM must sit between them.

8.6 Resources against the device

Each compressor uses about a sixth of the `XCKU5P`'s logic, and encoder and decoder together under a quarter (Table 8.5). Every compressor also fits Kiem's `XCZU3CG`, and the DRHE compressor uses fewer of its LUTs than his own IP does (50 against 65 %). His complete system used 59,353 LUTs, 13,461 of them outside his compressor; with DRHE's compressor in its

Table 8.5: Post-route resources of each core as a percentage of the XCKU5P (216,960 LUT, 433,920 FF, 480 BRAM, 1,824 DSP) and of Kiem's XCZU3CG (70,560 LUT, 141,120 FF, 216 BRAM, 360 DSP). BRAM in 36 Kb blocks. Final cores are the in-system figures of Table 7.8; the replica is out of context.

	% of XCKU5P				% of XCZU3CG			
Core	LUT	FF	BRAM	DSP	LUT	FF	BRAM	DSP
DRHE compressor	16	5	8	7	50	17	19	37
DRHE decompressor	7	2	7	3	20	7	15	13
LPC compressor	16	5	22	9	48	14	50	44
LPC decompressor	7	2	0	1.3	22	5	0	6
Replica of Kiem	8	1	2	3	24	4	5	13
Kiem's IP [11]	21	2	2	2	65	7	5	12

place the system would use about 69 % of the device, inside the usual 70–80 % congestion guideline. The decompressors sit at the central processor, not beside the sensor, so they are not part of this budget.

The cores compress four of ColoRadar's sixteen receive channels. Four DRHE compressors would fit the XCKU5P (65 % of its LUTs) but not the XCZU3CG. One time-multiplexed instance, using its throughput headroom, keeps the logic of one and needs 160 blocks for the state, which fits Kiem's device only tightly. LPC's memory is its limit: at about 100 blocks per four channels, sixteen channels need over 80 % of the XCKU5P's block RAM. The largest single saving available is DRHE's arithmetic: Kiem's fixed-point types would roughly halve its logic, cut its DSPs to a third and its block RAM to half, for 0.07 % of compression ratio (Section 8.3).

8.7 Every difference from Kiem, accounted for

Table 8.6 lists every number in which this thesis's DRHE and Kiem's differ, whether the published numbers measure the same thing, and the cause.

Measured post-route, as Kiem's figures are, this thesis's DRHE compressor uses fewer LUTs than his IP, meets the same clock and fits his device at half its logic. Against his design rebuilt in the same tool it is larger, by exactly what its distinctive decisions cost: floating-point arithmetic (most of the logic, every extra DSP, twice the block RAM), a less economical packer (about 4,200 LUTs), a closed loop (26 cycles in floating point, almost nothing in fixed point), one extra state write (II = 2, half the throughput and half the arithmetic), and per-transmitter state (twice the block RAM again). The closed loop guarantees the encoder and decoder never

Table 8.6: Every difference between this thesis's DRHE and Kiem's published design. "Final" is the lag-n_{Tx} compressor on the board; the replica is post-route, out of context. "Valid" says whether the published numbers measure the same thing.

Metric	Final	Replica	Kiem	Valid	Cause of the difference
LUT	35,444	17,141	45,892	yes	Against the replica: floating-point operators, then the packer. Replica against Kiem: about 5 % explained.
FF	23,528	5,050	9,243	yes	Registers inside float operators and along a 58-stage pipeline. Replica against Kiem: about 45 % explained.
DSP	132	48	44	yes	Every DSP is a float operator; sharing at $\mathrm{II} = 2$ halves the count.
BRAM	40	10	10	yes	32-bit state words need twice the memory of 16-bit ones; per-transmitter state doubles it again.
II	2	1	1	yes	One extra write per cycle to clear the state.
Depth	58	12	23	yes	Closed loop (26 cycles), then float operator latency.
$F_{\max}$	100 met	129.5	100 met	partly	All meet 100 MHz; Kiem on speed grade -1.
Throughput	5.12 meas.	11.54	12.8 theor.	theor. only	$\mathrm{II} = 2$ against $\mathrm{II} = 1$.
Added latency	13.1 µs; 21.3 µs with FFT	—	79.4 µs	with FFT	Same definition; his chain, like the 21.3 µs figure, includes the range FFT.
Compression ratio	3.75, 4.22	—	3.17	no	Different sensor and data scaling (Chapter 9).

diverge, the per-transmitter state is what makes prediction pay on a TDM-MIMO sensor, and the floating-point datapath buys only 0.07 % of ratio.

Chapter 9

Comparison with the Baseline

Kiem's compression ratios come from private recordings of a single-transmitter radar with real samples; this thesis's come from a public twelve-transmitter cascade with complex samples. Three questions cannot be answered across that gap directly. Kiem's DRHE compresses his data by 3.25 and this thesis's DRHE compresses ColoRadar by 3.75: are the numbers comparable? Kiem judged LPC less promising than DRHE, while this thesis's LPC is level with it: which is right? And is this thesis's DRHE the algorithm Kiem specified? To answer them, Kiem's MATLAB simulation was rebuilt from his text alone and every coder of this thesis was run on it. The hardware side of the comparison is Chapter 8.

9.1 Rebuilding Kiem's simulation

The replication is a separate MATLAB code base that shares no code with the evaluation chain of Chapter 5. It follows his Figure 3.1: beat signals of his five-target scene (his Table 3.2: one transmitter, four receivers, 1024 samples per ramp, 512 ramps, 14-bit ADC), white Gaussian noise set to a chosen noise floor after NCI, his pre-processing and range FFT, quantisation to FX16, every coder, and his second-stage processing and metrics. Every lossless path is decoded and compared with its input on every run.

Where his text leaves a choice open, the replication makes it explicitly and in one place. The one that matters is the FX16 scaling. Kiem's FFT core keeps the top 16 bits of its 32-bit output word, so the full scale of the *word* maps to 32,767. This thesis's chain maps the spectral peak of a full-scale *sinusoid* to 32,767 (Equation (2.11)). The two differ by exactly a factor of four, because the Hanning window halves a sinusoid's peak and a real sinusoid splits its amplitude

between positive and negative frequencies. The replication runs both. The other choices — target angles, an amplitude scaling that avoids ADC clipping, the definition of 0 dBFS, and the detector of Section 5.2 — do not affect the compression ratio.

9.2 What sets the ratio on Kiem's scene

Table 9.1 runs the workflow at the −75 dBFS noise floor of his Table 3.2, and at −87.9 dBFS, the level that a bisection finds DRHE reproducing his 3.25.

Table 9.1: Compression ratios on Kiem's rebuilt scene with his FFT core's scaling, at his stated noise floor and at the level where DRHE reproduces his 3.25. FX16 noise is 20.4 and 4.6 LSB respectively. Every lossless coder has FX16's detection metrics.

Coder	−75 dBFS	−87.9 dBFS	Kiem (his data)
DRHE, R4S4 + run-length, ideal Huffman	2.28	**3.25**	**3.25**
DRHE, S_4 only, ideal Huffman	2.28	**3.24**	**3.24**
DRHE, S_4, Kiem's fixed dictionary	1.85	3.23	3.17 (hardware)
LPC, slow time (this thesis)	2.06	3.38	—
LPC-Huffman, raw ADC [27], $p = 10$	1.32	1.54	not run
No prediction, fixed dictionary	2.03	3.26	—

DRHE gives 2.28 on Kiem's own scene, not 3.25. The scene, chain and algorithm are his, so his 3.25 is a property of his recordings. At −87.9 dBFS the replication reproduces his pattern exactly — 3.25 with run-length coding, 3.24 without, almost nothing lost to the fixed dictionary — but that operating point was chosen to give 3.25, and it sits 17 dB below the −70.7 dBFS he reports with it.

Prediction does almost nothing on white noise. With no prediction the fixed dictionary gives 2.03 and 3.26, level with DRHE and the slow-time LPC. Noise at ramp $m-1$ says nothing about noise at ramp m; the ratio is set by how many bits of noise each value carries. Figure 9.1 shows this across noise floors from −120 to −60 dBFS. Each 6 dB of noise doubles the noise in LSB and costs about one bit per value; at very low noise the fixed dictionary's four-bit minimum caps the ratio at 3.92; the two scalings give the same curve shifted by 12 dB.

This thesis's chain gives about 1.1 on such scenes. Synthetic scenes of Kiem's kind put through this thesis's own chain, with its full-scale scaling, compress to only 1.10–1.12 near

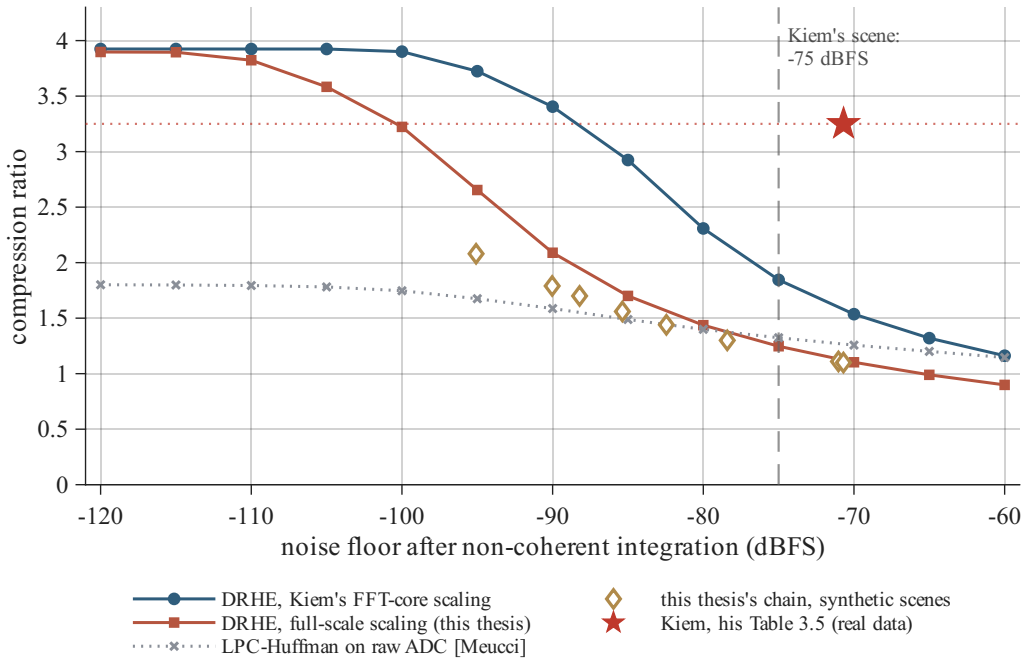


Figure 9.1: Compression ratio against noise floor on Kiem's rebuilt scene. DRHE with Kiem's fixed dictionary, with his FFT core's scaling and with this thesis's full-scale scaling (four times larger); LPC-Huffman of [27] on raw ADC; this thesis's own chain on synthetic scenes of the same kind; and Kiem's published 3.25 at −70.7 dBFS.

−70.7 dBFS. The replication with the full-scale scaling gives the same 1.12 at that noise floor, from an independent code base. Two factors multiply to explain it: the scaling puts four times as much noise into each code (about two more bits per value), and matching Kiem's stated noise floor with white noise puts 33 LSB of noise into each code even with his scaling, where his ratio needs about 4.6 LSB. The same mechanism explains why ColoRadar compresses as well as it does: its FX16 codes use fewer than ten of their sixteen bits (Section 7.3).

His noise floor and his ratio do not agree on white noise. His fixed dictionary records what his residuals looked like. Coding integer-rounded Gaussian residuals with it and with the ideal code for each spread shows that it is optimal (overhead under 0.1 %) for residuals with a standard deviation of 6 to 8 LSB, consistent with his ratio and his residual histograms, and costs 26 % at the 33 LSB that his stated noise floor implies. The most likely reconciliation is that on real data the mean "noisy part" of the spectrum is raised by clutter, leakage and sidelobes, while most cells, which set the ratio, sit much lower. That is an inference; it cannot be confirmed without his data.

9.3 The LPC he excluded

LPC-Huffman as Kiem describes it predicts each raw ADC sample from the previous p samples of the same chirp, with Levinson–Durbin coefficients per chirp and receiver sent as 32-bit floats. On his scene it loses to DRHE by a wide margin at every order up to $p = 10$ (Table 9.1), and stays below 1.8 across the whole sweep. **For the method he meant, Kiem's ranking holds** — but for a reason that has little to do with prediction. His chain shifts the 14-bit samples left by 18 bits, windows them, takes the FFT divided by N and keeps the top 16 bits, so white ADC noise of standard deviation σ reaches the FX16 code as

$$\sigma_{\mathrm{FX16}} = \sigma \cdot \frac{2^{18}}{2^{16}} \cdot \frac{\sqrt{\sum_k w_k^2/2}}{N} = 0.0541\,\sigma, \tag{9.1}$$

which the simulation measures exactly. The FFT scaling and truncation discard about 4.2 bits of noise per value before DRHE sees the data; a lossless coder of the raw samples must keep them. The truncation is the lossy step, and Kiem's FX16 row shows its cost in detections; DRHE inherits that cost and raw-ADC LPC does not.

The slow-time LPC of this thesis, which predicts in the same domain and along the same axis as DRHE (Section 4.5.6), is a different method, and on the same scene it is level with or slightly ahead of DRHE: 2.06 against 1.85 and 3.38 against 3.23. On ColoRadar it is level too: 3.697 against 3.751 with Kiem's dictionary, and 4.404 against 4.218 once each has a dictionary fitted to its own errors (Section 7.4). The two algorithms are within a few per cent on ratio; what separates them is that DRHE streams and LPC needs the whole frame.

9.4 Adding transmitters

The replication can also interleave the ramps over twelve transmitters, as in the ColoRadar cascade. On Kiem's five-target scene the transmitters make almost no difference, because there is nothing but noise to predict. On a denser scene of 300 slowly moving scatterers at -95 dBFS — a stand-in for a real scene, not Kiem's — prediction earns its keep: DRHE gives 3.40 against 1.95 without prediction on one transmitter. Interleaving twelve transmitters cuts it to 2.37 at lag 1, and predicting from the same transmitter (lag 12) recovers part of the loss, to 2.56. On ColoRadar, whose scene changes slowly, the same change of lag turns a 4.9 % loss against no prediction into a 7.8 % gain (Section 7.2).

9.5 The DRHE algorithm, line by line

Table 9.2 compares every element of Kiem's final DRHE with this thesis's MATLAB model and HLS design.

Table 9.2: Every element of Kiem's final DRHE scheme (his §3.5.3 and §4.3.4) against this thesis's MATLAB model and HLS design.

Element	Kiem	MATLAB model	HLS design	Same?
Input	FX16 range FFT, first $N/2$ bins, per RX	same	same (128-bit AXIS, 4 RX)	yes
Prediction	Equations (3.2)–(3.3), $\alpha = 0.6$, $\beta = 0.4$, phase wrapped once	same, at lag L	same, at lag L	yes; (e)
Back to Cartesian	rounded to integer	`round`	`roundf`	yes
Difference	16-bit two's complement	`wrap_int16`	`wrap_int16`	yes
Initial state	zero	zero per frame	zero per transmitter per frame	yes; (c)
State update	from the input	from the input	from the reconstruction	changed; (a)
S_4 and APPEND	his Table 3.7	same	same	yes
Residual -32768	no code; not addressed	counted	saturated to -32767	changed; (a)
Dictionary	fixed, his Appendix A	same	same, bit-reversed	yes; (d)
Arithmetic	fixed point	double	single-precision float	changed; (b)
Packing	58-bit word per channel	bit count	512-bit accumulator	layout; (d)
Lag	ramp $m-1$ (one TX)	n_{Tx}	n_{Tx}	extension; (e)

The equivalence is checked by output. The replication's DRHE, written from Kiem's text without reference to this thesis's code, and this thesis's MATLAB model at lag 1 produce identical residuals and identical bit counts on the replicated scene (18,186,374 bits at -75 dBFS and 10,370,342 at -87.9 dBFS). The HLS design reproduces the MATLAB model's bit counts on all 50 ColoRadar frames, an independent Python implementation reproduces the HLS bit counts, and the board reproduces the HLS bytes (Section 7.6). The DRHE of this thesis is therefore Kiem's, extended in one way and implemented differently in four:

(a) A residual of -32768 has no code in his scheme; the hardware saturates it to -32767, and updates its state from the reconstruction so that the encoder and decoder still agree afterwards (Section 6.4). Without saturation the closed loop gives exactly his output; on ColoRadar no residual comes near -32768.

(b) Floating point follows his own suggestion for future work. It buys 0.07 % of ratio and costs most of the logic, every extra DSP and twice the block RAM (Chapter 8). It also does not remove the need for a decoder built from the same source: a double-precision decoder drifts on 8 of the 50 DRHE frames.

(c) Each transmitter's state is reset on its first ramp of every frame, so every frame decodes on its own.

(d) The bits are his, in a different order: codewords are bit-reversed because the packer is least-significant-bit first.

(e) The lag is n_{Tx}; with $n_{\mathrm{Tx}} = 1$ it is his algorithm unchanged.

9.6 Every result, compared

The compression ratios measure different things. DRHE compresses ColoRadar by 3.751 with Kiem's own dictionary, and his recordings by 3.25. The algorithm is the same, but the ratios come from different mechanisms. On ColoRadar, unused container bits give a factor of 1.64, the sparsity of the range-FFT frame a further 2.12 through the entropy coder, and prediction at lag n_{Tx} adds 7.8 % (Section 7.3); at lag 1 the same predictor *costs* 4.9 %. Kiem's 3.25 is what a matched coder gives on residuals of 6 to 8 LSB. The two chains also differ in FX16 scaling, ADC width and gain, real against complex samples, and one against twelve transmitters, each of which moves the ratio on its own. The comparison of *algorithms* is settled by the audit; the comparison of *datasets* cannot be made, and neither the similarity of 3.25 and 3.75 nor their difference says anything about the algorithms.

Run-length coding adds little on both datasets. On ColoRadar, with ideal dictionaries, it gains at most 1.6 %, because fewer than a fifth of the residuals are zero even at lag n_{Tx}. His simplification holds.

A fixed dictionary costs little only on the data it was fixed on. On his scene at -87.9 dBFS his dictionary costs 0.3 %. On ColoRadar, a dictionary re-derived from frames 0–4 gains 12 % (DRHE) and 18 % (LPC) on the held-out frames 5–49, because ColoRadar's lag-n_{Tx} residuals are much smaller than the ones his dictionary was built for.

Detection metrics and noise floors are not comparable. His FX16 row loses 0.59 % of detections; this thesis's loses 3.77 % (Table 7.1). Both measure the cost of quantising to 16-bit fixed point, but on different sensors, scenes and chains. ColoRadar's codes use 9.77 of 16 bits, so the quantisation step is coarse relative to weak returns, and its detector integrates over transmitter-interleaved ramps. His noise floor of -70.7 dBFS and this thesis's -105.4 dBFS do not share a definition of 0 dBFS.

Same: the DRHE algorithm, to the bit; that run-length coding adds almost nothing; that DRHE beats every lossy scheme on ratio and detection together; that the datapath meets 100 MHz on his speed grade and fits his device.

Different, and explained: every hardware resource figure, the initiation interval and the throughput (Chapter 8); the value of his fixed dictionary on other data; the LPC ranking, which holds for his LPC and not for this thesis's.

Extended: the prediction lag, which is n_{Tx} on a TDM-MIMO cascade; a decompressor for every design; and verification of the hardware against simulation byte for byte.

Not comparable: the compression ratios and detection metrics on the two datasets.

Unexplained: his published LUT and flip-flop counts exceed what his described design needs, and his reported noise floor is 17 dB higher than a white-noise scene with his ratio would have.

Chapter 10

Discussion

10.1 Which algorithm for a sensor front end?

On compression ratio, DRHE and LPC are close, and which one leads depends on the dictionary rather than on the predictors (Section 7.4). On logic they are level. The decisive differences are elsewhere.

Streaming. DRHE's output depends only on samples already received, so it emits compressed data as the raw data arrives, needs no frame buffer, and its output lags its input by 13 µs. LPC's coefficients are sums over every ramp of the frame, so its first output bit has to wait for its last input sample: 3.0 ms of latency and a 3.0 Mibit buffer at this frame size. That is a property of the algorithm, not of this implementation; no amount of scheduling removes it while the coefficients are fitted to the frame being compressed.

Memory. A compressor between the range FFT and the memory bus of a radar sensor has little memory to spare, and LPC's buffer grows with the frame (Section 10.2). DRHE's state also grows, with the number of range bins and transmitters, but it is a fraction of a frame.

Where LPC is attractive. LPC's costs fall mainly on the compressor. Its decoder is the cheaper of the two in memory and arithmetic — no block RAM, fixed-point multiply–accumulates only — because the coefficients arrive ready-made, and it is as fast as DRHE's, one value per clock (Section 7.7). Where latency of a few milliseconds is acceptable and the sensor has memory to spare, LPC's 4 % better ratio is available. A backward-adaptive LPC,

fitting its coefficients to *previous* frames instead of the current one, would remove both the buffer and the latency at some cost in ratio, and is a natural next step.

Recommendation. For a streaming sensor-edge compressor, DRHE is the better choice: it is lossless, compresses by 4.2 with a matched dictionary, keeps up with the data with a wide margin, needs no frame buffer, and adds microseconds rather than milliseconds of latency. LPC compresses about 4 % better, but only by waiting for and storing every frame.

10.2 Scaling to larger frames

The hardware path compresses four of the cascade's sixteen receive channels, because four complex 16-bit samples fill the 128-bit stream. The following estimates scale the measured figures linearly; they are not measurements.

- **All 16 receive channels** quadruple the data per frame. DRHE would take about 2.5 ms per frame and LPC about 13 ms, both comfortably within the 200 ms frame period of the ColoRadar capture, but LPC would need a 12 Mibit frame buffer and its latency would grow to about 12 ms.
- **Kiem's use case** — 1024 ramps of 1024 samples, four channels, a 50 ms cycle and a 15 ms compression budget — needs 2.4 Gbit/s at the ADC. After the range FFT each frame is 8.4 MB of complex 16-bit data (512 bins of 16-bit I and Q per ramp and channel), delivered in the 21 ms of acquisition: about 3.2 Gbit/s at the compressor's input. DRHE's measured 5.1 Gbit/s keeps up without any buffer. LPC as built would take about 69 ms per frame, longer than the whole cycle, and would need an 8.4 MB frame buffer against roughly 4.4 MB of block and UltraRAM on this device.
- **The decoders** scale with the number of values, at about one per clock for both. At 16 receive channels each would take about 8 ms per frame. A frame of Kiem's size holds 21 times as many values as the frames measured here, so either decoder would take about 42 ms, inside the 50 ms cycle but with little margin. A decoder sits at the central processor, where several copies or a wider design are easier to afford than at the sensor.

DRHE scales with the data rate; LPC scales with the frame size, and on large frames it runs out of both time and memory. Its time per frame could be cut by about a third by overlapping its passes and amortising the per-bin pipeline fill (Section 6.3), and further only by processing several bins in parallel, but its buffer and its latency would remain.

10.3 What the compression ratio measures

Figure 10.1 follows one value through the compressor, in bits. The 16-bit container starts with 16. Because the FX16 scale is fixed to a theoretical full-scale signal and the ColoRadar capture sits far below it, the largest code over 50 frames is 437, so only $\log_2 437 + 1 = 9.77$ bits ever carry information; the other 6.2 are copies of the sign bit. Removing them is a factor of 1.64, and any entropy coder does it whether or not anything is predicted. The coder then exploits the sparsity of a range-FFT frame — most bins hold noise near zero and a few hold targets — for a further factor of 2.12, reaching 4.60 bits per value. Only then does prediction act, taking DRHE to 4.27 bits (a factor of 1.08), and the re-derived dictionary takes it to 3.79 (1.12).

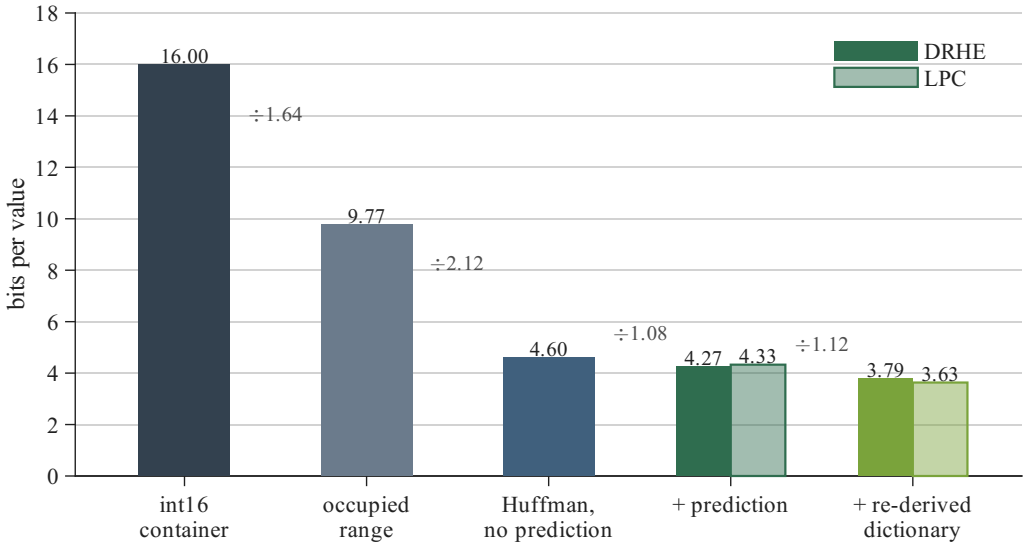


Figure 10.1: Bits per value through the compressor, 50 frames, four receive channels. Most of the reduction comes from the unused top of the 16-bit container and from the sparsity of the data, before any prediction. Factors are for DRHE; LPC is shown hatched.

Two consequences follow.

The ratios are partly a property of the capture. A sensor whose data filled the 16-bit container would have no unused top bits to remove. Its extra bits would instead hold finer detail of the receiver noise, which a lossless coder must keep, and the quantisation study of Section 7.1.1 shows what that does: scaled to fill the container, the same frame compresses by only 1.2. Ratios measured on different sensors are therefore not directly comparable unless their gain staging is. A no-prediction baseline should be reported alongside any radar compression ratio; it costs a few lines of MATLAB and separates the algorithm from the sensor.

The unused bits are a design choice. The same unused bits are why FX16 loses 3.77 % of targets: filling the container removes that loss and, with it, most of the compression (Section 7.1.1). How many bits of the noise floor are worth transmitting is a decision for the radar system designer; a lossless compressor does not change that trade, it only makes whatever choice is made cheaper to carry.

10.4 How far the dictionary and the lag generalise

The re-derived dictionaries were trained on five frames of one recording and tested on 45 later frames of the same recording, where they hold within 1 %. That shows they do not overfit the training frames; it does not show they transfer to other scenes, sensors or gain settings. Because a dictionary is a sixteen-entry table, a production design could hold several and choose per frame, or update one slowly from the statistics of past frames, without changing anything else in the hardware.

The prediction lag is specific in its value and general in its principle. It is $n_{\mathrm{Tx}} = 12$ because this radar rotates twelve transmitters in a fixed order; a radar that changed its MIMO schedule would need the lag reloaded, and one using Doppler- or code-division multiplexing has no interleaved ramp axis. The principle — a slow-time predictor must predict from the same antenna — applies to any slow-time coder carried over from a single-transmitter design to a time-multiplexed one, and the check is cheap: if the predictor does not beat the no-prediction baseline, it is predicting along the wrong axis.

10.5 Against the baseline

Kiem's DRHE carries over to a twelve-transmitter cascade unchanged except for the prediction lag. With his dictionary it compresses ColoRadar by 3.75 in hardware against his 3.17, and with a matched dictionary by 4.22, but these are different sensors and scenes, so the comparison is not like for like (Chapter 9). In hardware this thesis adds what his system did not have: a decoder for every design, and byte-for-byte agreement between the board and simulation. Every hardware difference from his design has an identified cause (Chapter 8); the largest is the floating-point predictor, which buys only 0.07 % of ratio over fixed point.

10.6 Observations on verifying a compressor

Real data and adversarial tests find different problems. Fifty real frames exercise the typical case exhaustively and the extreme case not at all: ColoRadar's prediction errors never leave category 10, so a third of the coder never runs. The eight-case synthetic suite exercises that third, and it found both the saturation of Section 6.4 and the missing end-of-frame word of Section 6.1. It says nothing, however, about whether a predictor helps on real data.

Lossless is not the same as useful. A compressor can be bit-exact at every level and compress by a plausible ratio while its predictor contributes nothing, as lag-1 prediction on this radar shows (Section 7.2). Only the no-prediction baseline detects it.

10.7 Threats to validity

- **The ratios are capture-specific.** Much of the ratio comes from an under-filled container (Section 10.3), a property of this recording's gain staging.
- **One recording.** All results use 50 consecutive frames of one ColoRadar sequence. Scenes with more or faster targets would give larger prediction errors and lower ratios, and the re-derived dictionaries were trained and tested on the same recording.
- **The detection metrics measure the quantisation step.** The three lossless schemes share FX16's detection metrics by construction. The detection chain is also simpler than a production one: its Doppler FFT runs across all 192 ramps in recorded order, it has no angle FFT, detections are matched cell by cell with no positional tolerance, and the threshold is not swept. The absolute rates of the lossy schemes should therefore be read as relative.
- **The method study uses MATLAB's range FFT.** Its FX16 codes come from a double-precision FFT. The range FFT on the FPGA gives the same codes except for 0.016 % that differ by one step, which changes the ratios in the fifth significant figure and the detection totals by three of 11,148 (Section 7.9).
- **BAQ is a block-floating-point stand-in.** A better-tuned, statistics-driven BAQ could lose fewer detections at the same ratio; it would still be lossy.
- **Channel count differs between stages.** Stage 1 uses all 16 receive channels; Stage 2

and the hardware use four. The two stages' ratios are close (3.75 and 3.751 for DRHE) but not identical by construction.

- **Power was not measured**, so the energy cost of compression is not reported.
- **Co-simulation covers one frame.** The 50-frame results are from C-simulation and the board, which exercise identical logic; the cycle-accurate RTL simulation was run on one frame.
- **The baseline was rebuilt from its description.** Kiem's source code and recordings are not public. The replica of his hardware and his simulation follow every type, function and parameter his thesis states; the parts it does not state are this thesis's reading of it.

Chapter 11

Conclusions and Future Work

11.1 Summary

This thesis set out to compare data compression methods for SoC-based radar on real, public, multi-transmitter data, to select the best lossless ones, and to implement their encoders and decoders on an FPGA and measure them on a board. The four objectives of Section 1.4 were met.

Eight methods were compared in MATLAB on 50 frames of ColoRadar data from a twelve-transmitter, sixteen-receiver Texas Instruments cascade. Two lossless predictive coders — Doppler Redundancy Hybrid Encoding and a second-order linear predictive coder — were specified, implemented in C++ for Vitis HLS with matching decompressors, and built with Vivado into a test system on an AMD KCU116 board, with a range-FFT front end so that the board can start from raw ADC samples. They were verified bit for bit from MATLAB through C-simulation and RTL co-simulation to the board, where the compressors' output matched the simulation byte for byte and the hardware decoders reconstructed every frame exactly. A third, independent implementation in Python reproduced their bit counts.

11.2 Findings

1. **Only lossless predictive coding meets the requirement.** DRHE (3.75) and LPC (3.70) are the only methods that compress by more than three without adding a missed target. The lossy quantisers reach up to 3.76 but miss 10 % to 29 % of targets, and floating-point storage compresses by only 1.04. All detection loss in the lossless chain comes from the

16-bit quantisation step, not from compression.

2. **On a TDM-MIMO radar, predict from the same transmitter.** Slow-time prediction from the adjacent ramp, which on this radar is another transmitter, compresses worse than no prediction at all. Predicting $n_{\mathrm{Tx}} = 12$ ramps back, from the same antenna, makes prediction worth 6 % to 8 %. For DRHE the change is an index into the state and nothing else.
3. **Most of the ratio comes before prediction.** Huffman coding alone compresses this data by 3.48, mainly because the fixed 16-bit scaling leaves six bits of every value unused and because a range-FFT frame is sparse. Filling those bits removes the quantisation error but also almost all of the compression. A no-prediction baseline should be reported with any radar compression ratio.
4. **The dictionary matters more than the predictor.** A Huffman dictionary re-derived from five frames for each algorithm's prediction errors raises DRHE from 3.75 to 4.22 and LPC from 3.70 to 4.40 on the board, holds on the 45 frames it was not trained on, changes only the contents of a table in hardware, and puts LPC ahead of DRHE on ratio.
5. **The hardware is exact and fast.** On the KCU116 at 100 MHz, both compressors produced output byte-identical to the simulation on all 50 frames, and the board's own decoders reconstructed every frame exactly. DRHE compresses a frame in 0.61 ms (5.1 Gbit/s) with 13 µs of latency; LPC in 3.25 ms with 3.0 ms of latency, because it must see the whole frame first. Both decoders reconstruct one value per clock, about 2 ms per frame (1.97 ms and 1.97 ms). Encoder and decoder together use about 23 % of the chip's logic for either algorithm. With the range FFT also on the FPGA the codes differ from MATLAB's in 0.016 % of values, by one step, and DRHE still takes in 8.3 Gbit/s of raw ADC data.
6. **DRHE is the better choice for a streaming sensor.** The two algorithms are close on ratio and level on logic, but DRHE needs no frame buffer, streams its output, and scales with the data rate, while LPC's buffer and latency grow with the frame until, at the frame size of Kiem's use case, it could not keep up.
7. **The baseline carries over, with one change.** Kiem's DRHE is reproduced bit for bit and works on a cascade once the prediction lag is set to the transmitter count; with a matched dictionary it compresses public cascade data by 4.2. Unlike his system it has a hardware decoder, and the board's compressed output reconstructs exactly on the board.

11.3 Future work

Measure power. Measure the board's power draw for each build, to give the energy cost of compression and decompression.

Move DRHE to fixed-point arithmetic. The floating-point datapath is the most expensive design decision and buys 0.07 % of ratio. Kiem's fixed-point types would remove most of its logic, DSPs and pipeline depth while remaining exactly lossless, since the closed loop makes the decoder follow whatever arithmetic the encoder uses. With fixed point, reading the state as zero on the first ramps instead of writing it would also reach $\mathrm{II} = 1$ almost for free.

A streaming LPC. A backward-adaptive LPC that fits its coefficients to previous frames would need no frame buffer and add no frame of latency, combining LPC's better prediction with DRHE's streaming. For the forward LPC, overlapping the three passes and amortising the per-bin pipeline fill would cut its time per frame by about a third, and processing several bins in parallel by more.

Dictionaries that adapt. Train dictionaries on more varied scenes, or let the hardware hold several and choose per frame, or update one slowly from past frames. A seventeenth category for the value -32768 would remove the need for saturation.

All sixteen channels. Widen the stream or time-multiplex the cores, and the range-FFT front end with them, to compress all sixteen receive channels.

The quantisation trade-off. Study adaptive or companded scaling as a deliberate design choice: how many bits of the noise floor are worth keeping for the detection margin they buy, and what that costs in link bandwidth.

Other multiplexing schemes and methods. Doppler- and code-division MIMO distribute the slow-time structure differently, and whether slow-time prediction helps on them is open. Learned and spectral methods such as AdaRadar [24] define the upper end of achievable ratios and could be compared against the lossless coders on the same data.

Bibliography

[1] C. Waldschmidt, J. Hasch and W. Menzel, "Automotive radar — from first efforts to future systems," *IEEE Journal of Microwaves*, vol. 1, no. 1, pp. 135–148, 2021.

[2] S. Yao *et al.*, "Exploring radar data representations in autonomous driving: a comprehensive review," arXiv preprint, 2023.

[3] M. I. Skolnik, *Radar Handbook*, 3rd ed. New York, NY: McGraw-Hill, 2008.

[4] V. Bandur *et al.*, "Making the case for centralized automotive E/E architectures," *IEEE Transactions on Vehicular Technology*, vol. 70, no. 2, pp. 1230–1245, 2021.

[5] Z. Liu, W. Zhang and F. Zhao, "Impact, challenges and prospect of software-defined vehicles," *Automotive Innovation*, vol. 5, no. 2, pp. 180–194, 2022.

[6] I. Barabás *et al.*, "Rethinking vehicle architecture through softwarization and servitization," *IEEE Access*, vol. 12, pp. 110782–110795, 2024.

[7] Texas Instruments, "FPD-Link III serializer/deserializer (SerDes) for sensor applications," Application Note SNLA344, 2021.

[8] Analog Devices, "GMSL SerDes technology for automotive applications," White Paper, 2022.

[9] K. Matheus and T. Königseder, *Automotive Ethernet*. Cambridge, UK: Cambridge University Press, 2021.

[10] IEEE, "IEEE P802.3bp 1000BASE-T1 PHY task force," 2016.

[11] L. Kiem, "Design and implementation of data compression methods for SoC-based radar systems," Master's thesis, Institute of Technical Informatics, Graz University of Technology, Graz, Austria, May 2025.

[12] A. Kramer, K. Harlow, C. Williams and C. Heckman, “ColoRadar: the direct 3D millimeter wave radar dataset,” *The International Journal of Robotics Research*, vol. 41, no. 4, pp. 351–360, 2022.

[13] Texas Instruments, *AWR2243 Single-Chip 76- to 81-GHz FMCW Transceiver Datasheet* (Rev. D), February 2024.

[14] K. Sayood, *Introduction to Data Compression*, 5th ed. Morgan Kaufmann, 2017.

[15] D. A. Huffman, “A method for the construction of minimum-redundancy codes,” *Proceedings of the IRE*, vol. 40, no. 9, pp. 1098–1101, 1952.

[16] B. Parhami, *Computer Arithmetic: Algorithms and Hardware Designs*, 2nd ed. Oxford University Press, 2010.

[17] IEEE, “IEEE standard for floating-point arithmetic,” IEEE Std 754-2019, 2019.

[18] B. Widrow and I. Kollár, *Quantization Noise*. Cambridge University Press, 2008.

[19] I. Kuon, R. Tessier and J. Rose, “FPGA architecture: survey and challenges,” *Foundations and Trends in Electronic Design Automation*, vol. 2, no. 2, pp. 135–253, 2008.

[20] A. Mani and Z. Yang, “Memory compression and decompression engine for TI mmWave radar,” Technical Report, Texas Instruments, 2019.

[21] NXP Semiconductors, “S32R RADAR signal compression,” Application Note AN5375, 2021.

[22] R. Kwok and W. T. K. Johnson, “Block adaptive quantization of Magellan SAR data,” *IEEE Transactions on Geoscience and Remote Sensing*, vol. 27, no. 4, pp. 375–383, July 1989.

[23] F. Cheng *et al.*, “Autoencoder-based feature compression for radar and vision sensor fusion in autonomous driving,” *IEEE Transactions on Intelligent Transportation Systems*, vol. 23, no. 8, pp. 12345–12355, 2021.

[24] J. Park, S. Y. Chun and M. Seok, “AdaRadar: rate adaptive spectral compression for radar-based perception,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2026.

[25] Z. Zivkovic and L. Li, "On-the-fly range-velocity radar data compression exploring Doppler spectrum redundancy," in *Proceedings of the 16th International Radar Symposium (IRS)*, Dresden, Germany, 2015, pp. 321–326.

[26] L. Li, "Embedded data compression in automotive FMCW radar system," MSc thesis, University of Twente, Enschede, The Netherlands, 2014.

[27] G. Meucci and F. Mancuso, "FMCW radar data compression," project report, Data Compression course, PhD programme in Information Engineering, University of Pisa, 2022. [Online]. Available: https://github.com/mandugo/FMCW-Radar-Data-Compression

[28] J. Deng and L. Huang, "Synthetic aperture radar image compression based on low-frequency rejection and quality map guidance," *Remote Sensing*, vol. 16, no. 5, art. 891, 2024, doi:10.3390/rs16050891.

Appendix A

The Huffman Dictionaries

Table A.1 gives the complete original dictionary [11, Table A.1], used in Stage 1, in the no-prediction baseline and in the original-dictionary board builds. The published codes are most-significant-bit first; the implementation packs least-significant-bit first, so the stored codewords are bit-reversed. Both forms are listed, and all sixteen entries were verified to agree with the published table.

The reversed code set remains prefix-free, which is what allows the decoder to match candidates from the low end of its bit buffer with no ambiguity.

Table A.1: The original Huffman dictionary. "Cost" is the total bits to code one residual in that category, $\ell + S_4$, from Equation (4.4).

S_4	$\lvert v \rvert$ range	ℓ	code (MSB-first)	stored (LSB-first)	cost
0	$v = 0$	4	`1010`	`0x0005`	4
1	1	3	`100`	`0x0001`	4
2	2–3	2	`01`	`0x0002`	4
3	4–7	2	`00`	`0x0000`	5
4	8–15	2	`11`	`0x0003`	6
5	16–31	5	`10110`	`0x000D`	10
6	32–63	6	`101110`	`0x001D`	12
7	64–127	7	`1011110`	`0x003D`	14
8	128–255	8	`10111110`	`0x007D`	16
9	256–511	9	`101111110`	`0x00FD`	18
10	512–1023	10	`1011111110`	`0x01FD`	20
11	1024–2047	11	`10111111110`	`0x03FD`	22
12	2048–4095	14	`10111111111111`	`0x3FFD`	26
13	4096–8191	14	`10111111111110`	`0x1FFD`	27
14	8192–16383	13	`1011111111110`	`0x0FFD`	27
15	16384–32767	12	`101111111110`	`0x07FD`	27

Note the non-monotonicity at categories 12–15: the longest codewords are assigned to categories 12 and 13 rather than to 15. This is a property of the published dictionary and is reproduced faithfully. It has no effect on this dataset, where no residual exceeds category 10.

The re-derived dictionaries

Table A.2 gives the codeword lengths of the dictionaries re-derived for each algorithm (Section 4.6), built from the S_4 counts of frames 0–4 with one added to every category, and the total cost of each category with each. The codewords themselves are the canonical Huffman codes for these lengths, bit-reversed for the packer; they are the tables `DRHE_TDM_HUFFMAN_TABLE` and `LPC_TDM_HUFFMAN_TABLE` in the HLS source.

Table A.2: Codeword length ℓ and total cost $\ell + S_4$ per category for the original and the two re-derived dictionaries.

	original		DRHE		LPC	
S_4	ℓ	cost	ℓ	cost	ℓ	cost
0	4	4	3	3	3	3
1	3	4	2	3	1	2
2	2	4	1	3	2	4
3	2	5	4	7	4	7
4	2	6	5	9	5	9
5	5	10	6	11	6	11
6	6	12	7	13	7	13
7	7	14	8	15	8	15
8	8	16	9	17	9	17
9	9	18	10	19	10	19
10	10	20	11	21	13	23
11	11	22	14	25	13	24
12	14	26	14	26	13	25
13	14	27	13	26	13	26
14	13	27	13	27	12	26
15	12	27	13	28	12	27

The re-derived dictionaries code their commonest category in three bits (DRHE, category 2) or two (LPC, category 1), where the original cannot code anything in fewer than four. They are longer above category 3, which costs little because those categories are rare after prediction.

Appendix B

Bitstream Layouts

B.1 DRHE

DRHE's bitstream is a single continuous sequence of coded residuals, in the raster order the samples arrive:

$$\underbrace{\mathrm{enc}(d_{\Re})\ \mathrm{enc}(d_{\Im})}_{\text{ramp 0, bin 0, ch 0}}\ \underbrace{\mathrm{enc}(d_{\Re})\ \mathrm{enc}(d_{\Im})}_{\text{ramp 0, bin 0, ch 1}}\ \cdots$$

where $\mathrm{enc}(v)$ is the Huffman codeword for $S_4(v)$ followed by $S_4(v)$ APPEND bits. There is no header: the frame dimensions arrive over the AXI4-Lite control interface. The final AXI word is zero-padded.

B.2 LPC

LPC's bitstream has two sections:

$$\underbrace{\left[a_1^{\Re}\ a_2^{\Re}\ a_1^{\Im}\ a_2^{\Im}\right]_{\text{ch,bin}}}_{\text{coefficient table, } 4\times16 \text{ bits each}}\qquad\underbrace{\mathrm{enc}(d_{\Re})\ \mathrm{enc}(d_{\Im})\ \cdots}_{\text{residuals, raster order}}$$

The coefficient table comes first because the decoder needs all of it before it can reconstruct anything. The residuals follow in the original raster order — not in the order the compressor computed them — which is what allows the decompressor to operate without a frame buffer (Section 6.3).

Appendix C

Reproducing the Results

All sources are in the thesis repository under `ThesisA/`. The build tree must sit on a path without spaces, which Vitis rejects.

C.1 Source files

File	Purpose
MATLAB (`Matlab_Sim/`)	
`loadColoRadarFrame.m`	reads a frame and applies the TDM flattening
`preprocessing.m`, `rangeFFT.m`	DC removal, window, range FFT
`compress_fx16.m`	FX16 quantisation, Equation (2.11)
`compress_drhe.m`, `decompress_drhe.m`	DRHE; lag from `config.predLag`, default n_{Tx}
`compress_lpc_huffman.m`, `decompress_lpc_huffman.m`	LPC at lags L, $2L$
`huffman_tables.m`, `huffman_code_lengths.m`	the original and re-derived dictionaries; retraining
`run_coloradar_benchmark.m`	Stage 1: all eight methods with detection metrics
`make_poster_figs.m`	Stage 2 breakdown, dictionary tables, and the figures of Chapter 7; parameters at the top
`export_frames_for_vitis.m`	writes the `int16` stimulus for the HLS testbenches
HLS (`hls_component/`)	
`drhe_tdm_common.h`, `drhe_tdm_compress.cpp`, `drhe_tdm_decompress.cpp`	DRHE cores and the DRHE dictionary
`lpc_tdm_common.h`, `lpc_tdm_compress.cpp`, `lpc_tdm_decompress.cpp`	LPC cores, Yule–Walker solver and the LPC dictionary

`*_tb.cpp`, `*_tb_edge.cpp`	50-frame and edge-case testbenches
Board (`fpga_kcu116/`)	
`hls/export_ips.bat`	C-simulation (writes the golden streams) and IP packaging
`vivado/build.bat`	block design, synthesis, implementation, bitstream; `decomp 1` adds the decompressor
`sw/src/main.c`, `board_map.h`	MicroBlaze firmware
`host/run_board.bat`, `run_board.tcl`	program the board, load frames, run, read results
`host/pc_decompress.bat`	PC decompression with the HLS decoder source
`host/compare_results.py`	byte comparison with C-simulation, reconstruction check, tables
Independent check (`verify_indep/`, Python)	
`indep.py`	the whole chain from raw frame to bit count
`retrain_complete.py`	dictionaries trained on frames 0–4, tested on 5–49
`golden_check.py`, `independent_decode.py`	residual-level check and decoding of the hardware bitstreams

C.2 Commands

C-simulation, synthesis and co-simulation of a core (DRHE shown):

```
vitis-run --mode hls --csim  --config test_tdm.cfg --work_dir hls_tdm
v++ -c    --mode hls         --config test_tdm.cfg --work_dir hls_tdm
vitis-run --mode hls --cosim --config test_tdm.cfg --work_dir hls_tdm
```

Board run, from `fpga_kcu116/host` (`<tag>` is `drhe`, `lpc`, `drhe_full` or `lpc_full`):

```
run_board.bat <tag> 50
pc_decompress.bat <tag>          (compress-only builds)
python compare_results.py <tag>
```

Figures and tables of Chapter 7, in MATLAB from `Matlab_Sim`: `make_poster_figs`. Simulation results are cached; set `P.recompute = true` after changing the algorithm code.

Appendix D

Verification Record

Table D.1 lists every check on which the results rest, where it was made, and its outcome.

Table D.1: The verification record.

Check	Where	Outcome
Stage 1, eight methods, 50 frames	MATLAB	Table 7.1; re-run reproduces every cell
DRHE and LPC lossless, original dictionary	HLS C-sim, 50 frames	MaxDiff 0 on every frame
DRHE and LPC lossless, re-derived dictionaries	HLS C-sim, 50 frames	MaxDiff 0; CR 4.21704, 4.40345
MATLAB against HLS bit counts	both	equal up to the last 256-bit padding word
Independent bit counts, original dictionary	Python, 50 frames	equal to HLS on every frame
Independent decode, re-derived dictionaries	Python	no residual mismatch
Edge cases, eight synthetic inputs	HLS C-sim	all pass within their allowed MaxDiff
RTL behaviour	co-simulation, 1 frame	MaxDiff 0, both algorithms
Timing closure, four builds	Vivado	all met at 100 MHz (Table 6.1)
Compressed bytes against C-sim, original dictionary	KCU116, 50 frames	byte-identical, both algorithms
Decompression of board output	PC, 50 frames	MaxDiff 0, both algorithms
Compressed bytes against C-sim, re-derived dictionaries	KCU116, 50 frames	byte-identical; CR 4.21704, 4.40345
On-chip decompression (`_full`)	KCU116, 50 frames	MaxDiff 0, both algorithms; 1.97 ms and 1.97 ms per frame

Check	Where	Outcome
Parallel LPC decoder against the first one	C-sim, 50 frames + 8 edge cases	bit-identical output
Frames ending on a word boundary	C-sim, 8 edge cases, both algorithms	TLAST always sent; ColoRadar output unchanged
DRHE matches Kiem's specification	rebuilt simulation	bit-identical output